RK3576 端侧多模态系统集成与性能优化工程手册

本手册面向 Linux C++与嵌入式 AI 工程学习、复现和项目验收，覆盖 RK3576 上的 Camera、ASR、Qwen、SmolVLM2、Kokoro、异步任务、性能测量、Qt 与稳定性。5.1～5.4 已经用户验收；5.5 的最终审核以本节验收清单及真实日志为准。隔离版本没有覆盖主工程。

当前为编写中的完整工作稿。5.5 尚在执行，缺失项不作为通过；交付时将根据最终验证重新生成。

1 工程环境与证据规则 7

2 开发时间线与最终系统关系 8

3 第五阶段 5.1 生命周期审计和安全退出 11

完整关键源码 include/app/stop_signals.h 12

完整关键源码 src/ai/rkllm_backend.cpp struct RunContext 13

完整关键源码 src/ai/rkllm_backend.cpp struct RKLLMBackend::Impl 13

完整关键源码 src/ai/rk3576_vlm_backend.cpp struct RknnOutputOwner 13

完整关键源码 src/audio/asr_pipeline.cpp void ASRWorker::run() 14

完整关键源码 src/audio/asr_pipeline.cpp void ASRWorker::runImpl() 14

本节面试题与完整答案 18

1. 为什么只使用智能指针还会出现生命周期错误？ 18

2. 为什么析构里的 join 可能一直等待？ 18

3. 为什么不把 ASR、LLM、TTS 一起 requestStop？ 18

4. shared_ptr<const ImageFrame> 是否是零拷贝？ 18

5. 两个 Engine 都有 mutex，能否证明 NPU 没有资源冲突？ 18

6. 有界队列容量 2 为什么可能存在 3 个任务？ 19

7. 如何避免 C SDK buffer 在 C++ 异常时泄漏？ 19

8. Windows 6/6 CTest 能证明什么？ 19

9 为什么不能在 signal handler 中调用条件变量 19

10 析构顺序为什么是并发接口的一部分 19

4 第五阶段 5.2 有界异步任务和 NPU 调度 19
5.2 多线程与异步 Pipeline 工程记录 19
操作 1：继承 5.1 验收与授权边界 20
操作 2：源码定位与最小方案 20
操作 3：真实线程与队列架构 21
操作 4：有界性、背压与终态 22
操作 5：生命周期、竞态与退出审查 22
操作 6：回声与 TTS 调度取舍 23
操作 7：构建问题与证据分类 23
操作 8：适用范围补充 23
完整关键源码 src/ai/llm_worker.cpp 24
完整关键源码 include/app/pipeline_admission.h 26
完整关键源码 src/audio/tts_worker.cpp void TTSWorker::run() 27
操作 1：公共类型与 C++ 语法 30
操作 2：PipelineAdmission 逐句所有权解释 31
操作 3：LLMWorker 的构造、提交与线程语句 32
操作 4：推理 run 循环的每个执行阶段 33
操作 5：TTSWorker 及不变的设备路径 34
操作 6：完整语音入口的语句与捕获 35
操作 7：测试语句及可证明范围 36
操作 8：面试题与参考答案 36
核心实测取舍 37
5 第五阶段 5.3 Camera 内存和预处理 37
完整关键源码 include/video/owned_frame_pool.h 38
完整关键源码 src/video/latest_frame_buffer.cpp 39
完整关键源码 src/video/image_preprocessor.cpp 39
5.3 关键 C++ 逐句解释与审查 41

OwnedFramePool：有界所有权，不能用 use_count 猜测独占.....42
VideoCapture：减少分配，保留 QBUF 前的所有权边界.....44
CameraService、LatestFrameBuffer 与退出.....45
Rgb512Preprocessor：算术保持一致，缓存坐标.....46
RK3576VLMBackend：输入与输出寿命.....48
埋点、诊断与测试的界限.....48
与实现对应的测试.....49
5.3 面试题与完整参考答案.....50
1. mmap 摄像头是否就是零拷贝？.....50
2. 为什么不直接把 V4L2 指针交给异步 VLM？.....50
3. shared_ptr 的 use_count()==1 能证明可以覆盖吗？.....50
4. 有界三帧池如何避免读写竞态？.....50
5. 为什么 shared_ptr 自定义 deleter 要在 mutex 外构造？.....51
6. Camera.stop 后，正在读旧帧的线程怎么办？.....51
7. stride 与 width 为什么不能互换？.....51
8. RGB 复用为什么必须考虑 Engine 锁与 SDK？.....51
9. NV12 转 RGB 优化后尺寸相同，为何还要逐像素比较？.....51
10. 如何判断缓存坐标是有效优化？.....52
11. DMA-BUF、物理连续内存和缓存一致性是什么关系？.....52
12. RGA 校验和 import 成功，为什么运行仍失败？.....52
13. 为什么 RSS 上升不一定是泄漏，但也不能忽略？.....52
14. 如何保证统计不“美化”结果？.....52
15. drain 与 cancel_pending 的安全退出区别是什么？.....53
6 第五阶段 5.4 统一 Profiling 和 TTS 优化.....53
完整关键源码 include/common/request_trace.h.....53
完整关键源码 include/common/image_profile.h.....54
完整关键源码 src/audio/tts_service.cpp bool TTSService::synthesize(.....56

完整关键源码 src/audio/alsa_playback_service.cpp bool AlsaPlaybackService::play(...58
5.4 关键源码逐句说明与生命周期审查.....62
操作 1：统一时钟、日志与请求所有权.....62
操作 2：沿已有队列传递上下文.....63
操作 3：VAD 时间的真实意义.....64
操作 4：RKLLM callback 和 VLM 状态.....64
操作 5：TTS 和 ALSA 的测量边界.....65
操作 6：固定输入、回放与信号等待.....65
操作 7：验证如何覆盖这些语句.....66
操作 8：手动 benchmark、CMake 与分析脚本.....67
操作 9：最终音频回放错误检查的最小修正.....68
5.4 性能工程面试题与参考答案.....70
操作 1：为什么不能使用系统时间计算请求延迟？.....70
操作 2：为什么需要 Request ID，frame_id 为什么不能代替？.....70
操作 3：TLS 为什么不能自动解决 SDK 回调关联？.....70
操作 4：TTFT 应该如何定义？.....71
操作 5：TTS 首批音频就绪，是否等于用户听到声音？.....71
操作 6：如何用 RTF 公平比较 TTS？.....71
操作 7：为什么线程数越多不一定越快？.....71
操作 8：如何定位 TTS 队列等待的根因？.....72
操作 9：如何区分冷启动、预热和稳态？.....72
操作 10：如何证明一次优化是实际计算收益？.....72
操作 11：取消等待与排空退出有什么不同？.....73
操作 12：长回放等待为什么会破坏信号回归？.....73
操作 13：为什么不能把 NPU sysfs 的 load 直接当利用率？.....73
操作 14：怎样评估资源泄漏与测试覆盖的限度？.....73
7 第五阶段 5.5 Qt 和持续稳定性.....74

5.5 Qt 集成和最终系统验收 74
操作 1 范围冻结与源码复核 74
操作 2 Qt 与运行环境只读预检 75
操作 3 分阶段热稳定性方案 75
操作 4 实施与验收映射 76
5.5 Qt 适配与稳定性工具逐句说明 77
BackendSession 的对象与启动语句 77
停止与窗口关闭 78
输入缓冲与界面内存 78
共用信号类与 Qt 入口 79
故障测试和长测工具 79
完整关键源码 include/gui/backend_session.h 80
完整关键源码 src/gui/backend_session.cpp 81
完整关键源码 include/gui/session_window.h 82
完整关键源码 src/gui/session_window.cpp 83
完整关键源码 src/gui/main.cpp 87
完整关键源码 src/audio/asr_llm_tts_smoke.cpp 88
5.5 Qt 与系统稳定性面试题及参考答案 112
1 为什么选择 QProcess 适配 CLI 而不是在窗口里重新创建模型 112
2 QProcess 的 started 是否意味着系统可接受语音 112
3 关闭 Qt 窗口时如何避免后台访问已销毁控件 112
4 为什么 Qt 中不能直接调用 waitForFinished 112
5 如何在 Qt 程序处理 SIGINT 而不在 handler 调用 Qt 113
6 2 线程与 4 线程的短时加速为什么不能决定最终默认值 113
7 如何区分 DVFS 和热降频 113
8 什么证据支持“没有发现明显内存泄漏” 113
9 为什么压力测试中的推理成功率和播放交付率要分开 113

10 固定音频回放能替代真实麦克风验收吗.....114
11 Fake 故障测试为什么仍然重要.....114
12 如何定义可复现的最终交付.....114
8 构建运行复现和重要命令.....114
9 最终验收边界和局限.....114
10 简历项目描述与面试讲述.....115
11 结果索引和源码一致性.....115

1 工程环境与证据规则

硬件为 LubanCat 3 RK3576，4 个 Cortex A53 和 4 个 Cortex A72，约 4GB 物理内存、当前无 Swap；Linux 6.1.99-rk3576、aarch64、Debian GCC12.2.0、CMake/CTest3.25.1。IMX415 通过 V4L2 `/dev/video11` 采集 3840×2160 NV12，stride3840、sizeimage12441600，四个 MMAP 驱动槽。HyperX Cloud III 每次动态发现，历史实际 ID 为 III，不能把历史数字声卡写死。

模块	真实模型或 Runtime	执行与限制
ASR	Silero VAD 与 SenseVoice INT8，sherpa onnx1.13.8	CPU；实时采集与预分段回放必须区分
TEXT	Qwen2.5 1.5B W4A16，RKLLM1.3.1	模型制品 toolkit1.2.3；RKNPU driver0.9.8
VISION	SmolVLM2 256M W4A16 加 scatterfix RKNN Vision	RGB512×512，64×576 浮点特征；语言制品 toolkit1.3.1
TTS	Kokoro multi lang v1_1，ORT1.28.2	CPU，sid17，speed1，silence_scale0.2，24kHz
Qt	Qt5.15.8 Widgets 与 Test	可选构建，现有 xcb 桌面，不安装新软件

原稳定目录是`/home/cat/projects/RK3576/rk3576_multimodal`，唯一授权写目录是其同级`rk3576_multimodal_stage5_review`。模型及 SDK 继续引用原路径，源码和构建使用新目录。根目录历史笔记和早期 architecture/run 文档不能替代源码或终端；没有 Git 仓库时使用真实 git diff no-index 并验证补丁，不虚构 commit。

```
#!/usr/bin/env bash
# Source this file. Historical paths from 4.md, verified on board before use.
STAGE5_BASE=/home/cat/projects/RK3576
STAGE5_ORIGINAL="$STAGE5_BASE/rk3576_multimodal"
STAGE5_REVIEW="$STAGE5_BASE/rk3576_multimodal_stage5_review"
STAGE5_ASR="$STAGE5_BASE/asr_runtime/sherpa-onnx-sense-voice-zh-en-ja-ko-yue-int8-2024-07-17"
STAGE5_VAD="$STAGE5_BASE/asr_runtime/vad/silero_vad.onnx"
STAGE5_QWEN="$STAGE5_BASE/rkllm_sdk/models/Qwen2.5-1.5B-Instruct_RK3576_w4a16.rkllm"
STAGE5_KOKORO="$STAGE5_BASE/tts_runtime/models/kokoro-multi-lang-v1_1"
STAGE5_VLM="$STAGE5_ORIGINAL/models/vlm/smolvlm2_256m_w4a16_rk3576.rkllm"
STAGE5_VISION="$STAGE5_ORIGINAL/models/vlm/smolvlm_vision_scatterfix_rk3576.rknn"
STAGE5_SHERPA="$STAGE5_BASE/asr_runtime/sherpa-onnx-v1.13.8-linux-aarch64-shared-cpu"
```

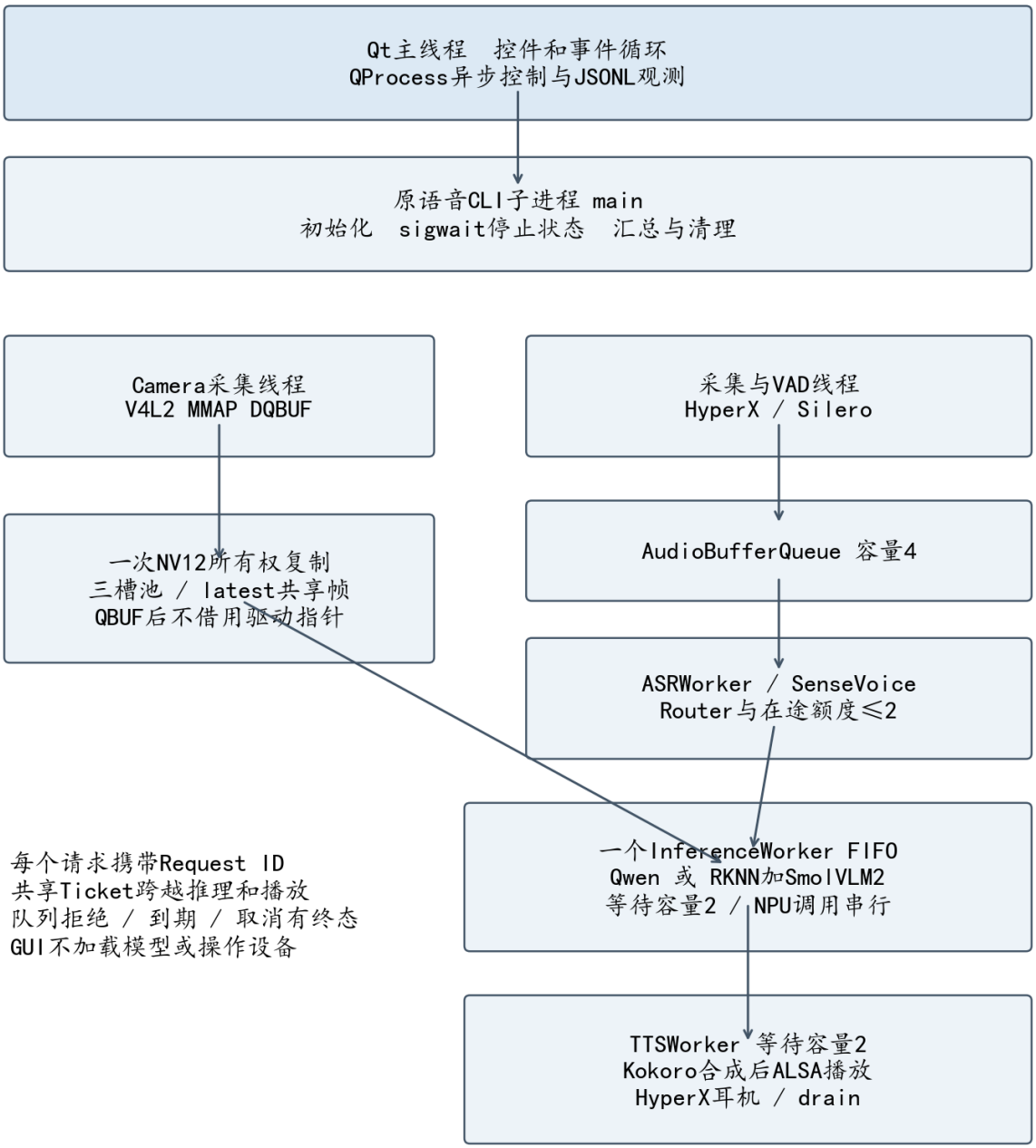
```
STAGE5_RKLLM="$STAGE5_BASE/rkllm_sdk"
STAGE5_RKNN_INCLUDE="$STAGE5_BASE/rknn_test/include"
STAGE5_RKNN_LIBRARY=/usr/lib/librknnrt.so
# Prefer the stable build's explicit SDK cache paths, never eval its contents.
```

路径变量只是环境引用，不复制模型。source 影响当前 shell，set -a 使变量导出到子进程；LD_LIBRARY_PATH 仅为本次进程指定已存在共享库，不修改系统缓存。每次实际参数、版本、声卡、PID、退出码、原工程指纹都与对应 case 共同保留。

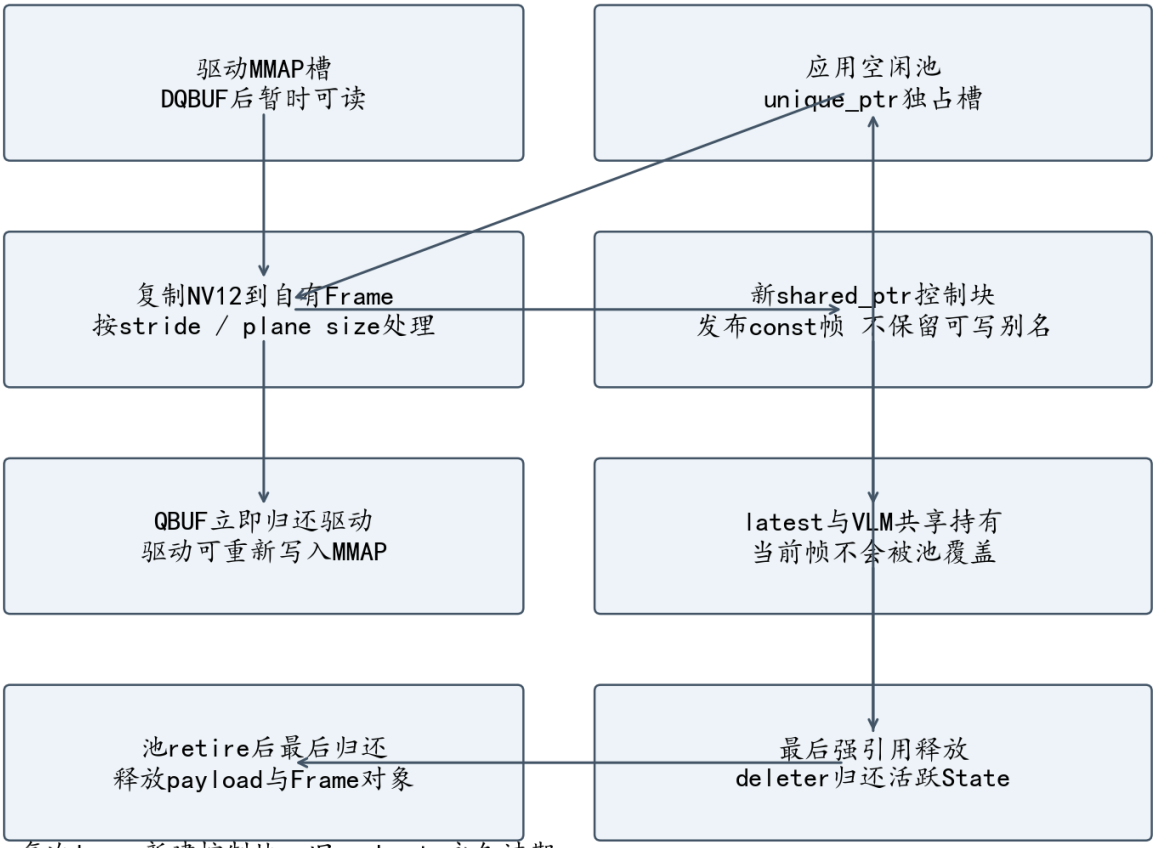
2 开发时间线与最终系统关系

阶段	时间依据	具体目标与结果
5.1	2026 年 10 月 17 日报告及 SSH 日志	生命周期审计、RKLLM 回调/输出释放、协作式 SIGINT 和 SIGTERM
5.2	2026 年 10 月 17 日报告及 ARM 日志	ASR 解耦、共同推理 FIFO、有界背压、取消与排空
5.3	2026 年 10 月 17 日报告及固定帧	三槽自有帧池、缓存预处理，RGA 失败保留
5.4	2026 年 10 月 17 日固定输入与听音	统一请求计时、TTS2/4 线程同条件 n30 比较
5.5	2026 年 10 月 17 日晚至 18 日实测	Qt 适配、热对照、最终源码长测、异常与完整交付

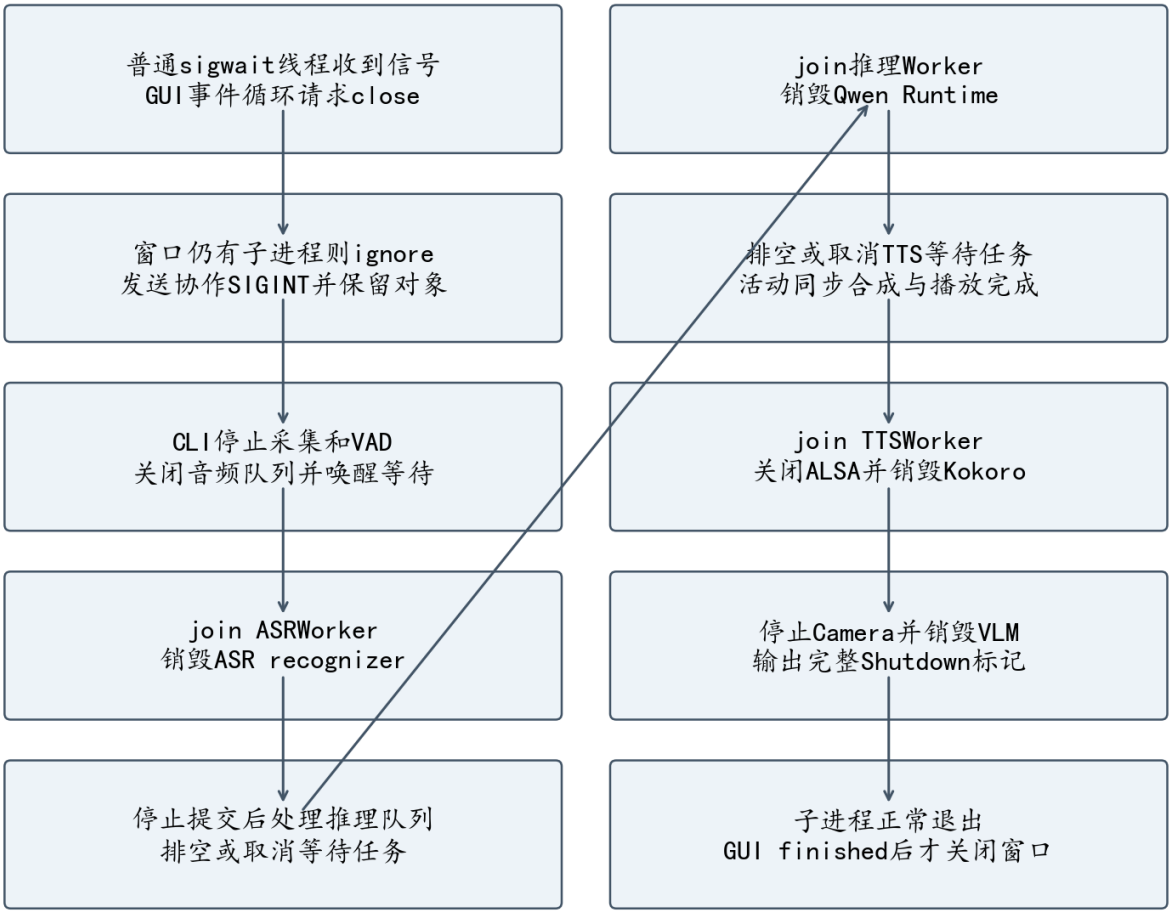
最终语音入口直接使用 IntentRouter，没有经过另一个 CLI 中的 RequestProcessor。Camera、采集/VAD、ASR、推理、TTS 和信号接收分别承担职责。TEXT 和 VISION 共用同一个推理消费者；ASR 只提交，不同步执行 VLM。TTS 仍先合成完整 PCM 再播放，下一条不能越过当前合成和 drain。本节没有引入新的 AI 框架或模型。


Qt主线程　控件和事件循环
QProcess异步控制与JSONL观测
原语音CLI子进程 main
初始化　sigwait停止状态　汇总与清理
Camera采集线程
V4L2 MMAP DQBUF
采集与VAD线程
HyperX / Silero
一次NV12所有权复制
三槽池 / latest共享帧
QBUF后不借用驱动指针
AudioBufferQueue 容量4
ASRWorker / SenseVoice
Router与在途额度≤2
每个请求携带Request ID
共享Ticket跨越推理和播放
队列拒绝 / 到期 / 取消有终态
GUI不加载模型或操作设备
一个InferenceWorker FIFO
Qwen 或 RKNN加SmolVLM2
等待容量2 / NPU调用串行
TTSWorker 等待容量2
Kokoro合成后ALSA播放
HyperX耳机 / drain

最终进程和线程架构



Camera 到 VLM 的所有权生命周期



安全退出和资源释放顺序

3 第五阶段 5.1 生命周期审计和安全退出

审计首先发现两个真实入口、Worker 对 Service 的借用、lambda 引用捕获以及 SDK 回调状态的寿命边界。join 只等待，不通知；开放空 ASR 队列会使析构永久等待。正确顺序是停止输入并关闭/唤醒队列，join 使用者，再销毁 Service 和 Runtime。5.1 原完整语音入口没有协作式信号处理，先记录真实缺陷，再最小修复，不能把 SIGKILL 叫安全退出。

RKLLM 的 Pimpl 保存参数和 callback，析构先 destroy handle，随后成员才析构；每次同步 run 的 RunContext 与用户 callback 状态持续到 run 返回。RKNN 输出只在 outputs_get 成功后归 owner，业务先复制 Embedding，再在正常/异常退栈路径 release。release 自身仍可能失败，当前记录错误；这不是 SDK 所有内部失败路径的证明。

完整关键源码 include/app/stop_signals.h

```
#pragma once
#include <atomic>
#include <cerrno>
#include <stdexcept>
#include <thread>
#include <pthread.h>
#include <signal.h>

// Construct before SDKs/workers: all descendant threads inherit the blocked set.
// sigtimedwait runs in an ordinary thread; there is no asynchronous signal handler.
class StopSignals {
public:
    StopSignals()
    {
        sigemptyset(&signals_);
        sigaddset(&signals_, SIGINT);
        sigaddset(&signals_, SIGTERM);
        if (pthread_sigmask(SIG_BLOCK, &signals_, &previous_) != 0)
            throw std::runtime_error("Cannot block stop signals");
        try
        {
            thread_ = std::thread([this] {
                const timespec interval{0, 100000000};
                while (!finished_.load())
                {
                    const int received = sigtimedwait(&signals_, nullptr, &interval);
                    if (received == SIGINT || received == SIGTERM)
                    {
                        int empty = 0;
                        signal_.compare_exchange_strong(empty, received);
                    }
                    else if (received < 0 && errno != EAGAIN && errno != EINTR)
                    {
                        wait_failed_.store(true);
                        break;
                    }
                }
            });
        }
        catch (...)
        {
            pthread_sigmask(SIG_SETMASK, &previous_, nullptr);
            throw;
        }
    }
    ~StopSignals()
    {
        finished_.store(true);
        if (thread_.joinable()) thread_.join();
        pthread_sigmask(SIG_SETMASK, &previous_, nullptr);
    }
    StopSignals(const StopSignals&) = delete;
    StopSignals& operator=(const StopSignals&) = delete;
    int received() const noexcept { return signal_.load(); }
    bool failed() const noexcept { return wait_failed_.load(); }
private:
    sigset_t signals_{};
    sigset_t previous_{};
    std::atomic<int> signal_{0};
```

```
    std::atomic<bool> finished_{false};
    std::atomic<bool> wait_failed_{false};
    std::thread thread_;
};
```

完整关键源码 src/ai/rkllm_backend.cpp struct RunContext

```
struct RunContext
{
    std::mutex mutex;
    std::string text;
    const LLMStreamCallback* on_chunk = nullptr;
    std::chrono::steady_clock::time_point start;
    double first_text_ms = -1.0;
    bool failed = false;
    RequestTracePtr trace;
};
```

完整关键源码 src/ai/rkllm_backend.cpp struct RKLLMBackend::Impl

```
struct RKLLMBackend::Impl
{
    std::string model_path;
    LLMHandle handle = nullptr;
    RKLLMParam params{};
    RKLLMCallback callbacks{};

    ~Impl() noexcept
    {
        if (handle != nullptr)
        {
            (void)rkllm_destroy(handle);
            handle = nullptr;
        }
    }
};
```

完整关键源码 src/ai/rk3576_vlm_backend.cpp struct RknnOutputOwner

```
struct RknnOutputOwner {
    rknn_context context;
    rknn_output output{};
    bool acquired = false;

    explicit RknnOutputOwner(rknn_context ctx) : context(ctx) {}
    RknnOutputOwner(const RknnOutputOwner&) = delete;
    RknnOutputOwner& operator=(const RknnOutputOwner&) = delete;
    ~RknnOutputOwner() noexcept {
        if (acquired) {
            auto begin=ImageProfile::Clock::now();
            const int code=rknn_outputs_release(context,1,&output);
            imageSample("rknn_outputs_release",ImageProfile::ms(begin));
            if(code)std::cerr<<"[RKNN release ERROR] ret="<<code<<'\n';
```

```
        }
    }
};
```

完整关键源码 src/audio/asr_pipeline.cpp void ASRWorker::run()

```
void ASRWorker::run()
{
    try
    {
        runImpl();
    }
    catch (const std::exception& e)
    {
        ok_ = false;
        error_ = std::string("ASRWorker exception: ") + e.what();
        queue_.close();
    }
    catch (...)
    {
        ok_ = false;
        error_ = "ASRWorker: unknown exception";
        queue_.close();
    }
}
```

完整关键源码 src/audio/asr_pipeline.cpp void ASRWorker::runImpl()

```
void ASRWorker::runImpl()
{
    AudioSegment segment;

    while (queue_.pop(segment))
    {
        using Clock = std::chrono::steady_clock;

        // 消费者真正从队列取出音频的时间。
        const auto popped_at = Clock::now();
        RequestTraceScope trace_scope(segment.trace);
        traceMark("asr_dequeued",popped_at);

        std::vector<float> samples;
        std::string error;

        if (!segment.samples.empty())
        {
            // VAD 已输出 Float32，直接移动。
            samples = std::move(segment.samples);
        }
        else
        {
            // 兼容原始 S16_LE 录音流水线。
            if (!AudioPreprocessor::convertS16ToFloat32(
                    segment.pcm,
                    16000,
                    1,
```

```
            samples,
            error))
    {
        error_ = "Preprocessing: " + error;
        queue_.close();
        return;
    }
}

if (samples.empty())
{
    error_ = "Empty ASR audio segment";
    queue_.close();
    return;
}

std::string text;
double latency_ms = 0.0;

// 推理开始时间。
const auto infer_start = Clock::now();
traceMark("asr_begin",infer_start);

if (!asr_.recognize(
        samples,
        text,
        latency_ms,
        error))
{
    traceMark("asr_failed");
    error_ = "ASR: " + error;
    queue_.close();
    return;
}

// 获取文字结果的时间。
const auto result_ready = Clock::now();
traceMark("asr_end",result_ready);

++processed_;

const double audio_duration_ms =
    1000.0 *
    static_cast<double>(samples.size()) /
    16000.0;

const double rtf =
    latency_ms / audio_duration_ms;

// 统一使用 steady_clock 时间差。
auto elapsed_ms = [](
    Clock::time_point first,
    Clock::time_point second)
{
    return std::chrono::duration<
        double, std::milli
    >(second - first).count();
};

{
    std::lock_guard<std::mutex> lock(
        print_mutex_
```

```
    );

    // 原始 ASR 输出格式保持不变。
    std::cout
        << "[ASR] Segment #"
        << segment.sequence
        << ": "
        << (text.empty() ? "<empty>" : text)
        << " | latency="
        << latency_ms
        << " ms"
        << " | RTF="
        << rtf
        << '\n';

    // 仅对来自 VAD 的有效时间戳计算。
    if (segment.timing_valid)
    {
        std::cout
            << "[Timing] Segment #"
            << segment.sequence

            << " endpoint_est_ms="
            << elapsed_ms(
                segment.audio_end_estimated_at,
                segment.vad_ready_at
            )

            << " vad_to_queue_ms="
            << elapsed_ms(
                segment.vad_ready_at,
                segment.enqueued_at
            )

            << " queue_wait_ms="
            << elapsed_ms(
                segment.enqueued_at,
                popped_at
            )

            << " preprocess_ms="
            << elapsed_ms(
                popped_at,
                infer_start
            )

            << " asr_wall_ms="
            << elapsed_ms(
                infer_start,
                result_ready
            )

            << " total_est_ms="
            << elapsed_ms(
                segment.audio_end_estimated_at,
                result_ready
            )

            << '\n';
    }
}
```

```
        // ASR 完成后调用业务回调；完整语音入口直接路由，未经过 RequestProcessor。
        // 必须在释放 print_mutex_ 后执行。
        if ((on_text_ || on_context_) && !text.empty())
        {
            try
            {
                if(on_context_)on_context_(text,segment);
                else on_text_(text);
            }
            catch (const std::exception& e)
            {
                error_ =
                    std::string("ASR text callback: ") +
                    e.what();

                queue_.close();
                return;
            }
            catch (...)
            {
                error_ =
                    "ASR text callback: unknown exception";

                queue_.close();
                return;
            }
        }
        else if(text.empty())traceMark("asr_empty_text");
    }

    ok_ = true;
}
```

语法与执行次序：std::thread(&ASRWorker::run,this)借用对象地址，因此对象必须活到join之后。入口 catch 捕获标准和未知异常，设置错误并关闭队列；不能让异常穿出线程入口触发 terminate。lock_guard 在 return 或异常时自动解锁；unique_lock 允许 condition_variable 临时释放并重新获取锁。状态与谓词在同一 mutex 下更新，notify_all 使空队列等待者重新判断 closed；通知本身不是可累计事件。

StopSignals 在所有线程创建前阻塞 SIGINT/SIGTERM，普通线程调用 sigtimedwait，atomic 发布首个信号。析构置 finished、join、恢复原 mask。没有 handler 调用 Qt、日志、malloc、mutex 或 join。构造/析构应由同一 owner 线程执行。活动同步推理仍允许完成；无法保证闭源 SDK 永久卡住时存在硬截止。

5.1 最终实机注册 CTest4/4、完整 TEXT/VISION 语音、实际耳机、自然及信号退出和 321 项保护检查通过。Windows 当时 6/6 只证明 portable 逻辑，不能替代 ARM64。完整缺陷原表现、修复 diff 和日志在 stage5_01_board_acceptance 及相应 reports 目录。

本节面试题与完整答案

以下问题保留 5.1 当时语境，其 Windows6 项计数与当时双 Engine 并发问题不是最终版本现状。

1. 为什么只使用智能指针还会出现生命周期错误？

unique_ptr/shared_ptr 管理其拥有的对象；本项目的 Worker 借用 Service，lambda 借用 mutex/counter。引用捕获不会延长它们寿命。必须使被引用状态先构造、后销毁，并在其销毁前 stop/join 使用者。Runtime handle 自动释放也不能代替线程先结束。

2. 为什么析构里的 join 可能一直等待？

join 只等待，不通知。消费者卡在开放空队列的 cv.wait 时，必须先在锁内改变 closed 状态，再 notify，谓词为真才能返回。即便队列已关闭，活跃的同步 SDK 卡住仍会使 join 等待；线程回收不是强制取消机制。

3. 为什么不把 ASR、LLM、TTS 一起 requestStop？

上游完成回调还可能生产下游请求。正常排空必须先结束采集/ASR，再确认 LLM 不再向 TTS 提交，最后排空 TTS。提前 stop TTS 会拒绝仍然合法的回答。异常退出允许取消等待任务，但要明确结果并保留状态寿命。

4. shared_ptr<const ImageFrame> 是否是零拷贝？

缓存交换与消费者取指针没有整帧复制，但之前 MMAP payload 已复制到 vector，后面还有 RGB 转换和 embedding 复制。局部共享对象不等于全链路零拷贝。QBUF 后驱动复用映射内存，去掉自有帧复制需要重设计归还/共享所有权和同步。

5. 两个 Engine 都有 mutex，能否证明 NPU 没有资源冲突？

不能。两个 mutex 保护两个实例，LLMWorker 与 ASR 的 VLM 分支仍可并行调用同一硬件。应记录重叠、Runtime 错误、延迟和资源；确认冲突后再选择共享调度，不能凭推测增加全局锁并声称已经解决硬件问题。

6. 有界队列容量 2 为什么可能存在 3 个任务？

代码把执行中的任务弹出 deque 后才推理，因此容量只限制等待任务。单消费者最多 1 个活跃任务加 2 个等待任务。音频队列满丢旧、LLM/TTS 满拒绝是不同策略，统计和业务反馈不能混用。

7. 如何避免 C SDK buffer 在 C++ 异常时泄漏？

取得成功后立即由不可复制的 RAII owner 管理，析构调用配套 SDK release；输出仍需复制到业务自有 vector 时也受保护。必须保证 owner 的 context 比它活得久、析构不抛异常、只释放已成功取得的资源。并不意味着 SDK release 自身永不失败。

8. Windows 6/6 CTest 能证明什么？

证明当前 MinGW 构建的六项测试符合其断言，覆盖队列/异常/状态/替身生命周期等可移植逻辑。它不编译真实 RKNN/ALSA 实现，不验证 ARM64 ABI、模型精度、NPU 并发、音质或板端性能。工程验收必须补目标端构建及真实设备日志。

9 为什么不能在 signal handler 中调用条件变量

POSIX 异步信号可能打断持有同一锁的线程；mutex、condition_variable 和 Qt 操作不保证异步信号安全，会死锁或破坏运行状态。本项目通过阻塞信号加普通 sigwait 线程，把通知和停止放回正常线程上下文。

10 析构顺序为什么是并发接口的一部分

局部对象逆声明顺序析构，成员也逆声明顺序析构。Worker 持有的引用不会自动延长 Service、mutex 或 counter 寿命；必须在析构依赖对象前 stop/join。只把句柄换成智能指针而不检查谁仍在使用它，仍可能造成 use-after-free。

4 第五阶段 5.2 有界异步任务和 NPU 调度

5.2 多线程与异步 Pipeline 工程记录

状态：5.2 工程验收通过，完整证据见 `stage5_02_board_acceptance.md`，本轮暂停在 5.3 之前。日期：2026-10-17。

操作 1：继承 5.1 验收与授权边界

已完整读取任务书及 stage5_01_board_acceptance.md。5.1 的真实语音闭环、用户听音及协作式信号退出是基线，不算作 5.2 已通过的证据。仅修改本地和 /home/cat/projects/RK3576/rk3576_multimodal_stage5_review。原工程、原 build/stage5_full、模型、SDK、驱动、设备树及系统配置不写入。没有上传用户明确不需要的 stage5_01_acceptance_local.tgz。

本地无 Git 元数据，以 backups/stage5_02_before/ 的 107 文件快照、SHA256 和 Git no-index 格式差异记录版本。5.1 初始包清单与最终修复源码有差异，本节继承正式验收的最终源码。

操作 2：源码定位与最小方案

ASRWorker::run() 同步调用 on_text。原 TEXT 分支提交 LLMWorker；VISION 分支却直接执行 vlm->infer(frame, vlm_question)，包含 RKNN Vision 和 SmolVLM2 RKLLM，阻塞 ASR 工作线程。CameraService 已独立采集，LatestFrameBuffer 保存共享最新帧；无需重建 Camera 框架。原 TTSWorker 在一个线程中合成和播放，等待容量 2，但局部队列限制不代表全部在途回答有界。

实施顺序为冻结基线及 NPU 实测、复用推理 FIFO、跨阶段额度及过期/取消、Fake 并发测试、ARM64 与真实语音回归。模型算法和 SDK 不改。

文件	具体改动	验收方法
common/work_control.h	停止模式、截止时间、等待期限、取消 flag 和统计	Fake 过期与取消断言
ai/llm_worker.h/.cpp	保留 TEXT 接口，新增 submitTask、显式 restart、终态统计	FIFO、满载、抛异常、停止重启
app/pipeline_admission.h	一个 Ticket 覆盖推理至播放全过程	最后引用释放与 close 并发测试
audio/asr_llm_tts_smoke.cpp	VLM 异步提交、执行时取最新帧、ID/期限传递、退出语义	同输入 A/B、真实语音、信号

文件	具体改动	验收方法
audio/tts_worker.h/.cpp	保留原合成及播放，加入过期/取消/统计/重启	Fake TTS 与 Kokoro/ALSA
audio/asr_pipeline.h/.cpp	保留原丢旧音频策略，增加 ID 日志、峰值及计数	原 ASR 测试和连续采集
TTS/ALSA service 头与实现、tts_smoke.cpp	公共头用前置声明，实际调用显式 include SDK	真实 SDK 全目标编译
tests/async_pipeline_tests.cpp	闸门控制的确定性线程测试	本地和 ARM64 CTest
tests/stage5_02_npu_probe.cpp	顺序/重叠 NPU 调用对照，手工诊断 target	真实 SDK 实测；不注册 CTest
tools/stage5_02_*、回放 TSV	隔离路径、PID 检查、超时、证据收集	原始退出码、日志、321 项指纹

操作 3：真实线程与队列架构

```
flowchart LR
  Main[主线程: stop 与 join] --> Capture[VadAudioWorker: ALSA capture + Silero VAD]
  Capture --> AQ[AudioBufferQueue: 4 个 AudioSegment]
  AQ --> ASR[ASRWorker: SenseVoice + IntentRouter]
  ASR --> Admit[PipelineAdmission: 默认 2 个跨阶段 Ticket]
  Admit --> IQ[复用 LLMWorker FIFO: TEXT/VISION closure]
  IQ --> AI[唯一推理消费者: Qwen 或 RKNN + SmolVLM2]
  Camera[CameraService 独立采集线程] --> Latest[LatestFrameBuffer 最新共享帧]
  Latest --> AI
  AI --> TQ[TTSWorker FIFO: 文本及回调及 TaskOptions]
  TQ --> TTS[同一 TTS 线程: Kokoro 合成后 ALSA 播放]
  Signal[独立 sigtimedwait 线程] --> Main
```

这些是应用线程，SDK 内部线程仍由各 SDK 管理。ASR 只负责识别、路由、申请额度和提交 closure，不等待推理或 TTS。视觉帧在执行时取最新共享帧，不把 4K 图像积在队列里。共享推理消费者覆盖本入口的两种 RKLLM 与 RKNN 调用；独立程序不受该 FIFO 管理，因此运行前仍检查设备占用。

操作 4：有界性、背压与终态

默认 max_inflight=2，Ticket 一直持有到 TTS 完成或其他终态回调及 closure 释放。局部推理等待和 TTS 等待容量均保留 2；默认额度下存在活动 TTS 时最多一个等待回答。额度满拒绝新请求，打印请求 ID、路由与原因并统计；不覆盖已接收回答。

默认推理排队上限 10000 ms、TTS 排队上限 12000 ms、跨阶段生存期 45000 ms，使用 steady_clock。在阶段执行前检查，过期不调用 SDK，但仍回调终态、计数、归还额度。活跃 SDK 不强制中断，因此这些期限不构成硬实时完成保证。

completed 表示所有终态，包括取消和过期；started 才是执行过 SDK/任务体，failed 是运行错误。策略拒绝、过期、成功播放必须分别报告，不能将 completed 当推理成功或把丢弃带来的吞吐下降隐藏。配置解析严格校验范围，非法值明确失败。提供 cancel(id)，本入口尚未实现用户口头打断命令。

操作 5：生命周期、竞态与退出审查

start 保留一次性语义；restart 仅由唯一 owner 在 requestStop、join 后调用，本轮统计重置、generation 递增。submit 与 requestStop 使用同一个队列 mutex；start/join/restart/析构不能由多个 owner 并发调用，回调不能 join 自己。

Drain 关闭新入口并完成已接收任务，仍尊重原期限。CancelPending 取消尚未开始的阶段，活跃 SDK 或播放正常结束；正在推理的请求结束后若取消，不再进入 TTS。取消关闭与提交竞争产生的 NotRunning 单独记为预期取消，不误报为运行失败。

Worker 不持有队列锁执行 SDK 或业务回调。回调后先销毁 task captures，再锁内发布 completed/active=0，最后 notify；main 观察到 idle 时，TTS 转交及额度归还均已完成。Admission 表内保存 weak_ptr<atomic<bool>>，避免锁内暂时取得最后一个 Ticket 引用并析构，再次锁同一 mutex。状态、帧缓冲、VLM/Camera 声明早于 Worker，反向析构先 join Worker，后销毁被借用对象。

停止顺序为停止/唤醒录音、关闭音频队列、join VAD/ASR、关闭 Admission、排空或取消推理、join 推理、destroy Qwen、排空或取消 TTS、join TTS、close ALSA/destroy TTS、Camera stop/join、VLM/RKNN destroy。继承 5.1 sigtimedwait，不在异步 signal handler 中加锁、打印或调用 SDK。

剩余边界：SDK 永久阻塞时，C++ join 无硬截止保证。外层保护只允许终止本次已核实 PID，必须将强制终止记为退出失败。没有以有限回归宣称无任何竞态，也没有声称已完成 ASan/TSan 或长期压力。

操作 6：回声与 TTS 调度取舍

HyperX 耳机降低回灌，不等于 AEC。保持播放期间 ASR 可继续处理，以端到端额度限制误触发积压。冷却、PTT、AEC、打断需要明确产品语义，本节未实现。没有把 TTS 拆为合成和播放两个线程，避免增加预合成 PCM 积压与资源生命周期；模型计算优化留在 5.4。

操作 7：构建问题与证据分类

首次 ARM64 构建暴露 `tts_smoke.cpp` 对 SherpaOnnxWriteWave 的间接 include 依赖。公共头解耦后补显式 SDK C API include，保留原失败日志及复测日志。修复后全目标构建成功，ARM64 CTest 5/5，0.29 s；本地 portable CTest 7/7，0.53 s。CTest 中 Fake/Mock 不代表硬件推理；下文另记真实模型回归。

完整数据见 `stage5_02_performance.md` 和 `stage5_02_board_acceptance.md`；完整关键源码见 `stage5_02_source_appendix.md`，逐句解释及面试题见 `stage5_02_statement_notes.md`。本节已完成，等待用户审阅后再进入 5.3。

操作 8：适用范围补充

CancelPending 针对尚未开始的推理/TTS 阶段；音频队列仍沿用 5.1 的关闭并排空方式。已采集音频段可能继续被 ASR 识别，但 Admission 已关闭，会明确拒绝新业务提交。因此取消退出仍可能包含残留 ASR 和当前 SDK/播放的耗时，不承诺瞬时停止所有计算。

仅默认 `STAGE5_MAX_INFLIGHT=2` 完成了本轮硬件验收。上调至 8 不会无限增长，但局部等待队列仍为 2，可能产生额外 QueueFull 及程序失败状态；放宽额度前需结合容量和实际负载重新验收。

并发控制没有消除 TTS 的 CPU/内存带宽竞争。真实连续测试中 VLM 有一次 2793.65 ms，不能把单消费者描述为所有延迟均固定或下降。模型成功率和请求交付率分别报告；播放成功仍需要用户听音确认。

前文是 5.2 当时完成记录；其“后续 5.3”属于时间线。最终仍复用同一队列和 Worker，而非两套并行 Demo。

完整关键源码 src/ai/llm_worker.cpp

```
#include "ai/llm_worker.h"
#include <algorithm>
#include <atomic>
#include <exception>
#include <iostream>
#include <stdexcept>
#include <utility>

LLMWorker::LLMWorker(LLMEngine& engine,std::size_t
capacity):LLMWorker(capacity){engine_=&engine;}
LLMWorker::LLMWorker(std::size_t capacity):capacity_(capacity) {
    if (!capacity_) throw std::invalid_argument("LLMWorker capacity must be greater than
zero");
}
LLMWorker::~LLMWorker(){requestStop();join();}
bool LLMWorker::start(std::string& error) {
    std::lock_guard<std::mutex> lock(mutex_);error.clear();
    if (started_ || stop_requested_) {error="LLMWorker can only be started once; use restart
after stop/join";return false;}
    try {thread_=std::thread(&LLMWorker::run,this);started_=true;++stats_.generation;}
    catch(const std::exception& e){error=std::string("LLMWorker thread creation failed:
")+e.what();return false;}
    return true;
}
bool LLMWorker::restart(std::string& error) {
    {std::lock_guard<std::mutex> lock(mutex_);
     if (!started_ || !stop_requested_ || thread_.joinable() || !pending_.empty())
{error="restart requires stopped and joined worker";return false;}
     started_=false;stop_requested_=false;rejected_=0;
     const auto generation=stats_.generation;stats_={};stats_.generation=generation;}
    return start(error);
}
LLMSubmitStatus LLMWorker::submit(std::uint64_t id,std::string question,LLMWorkerChunk
chunk,LLMWorkerDone done,TaskOptions options) {
    if (!engine_) throw std::logic_error("submit(text) requires an LLMEngine");
    return submitTask(id,[this,question=std::move(question)](const LLMStreamCallback&
callback){
        return engine_->inferStreaming(question,callback);
    },std::move(chunk),std::move(done),std::move(options));
}
LLMSubmitStatus LLMWorker::submitTask(std::uint64_t id,Inference inference,LLMWorkerChunk
chunk,LLMWorkerDone done,TaskOptions options) {
    if (!inference) throw std::invalid_argument("Empty inference task");
    const auto trace=options.trace;std::chrono::steady_clock::time_point
enqueued{};std::size_t depth=0;
    {std::lock_guard<std::mutex> lock(mutex_);
     if (!started_ || stop_requested_) {++stats_.rejected_stopped;if(trace)trace-
>mark("inference_rejected_stopped");return LLMSubmitStatus::NotRunning;}
     if (pending_.size()>=capacity_) {++rejected_;++stats_.rejected_full;if(trace)trace-
>mark("inference_rejected_full");return LLMSubmitStatus::QueueFull;}
     Task
task;task.id=id;task.inference=std::move(inference);task.on_chunk=std::move(chunk);task.on_d
one=std::move(done);

task.enqueued_at=std::chrono::steady_clock::now();enqueued=task.enqueued_at;task.options=std
::move(options);

pending_.push_back(std::move(task));++stats_.accepted;depth=pending_.size();stats_.peak_dept
h=std::max(stats_.peak_depth,depth);}
```

```
    if(trace)trace->mark("inference_enqueued",enqueued,static_cast<double>(depth));
    cv_.notify_one();return LLMSubmitStatus::Accepted;
}
void LLMWorker::requestStop(StopMode mode) noexcept {
    {std::lock_guard<std::mutex> lock(mutex_);
     if (!stop_requested_ || mode==StopMode::CancelPending) stop_mode_=mode;
     stop_requested_=true;}
    cv_.notify_all();
}
void LLMWorker::join(){if(thread_.joinable())thread_.join();}
std::size_t LLMWorker::pendingCount() const {std::lock_guard<std::mutex> lock(mutex_);return
pending_.size();}
std::size_t LLMWorker::rejectedCount() const {std::lock_guard<std::mutex>
lock(mutex_);return rejected_;}
WorkerStats LLMWorker::stats() const {std::lock_guard<std::mutex> lock(mutex_);auto
copy=stats_;copy.pending=pending_.size();return copy;}
bool LLMWorker::waitIdleFor(std::chrono::milliseconds timeout) {
    std::unique_lock<std::mutex> lock(mutex_);
    return cv_.wait_for(lock,timeout,[&]{return pending_.empty() && stats_.active==0;});
}
void LLMWorker::run() {
    using Clock=std::chrono::steady_clock;
    for (;;) {
        Task task;bool cancel=false;
        {std::unique_lock<std::mutex> lock(mutex_);
         cv_.wait(lock,[&]{return stop_requested_ || !pending_.empty();});
         if (pending_.empty() && stop_requested_) return;
         task=std::move(pending_.front());pending_.pop_front();stats_.active=1;
         cancel=stop_requested_ && stop_mode_==StopMode::CancelPending;}
        const auto begin=Clock::now();
        RequestTraceScope trace_scope(task.options.trace);
        traceMark("inference_dequeued",begin);
        const double waited=std::chrono::duration<double,std::milli>(begin-
task.enqueued_at).count();
        InferenceResult result;
        result.disposition=task.options.disposition(task.enqueued_at,begin,cancel);
        const auto disposition=result.disposition;
        if (disposition!=TaskDisposition::Completed) {

traceMark(disposition==TaskDisposition::Cancelled?"inference_cancelled":"inference_expired");
            result.error_message=std::string("Inference task ")+dispositionName(disposition);
            std::cerr<<"[Inference "<<dispositionName(disposition)<<" #"<<task.id<<"]
queue_ms="<<waited<<'\n';
        } else {
            {std::lock_guard<std::mutex> lock(mutex_);++stats_.started;}
            traceMark("inference_begin");
            std::atomic<bool> chunk_failed{false};
            try {
                result=task.inference([&](const std::string& chunk){
                    if (task.on_chunk) try {task.on_chunk(task.id,chunk);} catch(...)
{chunk_failed.store(true);}
                });
            } catch(const std::exception& e)
{result.success=false;result.text.clear();result.error_message=std::string("LLMWorker
inference exception: ")+e.what();}
              catch(...)
{result.success=false;result.text.clear();result.error_message="LLMWorker inference: unknown
exception";}
            if (chunk_failed.load())
{result.success=false;result.text.clear();result.error_message="LLMWorker chunk callback
threw an exception";}
            traceMark(result.success?"inference_end":"inference_failed");
```

```
        }
        const double service=std::chrono::duration<double,std::milli>(Clock::now()-
begin).count();
        bool callback_failed=false;
        if (task.on_done) try
{task.on_done(task.id,result,disposition==TaskDisposition::Cancelled?-1.0:waited);}
        catch(...) {callback_failed=true;std::cerr<<"[Inference callback error
#"<<task.id<<"]\n";}
        const bool failed=disposition==TaskDisposition::Completed && !result.success;
        task=Task{};
        {std::lock_guard<std::mutex> lock(mutex_);
         ++stats_.completed;stats_.active=0;stats_.failed+=failed;

stats_.cancelled+=disposition==TaskDisposition::Cancelled;stats_.expired+=disposition==TaskD
isposition::Expired;
         stats_.callback_errors+=callback_failed;stats_.queue_wait_sum_ms+=waited;

stats_.queue_wait_max_ms=std::max(stats_.queue_wait_max_ms,waited);stats_.service_sum_ms+=se
rvice;}
        cv_.notify_all();
    }
}
```

完整关键源码 include/app/pipeline_admission.h

```
#pragma once
#include "common/work_control.h"
#include <algorithm>
#include <map>
#include <mutex>
#include <stdexcept>

// One credit covers inference, TTS waiting, synthesis and playback, not just one queue.
// Captured shared tickets release credit on all callback/exception/cancellation paths.
class PipelineAdmission {
    struct State;
public:
    struct Stats {std::size_t accepted=0,rejected=0,inflight=0,peak=0;bool closed=false;};
    class Ticket {
    public:
        ~Ticket();
        TaskOptions options(std::chrono::milliseconds max_wait,RequestTracePtr trace={})
const {
            TaskOptions
result;result.deadline=deadline_;result.max_queue_wait=max_wait;result.cancelled=cancelled_;
result.trace=std::move(trace);return result;
        }
        bool cancelled() const noexcept {return cancelled_->load();}
    private:
        friend class PipelineAdmission;
        Ticket(std::shared_ptr<State> state,std::uint64_t id,TaskOptions::Clock::time_point
deadline)
            :state_(std::move(state)),id_(id),deadline_(deadline),cancelled_(std::make_share
d<std::atomic<bool>>(false)){}
        std::shared_ptr<State> state_;
        std::uint64_t id_;
        TaskOptions::Clock::time_point deadline_;
        std::shared_ptr<std::atomic<bool>> cancelled_;
        bool admitted_=false;
```

```
    };
    explicit PipelineAdmission(std::size_t
capacity):state_(std::make_shared<State>(capacity)) {
        if(!capacity)throw std::invalid_argument("Pipeline admission capacity must be
positive");
    }
    std::shared_ptr<Ticket> tryAcquire(std::uint64_t id,std::chrono::milliseconds lifetime)
{
        std::lock_guard<std::mutex> lock(state_->mutex);
        if(state_->stats.closed || state_->stats.inflight>=state_->capacity || state_-
>tickets.count(id)) {
            ++state_->stats.rejected;return {};
        }
        auto ticket=std::shared_ptr<Ticket>(new
Ticket(state_,id,TaskOptions::Clock::now()+lifetime));
        state_->tickets.emplace(id,ticket->cancelled_);
        ticket->admitted_=true;++state_->stats.accepted;++state_->stats.inflight;
        state_->stats.peak=std::max(state_->stats.peak,state_->stats.inflight);
        return ticket;
    }
    bool cancel(std::uint64_t id) {
        std::lock_guard<std::mutex> lock(state_->mutex);
        const auto found=state_->tickets.find(id);
        if(found==state_->tickets.end())return false;
        if(auto flag=found->second.lock()){flag->store(true);return true;}
        return false;
    }
    void close(bool cancel_waiting) {
        std::lock_guard<std::mutex> lock(state_->mutex);state_->stats.closed=true;
        if(cancel_waiting)for(auto& row:state_->tickets)if(auto flag=row.second.lock())flag-
>store(true);
    }
    Stats stats() const {std::lock_guard<std::mutex> lock(state_->mutex);return state_-
>stats;}
private:
    struct State {
        explicit State(std::size_t size):capacity(size){}
        std::mutex mutex;
        const std::size_t capacity;
        Stats stats;
        // Weak cancellation flags avoid destroying a Ticket recursively while this mutex is
held.
        std::map<std::uint64_t,std::weak_ptr<std::atomic<bool>>> tickets;
    };
    std::shared_ptr<State> state_;
};
inline PipelineAdmission::Ticket::~Ticket() {
    if(admitted_){std::lock_guard<std::mutex> lock(state_->mutex);--state_-
>stats.inflight;state_->tickets.erase(id_);}
}
```

完整关键源码 src/audio/tts_worker.cpp void TTSWorker::run()

```
void TTSWorker::run()
{
    for (;;)
    {
        Task task;
        bool cancelled = false;
```

```
        {
            std::unique_lock<std::mutex> lock(mutex_);

            // 没有任务时休眠，不进行忙轮询。
            cv_.wait(lock, [this] {
                return stop_requested_ ||
                       !pending_.empty();
            });

            // 队列清空且要求停止时，退出线程。
            if (pending_.empty() && stop_requested_)
            {
                return;
            }

            // FIFO: 取出队首任务。
            task = std::move(pending_.front());
            pending_.pop_front();

            // 停止请求到来后，
            // 尚未开始的排队任务转为取消结果。
            cancelled = stop_requested_ && stop_mode_ == StopMode::CancelPending;
            stats_.active = 1;
        }

        const auto begin = std::chrono::steady_clock::now();
        RequestTraceScope trace_scope(task.options.trace);
        traceMark("tts_dequeued",begin);
        const double waited = std::chrono::duration<double,std::milli>(begin -
task.enqueued_at).count();
        TTSWorkerResult result;
        result.disposition = task.options.disposition(task.enqueued_at, begin, cancelled);
        const auto disposition = result.disposition;
        const double queue_wait_ms = disposition == TaskDisposition::Cancelled ? -1.0 :
waited;
        if (disposition != TaskDisposition::Completed)
        {
            traceMark(disposition==TaskDisposition::Cancelled?"tts_cancelled":"tts_expired");
            result.error_message = std::string("TTS task ") + dispositionName(disposition);
            std::cerr << "[TTS " << dispositionName(disposition) << " #" << task.id << "]
queue_ms=" << waited << '\n';
        }
        else
        {
            { std::lock_guard<std::mutex> lock(mutex_); ++stats_.started; }
            try
            {
                TTSAudio audio;
                std::string error;
                traceMark("tts_synthesis_begin");

                // 1. 在工作线程中执行 TTS。
                if (!tts_.synthesize(
                        task.text, audio, error))
                {
                    result.error_message =
                        "TTS synthesize: " + error;
                }
                else if (
                    audio.sample_rate <= 0 ||
```

```
        audio.samples.empty())
    {
        result.error_message =
            "TTS returned invalid PCM";
    }
    else
    {
        result.synthesis_ms =
            audio.synthesis_ms;

        result.sample_rate =
            audio.sample_rate;

        result.audio_seconds =
            static_cast<double>(
                audio.samples.size()
            ) / audio.sample_rate;

        // 记录本次播放前的 XRUN 总数。
        const std::uint64_t xruns_before =
            playback_.xrunCount();

        const auto play_start =
            std::chrono::steady_clock::now();

        // 2. 在同一工作线程中播放 PCM。
        const bool played = playback_.play(
            audio.samples.data(),
            audio.samples.size(),
            error
        );

        const auto play_end =
            std::chrono::steady_clock::now();

        result.playback_ms =
            std::chrono::duration<
                double, std::milli
            >(
                play_end - play_start
            ).count();

        result.xrun_count =
            playback_.xrunCount() -
            xruns_before;

        if (!played)
        {
            result.error_message =
                "ALSA playback: " + error;
        }
        else if (result.xrun_count != 0)
        {
            result.error_message =
                "ALSA detected XRUN";
        }
        else
        {
            result.success = true;
        }
    }
}
```

```
            catch (const std::exception& e)
            {
                result.error_message =
                    std::string(
                        "TTSWorker exception: "
                    ) + e.what();
            }
            catch (...)
            {
                result.error_message =
                    "TTSWorker: unknown exception";
            }
        }

        const double service =
std::chrono::duration<double,std::milli>(std::chrono::steady_clock::now()-begin).count();

traceMark(result.success?"request_complete":disposition==TaskDisposition::Completed?"tts_fai
led":"tts_terminal");
        bool callback_failed = false;
        if (task.on_done)
        {
            try { task.on_done(task.id, result, queue_wait_ms); }
            catch (...) { callback_failed=true; std::cerr << "[TTS callback error #" <<
task.id << "]\n"; }
        }
        const bool failed = disposition == TaskDisposition::Completed && !result.success;
        task = Task{}; // release captured admission tickets outside the queue lock
        {
            std::lock_guard<std::mutex> lock(mutex_);
            ++stats_.completed; stats_.active=0; stats_.failed+=failed;
            stats_.cancelled+=disposition==TaskDisposition::Cancelled;
            stats_.expired+=disposition==TaskDisposition::Expired;
            stats_.callback_errors+=callback_failed;
            stats_.queue_wait_sum_ms+=waited;
stats_.queue_wait_max_ms=std::max(stats_.queue_wait_max_ms,waited);
            stats_.service_sum_ms+=service;
        }
        cv_.notify_all();
    }
}
```

5.2 关键语句解释与接口审查

本文件与 stage5_02_source_appendix.md 配套：附录给出未经省略的当前完整文件及 SHA256；此处按实际语句执行顺序解释。一个跨多行的 std::cout << ...; 是一条表达式语句，不把每个输出操作误当独立控制流程。

操作 1：公共类型与 C++ 语法

- #pragma once 避免头文件在一个翻译单元中重复展开。#include 引入类型声明，不初始化设备。std:: 指标准库 namespace，例如 std::chrono::steady_clock 属于嵌套 namespace；不是项目自建时钟。

- `enum class StopMode` 是强类型枚举，必须写 `StopMode::Drain`，避免与其他整数/枚举意外混用。`TaskDisposition` 的 Completed 表示执行路径正常进入，是否业务成功仍由 result.success 判断。
- `dispositionName(...) noexcept` 返回静态字符串字面量，生命周期覆盖整个进程，不返回局部 string.c_str()。嵌套条件表达式只选择三个固定名字。
- `using Clock=std::chrono::steady_clock` 是类型别名，不建立时钟对象。steady_clock 单调递增，适合等待和期限，系统校时不会改变结果。
- `Clock::time_point deadline=...::max()` 与 `milliseconds::max()` 表示默认不启用该限制。`milliseconds` 是 duration 的类型别名，实际模板包含计数表示类型和 `std::ratio<1,1000>` 周期。
- `std::shared_ptr<std::atomic<bool>> cancelled` 共享取消位的所有权。atomic 的 load/store 默认顺序一致性，保证主线程与 Worker 对标志的访问没有数据竞争；它不使其他对象自动线程安全。
- `disposition(enqueued,now,stopping) const noexcept` 不改变选项本身。先判停止/取消，再判 deadline 或 queue wait；因此同时发生时取消优先。短路 `&&` 在指针为空时不会解引用，毫秒最大值先判断再比较，避免不必要换算。
- `WorkerStats` 的 `{}` 和成员 `=0` 提供确定初始值；generation 区分显式重启轮次。size_t 是无符号容器计数类型，uint64_t 是固定宽度请求/轮次编号。统计只由 Worker mutex 保护，不能绕过 stats() 读成员。

操作 2：PipelineAdmission 逐句所有权解释

- `struct State;` 在类开头前置声明私有内部类型，使 Ticket 能先声明 `shared_ptr<State>`。完整 State 定义随后出现；没有在不完整类型上调用 sizeof 或 delete。
- `Stats` 为快照值类型，不包含 mutex。`class Ticket` 内 `friend class PipelineAdmission` 只允许外层类调用私有构造器，不把构造权限交给调用者。
- Ticket 构造器初始化列表按成员声明顺序建立 state、id、deadline 和新 atomic flag。`std::move(state)` 转移参数中的 shared_ptr 所有权，不移动底层 State 对象。
- `options(max_wait)` 创建独立 TaskOptions，把同一个 deadline 和 shared flag 复制到下游，允许不同阶段使用不同 max_queue_wait。它不把 Ticket 本身放进 flag，避免所有权环。

- `cancelled() const noexcept` 只读取 atomic，不锁 Admission mutex。调用者必须持有 Ticket，才能保证 flag 存活。
- Admission 构造器先 make_shared<State>(capacity) 建立状态，再检查 capacity 是否为零；抛 invalid_argument 时已经构造的 shared_ptr 自动析构，不泄漏。
- `tryAcquire` 的 `lock_guard<std::mutex> lock(state_->mutex)` 在构造时调用 mutex.lock()，离开块或异常退栈时析构调用 unlock()。模板参数 mutex 指定可锁对象类型，局部变量 lock 承担 RAII，不应误写成无名临时对象。
- closed、inflight>=capacity、tickets.count(id) 在同一临界区检查，拒绝时只增 rejected 并返回空 shared_ptr。重复 ID 不会覆盖已有取消标记。
- `shared_ptr<Ticket>(new Ticket(...))` 使用私有构造器，分配/控制块失败会释放已分配对象。此时 admitted_=false，因此析构不会错误扣减计数。
- `tickets.emplace(id,ticket->cancelled_)` 在所有 accepted/inflight 更新之前插入 weak flag，若 map 分配异常，Ticket 尚未计入额度，析构不会再次获取 State mutex。
- `admitted_=true` 后才增加 accepted/inflight 和 peak。`std::max` 比较历史峰值与当前在途数，不表示队列等待长度。成功返回前 lock_guard 已释放锁。
- `cancel(id)` 同锁查 map；找不到返回 false。weak_ptr.lock() 建立 flag 的临时强引用，存在时 atomic.store(true)。临时销毁的是 atomic flag 引用，不是 Ticket，因此不会锁内递归触发 Ticket 析构。
- `close(cancel_waiting)` 先设置 closed，拒绝此后新请求；若取消则逐个设置仍存活 flag。Drain 调用 close(false) 保留已有任务状态。
- `stats() const` 通过 shared State mutex 保护并按值返回快照，调用者不会持锁遍历生产对象。
- State 的 shared_ptr 由 Admission 和所有 Ticket 共享，Ticket 即使晚于 Admission 对象销毁，State mutex 仍存在。map 保存弱引用，既不保活 Ticket，也不形成循环。
- `Ticket::~Ticket()` 只有 admitted_ 为真时才锁状态、减 inflight、erase(id)。最后引用可来自推理或 TTS closure；Worker 在队列锁之外清除 closure，避免双锁反向依赖。

操作 3：LLMWorker 的构造、提交与线程语句

- 带 Engine 构造器委托给容量构造器，再保存非拥有指针 `engine_=&engine`。别名 `InferenceWorker=LLMWorker` 没有新对象布局和新队列。容量零抛 invalid_argument。

- 析构器依次 requestStop()、join()，不能由自身回调销毁 Worker。Engine、callback 引用和同步对象均必须晚于 join 才销毁。
- start 在 mutex 内清理 error、检查 started/stop_requested；创建 `std::thread(&LLMWorker::run,this)` 将成员函数指针和对象地址传给新线程。新线程需要同锁，直到创建者设置 started/generation 并解锁后才能操作队列。
- thread 创建异常被转换为 error + false，没有假装线程已启动。restart 在同锁检查已停止、已 join、队列空，重置本轮状态再调用 start；这依赖 start/join/restart 由同一 owner 串行执行。
- submit(text) 检查 engine_，无 Engine 的通用 Worker 不允许错误调用 TEXT 便捷接口。lambda `[this,question=std::move(question)]` 捕获 Worker 地址与自有文本，调用已有 inferStreaming；不引用提交栈上的 question。
- submitTask 检查 inference 非空；锁内检查 running/容量，分别增加 stopped/full 拒绝计数。创建 Task，移动函数对象和回调、写 enqueue 时间和 options，再 push_back。push 成功后增加 accepted/peak；异常前不虚增 accepted。
- 解锁后 notify_one，减少被唤醒线程立即争锁。通知不是条件本身；即使 notify 早于 wait，队列谓词仍可防丢唤醒。
- requestStop 在锁内设置 stop_requested。已经取消后不允许 Drain 降级回排空；随后 notify_all 唤醒空队列等待。该 noexcept 函数仍依赖有效 mutex，不可放进异步 signal handler。
- join 只在 joinable 时等待，不能同时由两个 owner 调用。

stats/pendingCount/rejectedCount 都在锁内读取；stats 按值返回，并用当前 deque.size() 更新 pending。

- waitIdleFor 使用 unique_lock，因为 condition_variable 在等待时必须暂时解锁、醒来重新加锁。谓词要求 pending.empty 且 active==0，虚假唤醒不会提前成功。

操作 4：推理 run 循环的每个执行阶段

1. 每轮创建局部 Task 和 cancel 标记。unique_lock 后 wait(stop||非空)。空且 stop 时 return；否则 move 队首并 pop，设置 active=1，在同锁读本轮停止模式。
2. begin-enqueued_at 转为 `duration<double,std::milli>` 得毫秒浮点等待时间；TaskOptions 根据队列时刻、当前时刻、stop 决定终态。

3. 取消/过期路径只填 error_message、输出 ID/queue_ms，不执行推理体。它们仍走后面的 done 和统计，不能静默丢失。

4. 可执行路径先增加 started。局部 atomic chunk_failed 供 SDK 回调报告用户 chunk 异常；回调 lambda 引用本轮 Task 和 flag，只在同步 inferStreaming 有效期内使用。

5. task.inference 返回 InferenceResult。std::exception 和未知异常都转为 success=false、清空 text 并记录原因，防止半段答案被当完整成功结果。chunk 回调异常同样使本次失败，不让异常穿过 C SDK 边界。

6. service 计时在 done 之前截止，因此表示任务体及必要包装耗时，不包括业务 done 的转交时间。waitIdle 则包含 done 完成，不会用这两种口径混称同一指标。

7. done 在队列锁外调用。Cancelled 为兼容旧接口传 -1 等待值；统计保留实际等待。done 抛异常单独计 callback_errors 并打印 ID，继续清理，不吞成成功。

8. failed 仅对真实执行且 result.success=false 计入。`task=Task{}` 在队列锁外清空函数对象及 Ticket 捕获；随后锁内增加 completed、归零 active、累积失败/取消/过期/回调异常/等待和服务时间。最后 notify_all 允许主线程观察完整终态。

操作 5：TTSWorker 及不变的设备路径

TTSWorker 的 start/submit/stop/stats/waitIdle/restart 与推理 Worker 的锁和谓词相同，但 start 先检查 tts.ready 与 playback.ready。它借用两个 Service，仍不拥有设备。

run 将队首 Task move 到局部，锁外判取消/过期。执行分支先增加 started，调用原 `tts_.synthesize(task.text,audio,error)`；失败填合成错误，成功仍验证 sample_rate>0 和 samples 非空。audio 是局部 TTSAudio，vector<float> 自动释放 PCM。

随后保存 synthesis_ms、sample_rate、audio_seconds，读取 XRUN 基线，记录 play_start，调用原 `playback_.play(audio.samples.data(),audio.samples.size(),error)`。data() 的借用指针只在同步 play 期间有效，audio 作用域覆盖该调用。play_end-play_start 转毫秒，XRUN 用调用前后计数差。写失败、XRUN 非零分别填错误；只有两者都正常才 success=true。

完成回调在锁外执行并捕获异常。service 和 waited 分别累计；与推理 Worker 一样，先清除 task 再发布 active=0。过期/取消时 synthesis_ms/playback_ms 保持默认零，不伪造模型执行。

TTS/ALSA 公共头改为不完整 C 类型指针，只需声明就能存储指针；具体 SDK include 在 cpp 中，析构器也在 cpp 中实现。`tts_smoke.cpp` 自己调用 SherpaOnnxWriteWave，因此必须自己 include C API；这是首次 ARM64 编译暴露并修复的具体依赖。

操作 6：完整语音入口的语句与捕获

初始化顺序继承 5.1：StopSignals 在 SDK 线程之前阻塞信号；初始化 Qwen、SenseVoice、Kokoro、ALSA，然后建立状态/互斥量/Admission/帧缓冲与 Worker。VLM/Camera 指针声明提前，但模型初始化步骤不变。异常反向析构先停止借用它们的 Worker。

每次 on_text 创建 DispatchTimer，router.route 产生 RouteDecision，next_id++ 分配唯一 ID，tryAcquire 申请 Ticket。额度失败增加 pipeline_rejected、打印 route/limit 并 return；局部计时析构仍输出本次提交耗时。计时包含同步日志开销，不是纯 CPU 指令耗时。

VISION 禁用时保持原 skipped 行为。启用时保留英文短问句，submitTask 的 `[&,vlm_question]` 按值保留问题，只按引用借用 owner 的 vlm/frames。执行时 frames.getLatest() 返回有所有权的共享帧；infer 同步结束后才释放局部帧，不捕获 ASR 的 recognized 引用。

VISION done 按值捕获 question_at 和 ticket，先在 state_mutex 下更新 completed/成功/失败/取消/过期，再单独在 print_mutex 下输出。没有同时持 state_mutex 和 print_mutex。成功非空但 ticket 已取消时只记 downstream_cancelled；未取消才向 TTS 提交，TTS done 再按值持 Ticket。submit 返回 Accepted 后计业务 accepted；失败路径记录，不泄漏额度。

TEXT 分支复用原 prompt。prefix_printed 为每请求 shared_ptr<bool>，所有流式和 done 对它的访问由 print_mutex 保护。问题和时间不是借用提交栈。TEXT done 依据 disposition 区分运行错误与取消/过期，成功答案交给已有 TTSWorker；下游捕获同一 Ticket 和原始时刻，用 steady_clock 计算 ASR 文本至播放结束时间。

原 ASRWorker 和 VAD 模型处理保持。仅测试环境指定 STAGE5_REPLAY_FILE 时，显式打印 NO_MICROPHONE_ASR_OR_VAD_INFERENCE，按 TSV 绝对偏移 sleep_until 后调用同一个 on_text；不启动生产者/识别者，也不把其 ok 计作通过。正常麦克风路径仍启动 consumer、producer，20ms 轮询完成/信号，停止捕获后 join 二者。

ASR join 后关闭 Admission，禁止再提交。wait_for_jobs 用 Worker waitIdleFor(100ms)，不是 accepted/completed 业务条件变量。CancelPending 信号设置所有 Ticket 标志并停止接收

推理；TTS 当前执行不受强制打断。等待期限到只报告失败，后续 join 仍需 SDK 协作，外层测试保护负责记录无限等待风险。

推理 join 后才 shutdown Qwen，TTS join 后才 close playback 和 shutdown TTS，Camera stop 后才 VLM shutdown。最后输出每级统计。passed 检查资源计数、终态完整、零运行/回调错误和零额度泄漏；允许有显式策略拒绝/过期，所以 PASS 是调度一致性结果，完整回答率要另看 successful。

操作 7：测试语句及可证明范围

Gate 使用 promise/shared_future：entered 通知任务已进入，release 阻塞执行直到测试主动释放，不用碰运气的 sleep 模拟重推理。OpenOnExit 在测试抛异常时释放 gate，避免析构 Worker join 死锁。ASR 测试用真实 ASRWorker/AudioBufferQueue 加 Fake recognize，第一推理阻塞时断言三段都已识别提交，随后检查 FIFO 和不同线程 ID。

满载测试容量一，先闸住活动任务，第二入队，第三必须 QueueFull；取消 flag 后断言第二执行体绝不调用。过期 deadline 在过去，同样不得执行任务体，但都要完成终态回调。restart 检查 generation=2；throwing done 必须 callback_errors=1 且 waitIdle 成功。

停止模式分别验证 Drain 执行两项、CancelPending 只执行活动一项且两项都有终态。Admission 检查总额度、按 ID 取消、归还和关闭，再运行 200 轮未引用释放与 close 竞争。Fake TTS 模拟同步 synth/play、队列满和过期/重启。它们证明指定并发交错，不证明真实 Runtime 无内部问题，也不是模型质量测试。

操作 8：面试题与参考答案

9. 为什么两个 Engine 各自 mutex 还不能证明 NPU 并行安全？锁的互斥域不同，共用 Runtime/driver 资源仍可能争用。本项目先实测调用重叠代价，再用一个消费者统一调度，不声称已证明并行崩溃。

10. 队列有界为什么还会积压？局部容量不包含其他阶段与活动任务。跨阶段 Ticket 才能限制尚未结束回答总数；PCM、音频段和最新帧仍须分别有界。

11. 为什么 condition_variable 要谓词？通知可能先于等待、也可能虚假唤醒；锁保护状态，谓词决定是否继续。停止必须改变谓词并 notify_all。

12. 为什么 done 后才 active=0？done 可能向 TTS 提交。先宣告空闲，main 会过早关闭 TTS 或销毁 callback 引用。

13. shared_ptr 能消灭死锁吗？不能。最后引用析构可能获得另一把锁；本项目将 Ticket 释放放到队列锁外，取消表只弱持 flag。

14. 为什么 cancellation 不等于中断？flag 只能影响协作检查点；硬中断 SDK 会破坏回调和模型状态。活跃阶段完成、等待阶段取消是明确可测试的语义。

15. ASR 异步化会使 NPU 更快吗？不会必然加速模型计算；它减少 ASR 被业务回调占用的时间。总吞吐和成功播放率必须单独报告。

16. 如何解释 P95？先说明样本数、nearest-rank 等算法和条件；4 个样本的 P95 等于最大值，不能宣传生产尾延迟已稳定。

17. 如何验证设备释放？Worker join、Runtime shutdown 日志之外，还查进程消失、Camera/ALSA/NPU fuser、ALSA status 和原工程指纹。仅打印 success 不够。

18. 为什么不用 TTS 播放期间关闭 ASR？会破坏连续输入。耳机环境本节保留输入并背压；AEC、PTT 和打断属于另需定义的能力。

核心实测取舍

真实 SDK 每类 n3：顺序 Qwen 调用均值 945.432ms，重叠时 2338.717ms；Vision 加 Smol 从 1927.090ms 到 2667.267ms。6/6 对均成功，所以没有证明并行必崩；选择共同消费者是降低竞争、明确顺序与背压。异步 VISION 回调 1737.730ms 到 0.034758ms 是提交阻塞改善，不能当 VLM 推理变成 0.035ms。

5.2 ARM64 CTest5/5，所有是 Fake/确定性逻辑测试；真实耳机和交替/连续输入另行验证。过载可以拒绝输入或过期旧任务，必须统计全部终态。推理成功率的分母是实际执行，语音交付率的分母是全部输入，不能混用。

5 第五阶段 5.3 Camera 内存和预处理

V4L2 采用 MMAP，不是 DMA-BUF。DQBUF 后按实际 plane stride/size 复制到自有 NV12，再 QBUF。驱动随即可以重用映射内存，因此异步 VLM 不能借用映射裸指针。LatestFrameBuffer 发布/读取只交换共享指针，没有每次再复制整帧；CPU 预处理融合 NV12 到 RGB 和 Resize，单独转换与 Resize 时间未拆分。

每帧仍有一次 12,441,600 字节自有 NV12 复制，按需生成 786,432 字节 RGB，SDK 输入内部转换/复制不透明，Vision 输出复制 147,456 字节浮点 Embedding。池有三槽，空闲是 unique_ptr 独占，发布是 shared_ptr<const ImageFrame>，最后强引用释放才归还。每次 lease

新建控制块，旧 weak_ptr 不能在地址复用后复活。池满只做 DQBUF/QBUF 并计数，不无界分配。

完整关键源码 include/video/owned_frame_pool.h

```
#pragma once
#include "common/types.h"
#include <array>
#include <exception>
#include <memory>
#include <mutex>
#include <utility>

// Bounded owned CPU frames. A lease returns only when its shared_ptr control block dies.
// Old weak_ptrs cannot lock a later lease even if the ImageFrame address is reused.
class OwnedFramePool {
    struct State {
        std::mutex mutex;
        std::array<std::unique_ptr<ImageFrame>,3> free;
        std::size_t allocated=0,in_use=0,acquired=0,reused=0,exhausted=0;
        bool retired=false;
    };
public:
    struct Stats {std::size_t allocated=0,in_use=0,acquired=0,reused=0,exhausted=0;};
    OwnedFramePool():state_(std::make_shared<State>()){}
    OwnedFramePool(const OwnedFramePool&)=delete;
    OwnedFramePool& operator=(const OwnedFramePool&)=delete;
    ~OwnedFramePool(){retire();}
    std::shared_ptr<ImageFrame> acquire() {
        std::unique_ptr<ImageFrame> owned;
        {
            std::lock_guard<std::mutex> lock(state_->mutex);
            if(state_->retired)return {};
            for(auto& slot:state_->free)if(slot){owned=std::move(slot);++state_-
>reused;break;}
            if(!owned) {
                if(state_->allocated==state_->free.size()){++state_->exhausted;return {};}
                owned=std::make_unique<ImageFrame>();++state_->allocated;
            }
            ++state_->in_use;++state_->acquired;
        }
        // Construct outside the mutex: shared_ptr allocation failure invokes the deleter.
        return std::shared_ptr<ImageFrame>(owned.release(),[state=state_](ImageFrame* frame)
mutable noexcept {
            auto owner=std::move(state); // weak control blocks must not retain the pool
after return.
            std::unique_ptr<ImageFrame> returning(frame);
            {
                std::lock_guard<std::mutex> lock(owner->mutex);
                --owner->in_use;
                if(!owner->retired) {
                    for(auto& slot:owner->free)if(!slot){slot=std::move(returning);return;}
                    std::terminate();
                }
            } // Retired frames and their vectors are deleted outside the pool mutex.
        });
    }
    void retire() noexcept {
        std::array<std::unique_ptr<ImageFrame>,3> discarded;
```

```
        {std::lock_guard<std::mutex> lock(state_->mutex);state_-
>retired=true;discarded.swap(state_->free);}
        // Free inactive storage now; external leases remain valid until their last release.
    }
    Stats stats() const {
        std::lock_guard<std::mutex> lock(state_->mutex);
        return {state_->allocated,state_->in_use,state_->acquired,state_->reused,state_-
>exhausted};
    }
private:
    std::shared_ptr<State> state_;
};
```

完整关键源码 src/video/latest_frame_buffer.cpp

```
#include "video/latest_frame_buffer.h"
#include "common/image_profile.h"
void LatestFrameBuffer::update(FramePtr frame) {
    const auto begin=ImageProfile::Clock::now();
    {std::lock_guard<std::mutex> lock(mutex_);latest_frame_.swap(frame);}
    const double swapped=ImageProfile::ms(begin);
    const auto old_id=frame?frame->frame_id:0;
    const auto release_begin=ImageProfile::Clock::now();frame.reset();
    const double released=ImageProfile::ms(release_begin);
    imageSample("latest_publish_swap",swapped,old_id);
    imageSample("latest_release_old",released,old_id);
}
LatestFrameBuffer::FramePtr LatestFrameBuffer::getLatest() const {
    const auto begin=ImageProfile::Clock::now();FramePtr frame;
    {std::lock_guard<std::mutex> lock(mutex_);frame=latest_frame_;}
    const double duration=ImageProfile::ms(begin);
    imageSample("latest_get",duration,frame?frame->frame_id:0);
    return frame;
}
void LatestFrameBuffer::clear(){update(nullptr);}
```

完整关键源码 src/video/image_preprocessor.cpp

```
#include "video/image_preprocessor.h"
#include <algorithm>
#include <cmath>
#include <cstdint>
namespace { constexpr size_t kWidth=512,kHeight=512,kImageBytes=kWidth*kHeight*3;
constexpr uint32_t kNV12=0x3231564e;
inline uint8_t toByte(float v) {
    return static_cast<uint8_t>(std::max(0.0f, std::min(255.0f, std::round(v))));
}

// MVP CPU preprocessing: packed NV12, BT.709 limited range -> RGB, resize 512x512.
// Unlike PIL/Lanczos this uses bilinear Y and nearest chroma; quality validation is pending.
// Only read LatestFrameBuffer's owned ImageFrame (do not access mmap after QBUF).
}
bool frameToRgb512Reference(const ImageFrame& f, std::vector<uint8_t>& rgb, std::string&
error) {
    if (f.pixel_format != kNV12 || f.width == 0 || f.height == 0 ||
        (f.width % 2) || (f.height % 2)) {
```

```
        error = "Expected packed NV12 frame with even width/height";
        return false;
    }
    // VideoCapture concatenates mmap plane payloads. Current IMX415 returns one plane.
    if (f.plane_sizes.size() != 1 || f.plane_strides.size() != 1) {
        error = "This first VLM adapter expects one packed NV12 plane";
        return false;
    }
    const size_t stride = f.plane_strides[0];
    const size_t width = f.width;
    const size_t height = f.height;
    if (stride < width || stride > 16384 || width > 8192 || height > 8192 ||
        stride * (height + height / 2) > f.data.size()) {
        error = "Invalid NV12 stride/data size";
        return false;
    }
    const uint8_t* yPlane = f.data.data();
    const uint8_t* uvPlane = f.data.data() + stride * height;
    rgb.resize(kImageBytes);
    for (size_t dy = 0; dy < kHeight; ++dy) {
        const float sy = (static_cast<float>(dy) + 0.5f) * height / kHeight - 0.5f;
        const float fy = std::max(0.0f, sy);
        const size_t y0 = std::min(height - 1, static_cast<size_t>(fy));
        const size_t y1 = std::min(height - 1, y0 + 1);
        const float wy = std::min(1.0f, fy - y0);
        for (size_t dx = 0; dx < kWidth; ++dx) {
            const float sx = (static_cast<float>(dx) + 0.5f) * width / kWidth - 0.5f;
            const float fx = std::max(0.0f, sx);
            const size_t x0 = std::min(width - 1, static_cast<size_t>(fx));
            const size_t x1 = std::min(width - 1, x0 + 1);
            const float wx = std::min(1.0f, fx - x0);
            const float yy = (1-wy) * ((1-wx) * yPlane[y0*stride+x0] + wx *
yPlane[y0*stride+x1]) +
                                    wy  * ((1-wx) * yPlane[y1*stride+x0] + wx *
yPlane[y1*stride+x1]);
            const size_t ux = (std::min(width - 1, static_cast<size_t>(fx)) / 2) * 2;
            const size_t uy = std::min(height - 1, static_cast<size_t>(fy)) / 2;
            const float u = float(uvPlane[uy*stride+ux]) - 128.0f;
            const float v = float(uvPlane[uy*stride+ux+1]) - 128.0f;
            const float Y = 1.164383f * (yy - 16.0f);
            const size_t out = (dy*kWidth+dx)*3;
            rgb[out+0] = toByte(Y + 1.792741f*v);
            rgb[out+1] = toByte(Y - 0.213249f*u - 0.532909f*v);
            rgb[out+2] = toByte(Y + 2.112402f*u);
        }
    }
    return true;
}


bool Rgb512Preprocessor::convert(const ImageFrame& f,std::vector<uint8_t>& rgb,std::string&
error) {
    if (f.pixel_format != kNV12 || f.width == 0 || f.height == 0 ||
        (f.width % 2) || (f.height % 2)) {
        error = "Expected packed NV12 frame with even width/height";
        return false;
    }
    // VideoCapture concatenates mmap plane payloads. Current IMX415 returns one plane.
    if (f.plane_sizes.size() != 1 || f.plane_strides.size() != 1) {
        error = "This first VLM adapter expects one packed NV12 plane";
        return false;
    }
```

```
    const size_t stride = f.plane_strides[0];
    const size_t width = f.width;
    const size_t height = f.height;
    if (stride < width || stride > 16384 || width > 8192 || height > 8192 ||
        stride * (height + height / 2) > f.data.size() ||
        f.plane_sizes[0] < stride*(height+height/2) || f.plane_sizes[0] > f.data.size()) {
        error = "Invalid NV12 stride/data size";
        return false;
    }

    if(width_!=width || height_!=height) {
        for(size_t dx=0;dx<kWidth;++dx) {
            const float sx=(static_cast<float>(dx)+0.5f)*width/kWidth-0.5f;
            const float fx=std::max(0.0f,sx);
            auto& x=columns_[dx];x.x0=std::min(width-1,static_cast<size_t>(fx));
            x.x1=std::min(width-1,x.x0+1);x.weight=std::min(1.0f,fx-x.x0);
            x.uv=(std::min(width-1,static_cast<size_t>(fx))/2)*2;
        }
        for(size_t dy=0;dy<kHeight;++dy) {
            const float sy=(static_cast<float>(dy)+0.5f)*height/kHeight-0.5f;
            const float fy=std::max(0.0f,sy);
            auto& y=rows_[dy];y.y0=std::min(height-1,static_cast<size_t>(fy));
            y.y1=std::min(height-1,y.y0+1);y.weight=std::min(1.0f,fy-y.y0);
            y.uv=std::min(height-1,static_cast<size_t>(fy))/2;
        }
        width_=static_cast<uint32_t>(width);height_=static_cast<uint32_t>(height);
    }
    const uint8_t* yPlane=f.data.data();const uint8_t* uvPlane=yPlane+stride*height;
    rgb.resize(kImageBytes);
    for(size_t dy=0;dy<kHeight;++dy) {
        const auto& y=rows_[dy];const float wy=y.weight;
        for(size_t dx=0;dx<kWidth;++dx) {
            const auto& x=columns_[dx];const float wx=x.weight;
            const float yy=(1-wy)*((1-
wx)*yPlane[y.y0*stride+x.x0]+wx*yPlane[y.y0*stride+x.x1])+
                           wy*((1-wx)*yPlane[y.y1*stride+x.x0]+wx*yPlane[y.y1*stride+x.x1]);
            const float u=float(uvPlane[y.uv*stride+x.uv])-128.0f;
            const float v=float(uvPlane[y.uv*stride+x.uv+1])-128.0f;
            const float Y=1.164383f*(yy-16.0f);const size_t out=(dy*kWidth+dx)*3;
            rgb[out+0]=toByte(Y+1.792741f*v);
            rgb[out+1]=toByte(Y-0.213249f*u-0.532909f*v);
            rgb[out+2]=toByte(Y+2.112402f*u);
        }
    }
    return true;
}
```

5.3 关键 C++ 逐句解释与审查

完整文件见 stage5_03_source_appendix.md，此处解释新增或改变的关键语句；原有长篇注释保留在完整源码，不以本表代替源码。行号以最终附录为准。所有权安全依据是 C++ 强引用和锁的规则，实测只是有限场景证据。

OwnedFramePool：有界所有权，不能用 use_count 猜测独占

语句	解释与约束
`struct State`	独立于 CameraService 的池状态，外部 lease 可以跨 Camera 停止存活，不能捕获 Camera 的裸 `this`。
`std::array<std::unique_ptr<ImageFrame>,3> free`	最多三个自有帧对象。unique_ptr 表示空闲对象只有池拥有；不是三个驱动 MMAP Buffer。
`allocated/in_use/acquired/reused/exhausted`	分别统计当前代创建过的帧、未归还 lease、取得次数、复用次数和满池次数；都由同一个 mutex 保护。
`bool retired`	停止后拒绝新 lease，归还帧改为释放，不能把旧帧加入下一代池。
`state_(make_shared<State>())`	让状态可被 lease 的 deleter 延长寿命；池对象自身的生命周期与状态生命周期分离。
删除拷贝构造和赋值	禁止两个外层池意外共享同一停止职责。
`~OwnedFramePool(){retire();}`	析构关闭入口、释放空闲缓存，不销毁仍被消费者持有的像素。
`unique_ptr<ImageFrame> owned`	从空闲列表取出到局部独占对象，后续控制块创建失败也有明确回收方式。
`lock_guard lock(state_->mutex)`	只保护池管理数据，不包住预处理、RKNN 或像素拷贝。
`if(retired)return {}`	已关闭的池不能重新发放帧。Camera 重启创建新 State。
`for(slot) if(slot){owned=move(slot);...}`	将空闲对象转移出来，不复制 ImageFrame 的 vector，空槽不会与使用者同时写同一帧。

语句	解释与约束
`if(allocated==free.size()){++exhausted;return {};}`	满池返回空而不额外分配第 4 帧。Camera 对应 DQ/QBUF 丢弃并计数。
`owned=make_unique<ImageFrame>();++allocated`	仅没有空闲对象且未达到三帧时创建对象；分配抛异常则尚未递增成功分配数。
`++in_use;++acquired`	已获得一个将交付给调用方的独占帧，强引用控制块创建失败的 deleter 也会减少 in_use。
`shared_ptr(owned.release(), [state=state_](...) mutable noexcept {...})`	每次 lease 创建新控制块。帧地址可以复用，旧 weak_ptr 却不能锁定下一次 lease。shared_ptr 构造分配失败也会调用传入 deleter；因此必须在池锁外构造。
`auto owner=move(state)`	最后一个强引用释放时把 deleter 捕获的状态转移到局部。旧 weak_ptr 的控制块即使仍在，也不会把整个池缓存长期留住。不是等待所有 weak_ptr 析构才回收帧。
`unique_ptr returning(frame)`	原始指针重新进入独占 RAII 所有权；退出函数时若没有放回池就自动释放。
`--owner->in_use`	归还数与取得数配对，锁内修改。
`if(!retired) for(empty slot) slot=move(returning)`	活跃池接收归还对象，保留 data/stride/size vector 容量，下次取得时才允许写入。
`std::terminate()`	仅表示内部不变量被破坏：最多三个已分配对象的池不可能在归还时没有空槽。不是正常满池策略，满池发生在 acquire。
retire 分支离开锁后 `returning` 析构	大块像素存储释放不在池 mutex 内，减少持锁时间。
`discarded.swap(state_->free)`	retire 锁内仅交换 unique_ptr 数组；局部数

语句	解释与约束
	组在解锁后析构。正在被 VLM 持有的帧不在 free 数组。
`Stats stats() const`	加同一 mutex 后按值复制计数，不暴露可变内部引用。

边界：池构造/销毁必须遵守外层生命周期，不能在另一个线程销毁同一个 Pool 对象的同时调用它的成员；外部 lease 可以晚于 Pool 对象销毁。Camera 的 start/stop 由已有 lifecycle_mutex 串行化，stop join 后才 retire。

VideoCapture：减少分配，保留 QBUF 前的所有权边界

语句/函数	逐句含义
`captureFrame(...) -> captureFrameImpl(&output,false,...)`	保留旧接口成功后才替换输出的语义，供回退路径与原调用者使用。
`captureReusableFrame(...) -> captureFrameImpl(&output,true,...)`	调用者保证 frame lease 独占；直接填入其已有容量，失败可能留下半成品，Camera 不得发布失败帧。
`discardFrame(...) -> captureFrameImpl(nullptr,true,...)`	仍正常 poll、DQBUF、QBUF，避免驱动停止流转，但不复制像素；只用于池满时。
`poll` 的开始/结束埋点	单独记录等下一帧的时间，不混入图像计算时间。超时仍沿用原错误/超时分类。
`std::array<v4l2_plane,VIDEO_MAX_PLANES>`	固定描述数组放栈上，消除旧 vector 的每帧堆分配；不是把 12 MB 像素放在线程栈上。
DQBUF 成功后 sequence 差值	无符号差值可处理序号回绕；只统计连续捕获中的驱动序号缺口，不把主动 pool discard 和 latest 替换混成一个数。
`ImageFrame temporary; ImageFrame&`	旧语义使用临时对象，新语义直接使用独占

语句/函数	逐句含义
frame=reuse?*output:temporary	lease。没有把一个 ImageFrame move 赋值给自身。
clear() metadata/data	清 size，保留 capacity，不清空底层整幅数据。随后重新填入全部有效字节，旧帧不能在这时还有读者。
reserve(total_bytes)	只有容量不足才扩容；首次三个池槽分别热身，后续通常不增长。capacity 指标不等于全进程 malloc 总数。
insert(end,start,start+used)	每个有效 plane 的 CPU 显式拷贝仍然存在；mmap 指针不能跨 QBUF 借给 VLM。
bytesused-data_offset 与 plane 记录	按有效负载复制，不使用映射最大长度当作有效图像大小；保留 stride 和 plane_sizes 作后续校验。
QBUF	只有数据已复制到自有帧才把驱动 Buffer 归还；discard 分支不读取像素，直接归还。
if(!reuse) *output=move(temporary)	旧接口只在成功后提交结果，vector 的存储转移，不发生第二次整帧拷贝。
capture_stats_={}	每次初始化清除上一代驱动计数，防止重启把不同流的序号连接起来。

CameraService、LatestFrameBuffer 与退出

start 先锁生命周期、join 已退出的旧线程、shutdown 旧设备、clear latest、清统计。在 try 内解析 STAGE5_CAMERA_POOL，只接受 0 和 1，然后创建新池、初始化设备、设置 running、创建线程。非法配置成为可观察的 start=false/last_error；不在工作线程中才抛一个启动配置错误。

captureLoop 每次 acquire 独占可写 lease，成功才 captureReusableFrame；池满调用 discardFrame，并增加 pool_discarded。捕获成功后转换为 shared_ptr<const ImageFrame> 发布，

不再通过可写别名继续修改。回退使用旧 captureFrame 和 make_shared(move(frame))。Stats 在采集线程写，外部按已有 stats_mutex 读；底层 CaptureStats 只有采集线程或 join 后的 stop 访问。

LatestFrameBuffer::update 的 swap 和 getLatest 的 shared_ptr 复制在 mutex 内；旧引用 reset 在锁外。getLatest 取得的是强引用，不是某个像素裸指针。重置 latest 不会使 VLM 已取得的 frame 失效。这里原本没有 NV12 memcpy，本节没有宣称消除一个并不存在的拷贝。

stop 保持 running=false→join→shutdown→clear→统计→retire 的顺序。循环尾也会 clear/shutdown，依赖 5.1 已验证的幂等关闭。池停止时空闲对象立即释放；外部 lease 可以继续读，最后归还时因 retired 被释放。重启不会重新使用旧 State。没有新增 Worker，也没有改变 5.2 的推理 FIFO 或信号处理线程。

Rgb512Preprocessor：算术保持一致，缓存坐标

语句	含义		
验证 NV12、非零偶数宽高	UV 为 2×2 子采样，奇数布局不在当前支持范围。		
验证单 plane、stride、尺寸、data 与 plane_sizes	只接受一块打包 NV12，限制尺寸避免不受控索引；candidate 比 reference 多检查 plane_sizes 有效界限。		
`if(width_!=width		height_!=height)`	输入尺寸不变时不重复为每个输出像素计算坐标。stride 不进入坐标表，真正读取时仍使用本帧 stride。
`(dx+0.5f)*width/512-0.5f`	保留半像素中心变换和原 float 运算顺序，不能用另一个插值库替换		

语句	含义		
	后假定像素相同。		
max(0, sx)、min(width-1,...)	边界采样钳制；x1/y1 不越界。		
缓存 x0/x1/weight/uv 与对应 y 字段	每个方向 512 项 std::array，无额外堆分配；UV 仍用旧算法的最近邻规则。		
rgb.resize(512*512*3)	复用 Impl 的 vector 容量；首次增长初始化的成本包含在融合预处理里，热身后 size 不变。		
yy 双线性表达式	按原次序混合 Y，保留 float 精度、舍入与编译选项；测试在本地与 ARM 分别比较，不把跨编译器相同当成无需验证的保证。		
u/v -128, 1.164383*(yy-16)	BT.709 limited-range 数值约定，仍与旧路径一致。与设备 Full Range 元数据的既有差异另记。		
三个 toByte 写入 RGB	R/G/B 顺序、饱和/舍入沿用旧函数，没有中间全尺寸 RGB，也没有 crop。		

RK3576VLMBackend：输入与输出寿命

Impl 新增 cached 开关、坐标表和两个复用 vector；由 VLMEngine 已有 mutex 串行保护。initialize 在 Runtime 创建前拒绝未知预处理模式。reference 模式使用局部 RGB/embeddings vector，保留旧分配路径。

`auto& rgb=...` 选择持久或局部存储；`capacity()<kImageBytes` 只记录容量增长。预处理完成后栈上 `rknn_input img{}` 零初始化使 pass_through 保持 0；设置 UINT8、NHWC、786432 B、buf=rgb.data()。应用没有另一个 RGB memcpy。rknn_inputs_set 与 rknn_run 各自计时，闭源 SDK 的转换/拷贝不被当成“没有”。

`RknnOutputOwner` 先构造，outputs_get 成功才 acquired=true；校验 result.buf 和浮点输出精确大小。`embeddings.assign(ptr,ptr+36864)` 在 owner 存活时复制，随后检查所有元素 finite。正常、早返回和 assign 抛异常都沿用 5.1 的 RAII 输出释放；释放后不再使用 ptr。持久 embedding vector 不因 SDK 输出释放失效。诊断 dump 检查文件状态，失败返回错误，不伪造比较文件。

RKLLM 同步调用借用 embedding.data()；n_image_tokens=64、n_image=1、height/width=512、提示模板保持不变。栈上的 GenerationState 覆盖整个同步 rkllm_run，未改为异步回调；5.1 的 callback/destroy 顺序保持。Worker 停止并 join 后 Engine 才 shutdown，Impl close 依次 destroy LLM 和 RKNN 后成员 vector 才析构。

埋点、诊断与测试的界限

ImageProfile 的静态初始化由 C++ 保证只完成一次。enabled 在构造后不再改变，多个线程用同一 mutex 写 JSONL；计时结束后才序列化和 flush。诊断 I/O 仍可能影响整个系统负载，上层 wall time 不能与内层计时直接相减得到“纯计算”。打不开或写失败有 ERROR，测试 harness 明确检查，不把诊断失败当成功。该 logger 不保存 frame 引用。

memory_probe 复用生产 CameraService 和 VLMEngine，不另建推理框架；固定帧从原始 fixture 读入，真实 Camera 模式保存前三帧。首次保存会增加 Frame Age，因此主比较剔除前三个请求。rga_probe 是独立手动实验目标，不被完整语音调用；其三条硬件路径实际失败，不能以编译或 imcheck 成功替代图像正确性。

与实现对应的测试

image_pipeline_tests 在 Fake Camera 与实际 CPU 代码上检查：尺寸/padding/非法布局、RGB 与 reference 精确相等、颜色通道与上下方向、复用 capacity、三槽满载、旧 weak_ptr、持有帧期间更新、池析构后 lease、retire、Camera stop/restart、非法启动配置与主动 discard。它不包含 V4L2 ioctl、真实 DMA 或 RKNN 硬件。

async_pipeline_tests、llm_worker_tests、asr_pipeline_tests 保留原确定性 Fake Worker/队列测试；ARM64 CTest 的真实 SDK 链接并不把这些 Fake 变成 NPU 推理测试。另以 memory_probe、固定帧 SDK 推理、真实语音与信号测试提供硬件证据。没有运行 ASan/TSan 或长期泄漏检测，不给出数学意义上的无竞态/无泄漏证明。

5.3 指标	优化前	优化后	统计边界
融合预处理平均/P50/P95 ms	11.052/10.540/12.640	8.325/8.378/9.495	各 n12，含首次调用
显式 NV12 复制平均 ms	9.459	9.736	每帧仍一次全帧复制，没有加速
Frame Age 平均 ms	26.851	28.384	各 n9，剔除前三个保存 fixture 请求，未改善
RKNN run 平均 ms	1160.708	1163.491	各 n12，模型计算没有加速
Vision 进程采样 RSS 峰值 MiB	554.426	552.043	非完整多模态进程，不宣称大幅省内存

预处理平均下降 24.67%，约 2.727ms，不等于完整语音按比例加速。暖态源码可见每帧分配点 5 降 1，最终 636 次 acquire、633 次复用、3 帧创建、0 满池，停止 in_use=0；每次 lease 控制块仍分配，不是零分配。RGA 三条硬件路径首 job 失败，未用于生产；未实现端到端零拷贝。NV12 元数据 Full Range 与原 limited 算法的既有差异、模型内部 mean/std 未反查等限制保留，优化只要求与原 CPU 路径回归一致。

5.3 面试题与完整参考答案

1. mmap 摄像头是否就是零拷贝？

不是。mmap 把驱动 Buffer 映射到用户虚拟地址，减少 read 系统调用搬运，但本工程在 DQBUF 和 QBUF 之间仍把 NV12 复制到 ImageFrame::data。此后 SDK 的 inputs_set 还可能转换/传送。要声称端到端零拷贝，必须证明各消费者共享同一合适的存储、同步与所有权都成立，且没有暗含 staging memcpy；不能仅凭一个 mmap 或 DMA-BUF FD。

2. 为什么不直接把 V4L2 指针交给异步 VLM？

QBUF 之后驱动可以立即重写 Buffer，而推理可能持续数秒。shared_ptr 只管理包装对象，不能约束驱动重用。可以建立 dequeue lease 并延迟 QBUF，但这会占用有限驱动槽、影响采集；还需要停止时处理所有外借 Buffer。5.3 选择保留一次所有权拷贝，池化自有帧，风险更小。

3. shared_ptr 的 use_count()==1 能证明可以覆盖吗？

不能用一次计数观察替代同步协议，其他线程可能从 weak_ptr.lock 取得强引用。若复用同一个控制块，旧 weak_ptr 甚至可能在帧被重新填充后读到另一代数据。本实现每次 lease 生成新控制块，只有最后一个强引用消失、deleter 执行后才回池；旧 weak_ptr 永久过期，地址复用不会复活旧 lease。

4. 有界三帧池如何避免读写竞态？

空闲帧由 unique_ptr 独占，acquire 将它移出池后才填像素，发布后只交付 shared_ptr<const ImageFrame>。采集线程不保留可写别名。latest 与 VLM 可以共享一帧，最后强引用释放才归还。三槽支持一个推理持有帧、一个 latest 和一个正在采集的帧；若额外消费者保留更多帧，池满只 DQ/QBUF、计数丢弃，不无界分配。池大小是应用策略，不能假定任意消费者都不触发背压。

5. 为什么 shared_ptr 自定义 deleter 要在 mutex 外构造？

shared_ptr 的控制块分配可能抛 bad_alloc；接受裸指针和 deleter 的构造失败时会调用 deleter。如果此时仍持池 mutex，deleter 再锁池会死锁。因此在锁内完成槽位和计数转换，锁外构造控制块；失败也由同一归还逻辑恢复 in_use。

6. Camera.stop 后，正在读旧帧的线程怎么办？

停止采集、join、关闭驱动、clear latest 后 retire 池。空闲像素立即释放；外部强引用仍保护已取得的自有帧。State 被 lease 延长寿命，归还时发现 retired 就直接删除。Camera 重启用新 State，不接纳旧代帧。消费者必须持有 FramePtr，不能只提取 data() 裸指针后丢掉所有者。

7. stride 与 width 为什么不能互换？

width 是有效像素宽度，stride 是相邻行起点间的字节跨度，可能包含 padding。当前单 plane NV12 的 Y 起点 y*stride，UV 起点 stride*height，UV 同样按 stride 走行。本板实测 stride=3840=width，但测试额外验证 padding=16。多 plane、独立 UV stride、垂直 padding 不能套用这个公式，需要新的格式元数据和验证。

8. RGB 复用为什么必须考虑 Engine 锁与 SDK？

一个 vector 复用意味着其内容会在下次请求覆盖；若两个推理同时使用它，SDK 可能读取正在改变的输入。当前 VLMEngine mutex 串行化 backend infer，inputs_set→run→outputs_get 都在本次调用内完成；RGB 在 SDK 使用期间不 resize/reassign。Embeddings 独立复制后同步 rkllm_run，直到返回才允许下一次覆盖。若未来改 SDK async，必须重新设计事件和多 buffer 生命周期，现有结论不能直接继承。

9. NV12 转 RGB 优化后尺寸相同，为何还要逐像素比较？

矩阵 BT.601/709、limited/full、RGB/BGR、插值半像素坐标、UV 采样、饱和与舍入都能改变结果。相同 512×512 不等于相同模型输入。5.3 先比较固定原始帧的 RGB 字节，再比较 64×576 float Embeddings 和回答；还用合成红色/方向图确认通道与上下关系。真实画面用户确认补充语义证据，但不替代色彩标定。

10. 如何判断缓存坐标是有效优化？

先在旧算术上建立基线，保持模型、帧、编译模式一致，分离 poll 等待和预处理计算。缓存每个输出行/列的坐标，消除重复的浮点除法/边界计算；对相同输入验证像素。报告样本数、平均、nearest-rank P50/P95、最小/最大，并承认小样本和系统负载影响。总延迟还受约秒级 RKNN、语言生成和 TTS 影响，不能把几毫秒预处理改善说成完整交互等比例加速。

11. DMA-BUF、物理连续内存和缓存一致性是什么关系？

DMA-BUF 是共享内存对象及其 FD/同步约定，不自动保证物理连续；system heap 通常提供可通过 IOMMU 映射的页，是否支持要由实际硬件驱动/Runtime 确认。CPU 映射读写 DMA 缓冲要遵守 begin/end CPU access 或相应 sync ioctl。导入 FD 成功只说明对象被接受，不证明硬件处理成功。本节 system heap FD 实验仍在首个 RGA job 失败，不能把它描述为完成 DMA 摄像头链路。

12. RGA 校验和 import 成功，为什么运行仍失败？

参数校验和导入是不同阶段，硬件提交还依赖实际驱动、地址映射、格式组合、stride/缩放能力和运行状态。本板虚拟地址、DMA fused、DMA resize 三条路径均在第一个 job 报错，并有内核硬件超时。证据不足以指定唯一根因；正确做法是保留日志、停止扩大试验、保持 CPU 生产路径，不把驱动自己 soft reset 当作应用修复成功。

13. 为什么 RSS 上升不一定是泄漏，但也不能忽略？

池保留三帧容量和 RGB/embedding 缓存，允许以常驻内存换取分配减少；allocator 与 SDK 也可能缓存。泄漏指不可达或不再需要的资源未释放，不能仅凭单个 RSS 值判断。需要观察 bounded 峰值、停止时归还计数、设备 FD 关闭、重复生命周期和长期趋势。5.3 只有有限轮次采样和生命周期测试，没有长期泄漏工具证据，因此明确保留风险，并公开峰值增加的代价。

14. 如何保证统计不“美化”结果？

保留原始版本、所有失败轮次、退出码、输入元数据和测量脚本。区分源代码可见分配事件与 malloc 总次数；区分 Host Frame Age 与曝光时间；融合操作不拆出虚构独立耗时；

P95 小样本常等于最大值。排除前三个保存 fixture 的 Frame Age 时必须写明规则并保留原数据。RGA 没有成功样本就标未测量，SDK 内部未知就标未知。

15. drain 与 cancel_pending 的安全退出区别是什么？

两者先关闭新输入，再处理已接收工作。drain 排空有效的已接收任务；cancel_pending 取消等待执行或尚未提交到下游的任务，同时允许已经进入 SDK 的调用返回。取消不是在任意指令点破坏 Runtime。之后按 ASR→推理 Worker→TTS→设备/模型销毁的依赖顺序 join；信号处理机制沿用 5.1，不在异步 signal handler 中调用 mutex、日志、析构或 SDK。强杀只能是超时后的失败保护，不能证明优雅退出。

6 第五阶段 5.4 统一 Profiling 和 TTS 优化

统一使用 steady_clock，Request ID 随 AudioSegment、TaskOptions 及完成回调传递。scope 恢复线程局部上下文，SDK 回调显式携带共享 Trace，没有全局无界请求 map。Frame ID 和 Request ID 不同，不能拼错。每次请求记录 VAD 检测与端点、ASR 队列与推理、路由、推理队列/执行、首文本 chunk、整段答案、TTS 队列/合成、首音频回调、自有 PCM、ALSA 首次成功提交及 drain。

SDK 首个非空文本 chunk 不等于精确首 token；首音频 callback 只保证一批借用 float，当前实现仍等待完整生成再播放。ALSA 首次 writei 提交与 drain 不是耳机真实声学输出。ASR API 包括 stream、AcceptWaveform、Decode 与取文本；VAD 采样索引给出的是主机时间估算，XRUN 使其无效。所有缺失观测保持未测量，不能用 0 或文本长度推算。

完整关键源码 include/common/request_trace.h

```
#pragma once
#include "common/image_profile.h"
#include <memory>
#include <utility>

// One bounded-lifetime object travels through existing queue/task ownership.
// No central map retains completed requests. Raw timestamps use steady_clock.
class RequestTrace {
public:
    using Clock=std::chrono::steady_clock;
    explicit RequestTrace(std::uint64_t id):id_(id){}
    std::uint64_t id() const noexcept {return id_;}
    void mark(const char* event,Clock::time_point at=Clock::now(),double value=-1) const
noexcept {
        ImageProfile::instance().request(id_,event,at,value);
    }
    void text(const char* event,std::string_view value) const noexcept {
```

```
        ImageProfile::instance().text(id_,event,value);
    }
private:
    const std::uint64_t id_;
};
using RequestTracePtr=std::shared_ptr<RequestTrace>;
inline RequestTracePtr makeRequestTrace(std::uint64_t id) {
    return
ImageProfile::instance().enabled()?std::make_shared<RequestTrace>(id):RequestTracePtr{};
}
inline RequestTracePtr& currentRequestTrace(){static thread_local RequestTracePtr
trace;return trace;}
class RequestTraceScope {
public:
    explicit RequestTraceScope(RequestTracePtr
trace):previous_(std::move(currentRequestTrace())),previous_id_(image_profile_request_id) {

currentRequestTrace()=std::move(trace);image_profile_request_id=currentRequestTrace()?curren
tRequestTrace()->id():0;
    }

~RequestTraceScope(){currentRequestTrace()=std::move(previous_);image_profile_request_id=pre
vious_id_;}
    RequestTraceScope(const RequestTraceScope&)=delete;
    RequestTraceScope& operator=(const RequestTraceScope&)=delete;
private:
    RequestTracePtr previous_;
    std::uint64_t previous_id_;
};
inline void traceMark(const char* event,RequestTrace::Clock::time_point
at=RequestTrace::Clock::now(),double value=-1) noexcept {
    if(currentRequestTrace())currentRequestTrace()->mark(event,at,value);
}
```

完整关键源码 include/common/image_profile.h

```
#pragma once
#include <chrono>
#include <cstddef>
#include <stdexcept>
#include <cstdint>
#include <cstdlib>
#include <fstream>
#include <iostream>
#include <mutex>
#include <string>
#include <string_view>

// Frame IDs and request IDs belong to different namespaces. SDK callbacks carry
// an explicit RequestTrace; ordinary synchronous stages inherit this scoped ID.
inline thread_local std::uint64_t image_profile_request_id=0;

// Optional diagnostic JSONL. Each duration ends before file I/O; no frame data held.
class ImageProfile {
public:
    using Clock=std::chrono::steady_clock;
    static double ms(Clock::time_point start,Clock::time_point end=Clock::now()) {
        return std::chrono::duration<double,std::milli>(end-start).count();
    }
```

```
    static ImageProfile& instance(){static ImageProfile profile;return profile;}
    bool enabled() const noexcept {return enabled_;}
    void sample(const char* stage,double duration,std::uint64_t id=0,std::size_t
bytes=0,std::size_t allocations=0) noexcept {
        if(!enabled_ || (!background_ && image_profile_request_id==0))return;
        try {
            std::lock_guard<std::mutex> lock(mutex_);

output_<<"{\"stage\":\""<<stage<<"\",\"ms\":"<<duration<<",\"id\":"<<id<<",\"request_id\":"<
<image_profile_request_id<<",\"bytes\":"<<bytes<<",\"allocations\":"<<allocations<<"}\n";
            output_.flush();
            if(!output_ && !failed_){failed_=true;std::cerr<<"[ImageProfile ERROR] write
failed\n";}
        } catch(...) {std::cerr<<"[ImageProfile ERROR] exception\n";}
    }
    void request(std::uint64_t id,const char* event,Clock::time_point at,double value=-1)
noexcept {
        if(!enabled_)return;
        try {
            const auto
ns=std::chrono::duration_cast<std::chrono::nanoseconds>(at.time_since_epoch()).count();
            std::lock_guard<std::mutex> lock(mutex_);

output_<<"{\"kind\":\"request\",\"stage\":\"request_event\",\"ms\":0,\"id\":0,\"bytes\":0,\"
allocations\":0,\"request_id\":"<<id<<",\"event\":\""<<event<<"\",\"at_ns\":"<<ns<<",\"value
\":"<<value<<"}\n";
            output_.flush();
            if(!output_ && !failed_){failed_=true;std::cerr<<"[ImageProfile ERROR] request
write failed\n";}
        }catch(...){std::cerr<<"[ImageProfile ERROR] request exception\n";}
    }
    // Complete user-visible text shares the existing JSONL mutex and clock.
    // Console streaming from several threads is not a reliable GUI protocol.
    void text(std::uint64_t id,const char* event,std::string_view value) noexcept {
        if(!enabled_)return;
        try {
            const auto
ns=std::chrono::duration_cast<std::chrono::nanoseconds>(Clock::now().time_since_epoch()).cou
nt();
            std::lock_guard<std::mutex> lock(mutex_);

output_<<"{\"kind\":\"request\",\"stage\":\"request_event\",\"ms\":0,\"id\":0,\"request_id\"
:"<<id
                    <<",\"event\":\""<<event<<"\",\"at_ns\":"<<ns<<",\"value\":-
1,\"text\":\"";
            constexpr char hex[]="0123456789abcdef";
            for(unsigned char c:value) {
                if(c=='"')output_<<"\\\"";
                else if(c=='\\')output_<<"\\\\";
                else if(c=='\n')output_<<"\\n";
                else if(c=='\r')output_<<"\\r";
                else if(c=='\t')output_<<"\\t";
                else if(c<0x20)output_<<"\\u00"<<hex[c>>4]<<hex[c&15];
                else output_.put(static_cast<char>(c));
            }
            output_<<"\"}\n";output_.flush();
            if(!output_ && !failed_){failed_=true;std::cerr<<"[ImageProfile ERROR] text
write failed\n";}
        }catch(...){std::cerr<<"[ImageProfile ERROR] text exception\n";}
    }
private:
    ImageProfile(){
```

```
        const char* path=std::getenv("STAGE5_PROFILE");
        background_=path==nullptr; // Legacy image profiling retains all Camera samples.
        if(!path)path=std::getenv("STAGE5_IMAGE_PROFILE");
        if(path) {
            enabled_=true;output_.open(path,std::ios::out|std::ios::trunc);
            if(!output_){enabled_=false;failed_=true;std::cerr<<"[ImageProfile ERROR] cannot
open log\n";}
        }
    }
    bool enabled_=false,failed_=false,background_=true;
    std::mutex mutex_;
    std::ofstream output_;
};
inline void imageSample(const char* stage,double ms,std::uint64_t id=0,std::size_t
bytes=0,std::size_t allocations=0) {
    ImageProfile::instance().sample(stage,ms,id,bytes,allocations);
}
```

完整关键源码 src/audio/tts_service.cpp bool TTSService::synthesize(

```
bool TTSService::synthesize(
    const std::string& text,
    TTSAudio& output,
    std::string& error)
{
    output = TTSAudio{};
    error.clear();

    if (tts_ == nullptr)
    {
        error = "TTSService is not initialized";
        return false;
    }

    if (text.empty())
    {
        error = "Input text is empty";
        return false;
    }

    SherpaOnnxGenerationConfig gen{};

    // 关键：使用初始化时选定的说话人。
    gen.sid = speaker_id_;

    gen.speed = 1.0f;
    gen.silence_scale = 0.2f;

    using AudioDeleter = decltype(
        &SherpaOnnxDestroyOfflineTtsGeneratedAudio
    );

    using AudioOwner = std::unique_ptr<
        const SherpaOnnxGeneratedAudio,
        AudioDeleter
    >;

    const auto start =
        std::chrono::steady_clock::now();
```

```
    traceMark("tts_generate_begin",start);
    AudioProgress progress;progress.trace=currentRequestTrace();

    // 真正执行 ONNX TTS 推理。
    AudioOwner audio(
        SherpaOnnxOfflineTtsGenerateWithConfig(
            tts_,
            text.c_str(),
            &gen,
            progress.trace?audioProgress:nullptr,
            progress.trace?&progress:nullptr
        ),
        &SherpaOnnxDestroyOfflineTtsGeneratedAudio
    );

    const auto finish =
        std::chrono::steady_clock::now();
    traceMark("tts_generate_end",finish);

    if (!audio ||
        !audio->samples ||
        audio->n <= 0 ||
        audio->sample_rate <= 0)
    {
        error =
            "TTS generation returned invalid audio";
        return false;
    }

    try
    {
        traceMark("tts_pcm_copy_begin");
        // 复制 SDK PCM 到 C++ 自有缓冲区。
        output.samples.assign(
            audio->samples,
            audio->samples + audio->n
        );
        traceMark("tts_pcm_copy_end");
    }
    catch (const std::exception& e)
    {
        error =
            std::string("PCM copy failed: ") +
            e.what();

        return false;
    }

    output.sample_rate = audio->sample_rate;
    traceMark("tts_full_audio_ready",RequestTrace::Clock::now(),static_cast<double>(audio-
>n));

    output.synthesis_ms =
        std::chrono::duration<double, std::milli>(
            finish - start
        ).count();

    // audio 离开作用域时自动调用 SDK 删除器。
    return true;
}
```

完整关键源码 src/audio/alsa_playback_service.cpp bool AlsaPlaybackService::play(

```
bool AlsaPlaybackService::play(
    const float* samples,
    std::size_t sample_count,
    std::string& error)
{
    error.clear();

    if (pcm_ == nullptr)
    {
        error = "ALSA device is not initialized";
        return false;
    }

    if (samples == nullptr || sample_count == 0)
    {
        error = "Invalid audio samples";
        return false;
    }

    // 1. 先在内存中完成全部格式转换,
    // 避免播放期间进行耗时的逐样本转换。
    std::vector<std::int16_t> pcm_data;
    traceMark("playback_convert_begin");

    try
    {
        pcm_data.resize(sample_count);
    }
    catch (const std::bad_alloc&)
    {
        error = "Cannot allocate playback PCM buffer";
        return false;
    }

    for (std::size_t i = 0; i < sample_count; ++i)
    {
        float value = samples[i];

        if (!std::isfinite(value))
        {
            value = 0.0f;
        }

        value = std::max(
            -1.0f,
            std::min(1.0f, value)
        );

        const float scale =
            value < 0.0f ? 32768.0f : 32767.0f;

        pcm_data[i] = static_cast<std::int16_t>(
            std::lround(value * scale)
        );
    }

    // 2. 准备播放设备。
    traceMark("playback_convert_end");
```

```
    int rc = snd_pcm_prepare(pcm_);

    if (rc < 0)
    {
        error = std::string("snd_pcm_prepare: ") +
                snd_strerror(rc);
        return false;
    }

    // 查询 period，以便按照设备缓冲节奏提交数据。
    snd_pcm_uframes_t buffer_frames = 0;
    snd_pcm_uframes_t period_frames = 0;

    rc = snd_pcm_get_params(
        pcm_,
        &buffer_frames,
        &period_frames
    );

    if (rc < 0 || period_frames == 0)
    {
        error = "Cannot query playback period size";
        return false;
    }

    std::size_t offset = 0;
    int consecutive_recoveries = 0;
    int wait_timeouts = 0;
    bool first_submission=true;

    // 3. 持续将音频送入 ALSA。
    while (offset < pcm_data.size())
    {
        const std::size_t remaining =
            pcm_data.size() - offset;

        const snd_pcm_uframes_t frames =
            static_cast<snd_pcm_uframes_t>(
                std::min(
                    remaining,
                    static_cast<std::size_t>(
                        period_frames
                    )
                )
            );

        const auto submit_begin=std::chrono::steady_clock::now();
        const snd_pcm_sframes_t written =
            snd_pcm_writei(
                pcm_,
                pcm_data.data() + offset,
                frames
            );

        if (written > 0)
        {
            if(first_submission){

traceMark("alsa_first_submit_begin",submit_begin,static_cast<double>(written));

traceMark("alsa_first_submit_complete",std::chrono::steady_clock::now(),static_cast<double>(
written));
```

```
        first_submission=false;
    }
    offset += static_cast<std::size_t>(written);

    consecutive_recoveries = 0;
    wait_timeouts = 0;
    continue;
}

if (written == -EAGAIN || written == 0)
{
    const int wait_rc =
        snd_pcm_wait(pcm_, 1000);

    if (wait_rc < 0)
    {
        error = std::string("snd_pcm_wait: ") +
                snd_strerror(wait_rc);
        return false;
    }

    if (wait_rc == 0)
    {
        if (++wait_timeouts > 5)
        {
            error = "ALSA wait timeout";
            return false;
        }
    }

    continue;
}

// EPIPE 表示播放缓冲区欠载。
if (written == -EPIPE)
{
    ++xrun_count_;

    std::cerr
        << "[ALSA XRUN] write offset="
        << offset
        << " total=" << pcm_data.size()
        << '\n';
}

const int recover_rc = snd_pcm_recover(
    pcm_,
    static_cast<int>(written),
    1
);

if (recover_rc < 0)
{
    error = std::string("snd_pcm_recover: ") +
            snd_strerror(recover_rc);
    return false;
}

if (++consecutive_recoveries > 5)
{
    error = "Too many ALSA recoveries";
    return false;
```

```
        }
    }

    // 4. 记录全部数据写入后的 PCM 状态。
    snd_pcm_state_t state = snd_pcm_state(pcm_);

    std::cout
        << "[ALSA write]"
        << " submitted_frames=" << offset
        << " state=" << snd_pcm_state_name(state)
        << '\n';

    // 对短音频：如果还未达到自动启动阈值，
    // 显式启动播放，再进行 drain。
    if (state == SND_PCM_STATE_PREPARED)
    {
        rc = snd_pcm_start(pcm_);

        if (rc < 0)
        {
            error = std::string("snd_pcm_start: ") +
                    snd_strerror(rc);
            return false;
        }
    }

    // 5. 等待 ALSA 将缓冲区中的音频播放完。
    traceMark("alsa_drain_begin");
    rc = snd_pcm_drain(pcm_);

    if (rc < 0)
    {
        if (rc == -EPIPE)
        {
            ++xrun_count_;
        }

        state = snd_pcm_state(pcm_);

        error = std::string("snd_pcm_drain: ") +
                snd_strerror(rc) +
                " state=" +
                snd_pcm_state_name(state);

        return false;
    }

    // XRUN 即使恢复成功，也说明本次播放可能存在断续。
    if (xrun_count_ != 0)
    {
        error = "Playback completed with detected XRUN";
        return false;
    }

    traceMark("alsa_drain_complete");
    return true;
}
```

5.4 关键源码逐句说明与生命周期审查

本文与 完整前后源码附录、性能报告 配套。附录从实际文件导出，不用伪代码替代生产入口。这里解释新增语句的语法、调用次序及边界；既有大段设备初始化和模型参数沿用 5.1～5.3。

操作 1：统一时钟、日志与请求所有权

include/common/image_profile.h 继续使用原来的单例、文件和互斥锁。using Clock=std::chrono::steady_clock 是类型别名，时间不随系统校时跳变。time_since_epoch() 仅供同一进程内相减，不解释为 UTC。duration_cast<nanoseconds> 后输出整数，分析器先做整数相减再转毫秒，避免大绝对值浮点数抵消。

inline thread_local std::uint64_t image_profile_request_id=0 为每个线程保存独立的请求编号；inline 允许头文件被多个翻译单元包含。它不是共享状态，不需要互斥。SDK 自建回调线程不会自动继承这个值，必须显式传 RequestTracePtr。

构造函数先查 STAGE5_PROFILE，不存在才查旧的 STAGE5_IMAGE_PROFILE。background_=path==nullptr 在回退前求值：旧图像诊断仍保留背景 Camera 采样；新请求诊断省去没有请求关联的每帧日志，避免每秒数百条日志干扰 CPU。它没有关闭 Camera 或改变帧池。

request(id,event,at,value) 在锁外把时点转成整数，在 lock_guard 作用域内写一整行再 flush。request_id 和图像样本的 id 分开：一次请求可以使用一帧，多次请求也可以使用同一帧。value 供队列深度、路由或样本数使用，不混作时间。事件名来自编译期常量，不接收未经转义的用户文本。

enabled_、background_ 在单例构造后只读；failed_ 和文件写入受同一把锁保护。写失败显式打印 ERROR，不编造零耗时；它是诊断失败，验收脚本将其列为失败条件。捕获异常避免诊断代码跨 SDK C 回调传播。日志 I/O 会影响相邻区间，微秒级 router 数据是带埋点的墙钟区间，并非无扰动纯计算时间。

include/common/request_trace.h 的 RequestTrace 只有不可变 ID，没有可变阶段表。makeRequestTrace 仅在诊断启用时分配一个控制块；没有中央 map 永久保留请求。shared_ptr 使音频段、排队任务和完成回调各持有自己的强引用，最后一个持有者释放时自动销毁。

currentRequestTrace() 返回函数局部 thread_local shared_ptr 的引用。首次实现用头文件中的 inline TLS shared_ptr，在 MinGW 链接时出现初始化符号重复；改为 accessor 后本地构建通过，原失败日志保留。ARM64 另行构建验证，不能从 Windows 结果推断。

RequestTraceScope 构造时 std::move 取走原 TLS 引用，设置本次引用和数字 ID；析构时恢复上一层。移动 shared_ptr 转移引用，不复制请求对象。禁止复制 scope，防止两次恢复。嵌套同步调用和异常展开均按栈次序恢复；回调需要跨线程时显式捕获 shared_ptr，不能捕获 scope 的地址。

操作 2：沿已有队列传递上下文

include/common/work_control.h 给 TaskOptions 增加 RequestTracePtr trace，未改变 deadline、max_queue_wait、原子取消标志或 disposition 优先级。PipelineAdmission::Ticket::options(max_wait,trace={}) 的第二参数有默认值，旧调用仍有效；std::move(trace) 转移本次局部引用，票据的 deadline/cancelled 仍共享原状态。

AudioSegment 增加同类型成员。音频 vector 的所有权仍随原队列移动，不为埋点再复制音频。AudioBufferQueue::push 先保留一个 trace 引用和实际 enqueued_at；队列关闭时记拒绝，满载覆盖最旧段前记被替换请求。成功 push 仍在同一队列锁下，日志和唤醒沿原流程；分析按时间戳而非文件行出现顺序排序，因为出队线程可能先写日志。

ASRWorker 新增可选 ContextCallback(text,const AudioSegment&)，原 TextCallback 和原构造调用保留。若提供 context callback 就只调用它，避免一次识别发出两次业务请求。const AudioSegment& 只在同步 callback 执行期间有效；入口只复制 trace 引用，不保存 segment 引用，也不保存 samples 裸指针。

出队后创建 RequestTraceScope(segment.trace)，随后记 dequeue、ASR begin/end，并包住原业务 callback。原音频预处理、识别参数及队列关闭逻辑保留。ASRService 内部 stream 创建、AcceptWaveform、Decode 另设标记：外层 API 时间包含这些步骤及文本取回，Decode 区间也不是纯 ONNX 算子时间。

LLMWorker 和 TTSWorker 从既有 TaskOptions 取上下文。记录成功入队时的等待队列深度、真正出队、服务开始及终态；取消和过期在原 disposition 分支记录。工作函数在队列锁外执行，未把模型推理搬入队列临界区。拒绝路径记明确事件，原统计和终态 callback 仍执行。

循环末尾 task=Task{} 释放 inference/done 捕获的票据、上下文和其他对象；scope 的额外引用在本次循环离开时释放。测试通过 weak_ptr 检查 joined 后不再持有上下文。新增的唯一动态对象受原有界队列与在途上限约束，没有无界增长容器。

操作 3：VAD 时间的真实意义

VadAudioWorker 每次喂入原 512 样本窗口后观察 SDK 的 detected 状态，记录上升沿的主机 steady 时点。确认完整 segment 可取时，先记录 endpoint，再复制原有 pre-roll 和语音数据。segment 创建后才有 sequence，因此用 trace->mark(event, earlier_time) 回填已观察到的时点，而不是重新调用 now 冒充起点。

audio_start_estimated 和 audio_end_estimated 用最后一次 read 完成时点与采样索引差回推。只沿用原有无 XRUN/suspend 的 timing_valid 条件；不是声卡 ADC 时间戳，也不是精确人声起止。检测起点、估算起点和预滚起点分别解释，不能相互代替。

可选 STAGE5_CAPTURE_AUDIO_FIXTURES 只在专用测试打开，复制完 VAD 段后写出自有 float 样本及元数据。开始/结束独立计时；写文件会增加 vad_to_queue 和整请求延迟。报告区分留存语音的自然基线与不写 fixture 的后续测量，不拿这点额外 I/O 伪造加速。

操作 4：RKLLM callback 和 VLM 状态

Qwen 的 RunContext 增加 shared_ptr，在同步 rkllm_run 前复制当前上下文。原 mutex 保护 text、failed、first_text_ms。首次非空 chunk 时记录同一个 now；SDK callback 借用的 result->text 仍复制进 string，不能保留该指针。

SmolVLM2 的 GenerationState 增加 mutex、decoder_start、first_text_ms 和 trace。callback 的所有状态读取/写入置于 lock_guard 内；异常展开先销毁 guard，再进入 catch 重新加锁置 failed，避免同线程重复锁。同步 run 返回后主线程先记 decoder_end，再锁 state 检查 finished/failed/text 并移动文本。

state 仍为同步调用栈上的对象，SDK 参数保持 is_async=false。run 返回并确认回调结束之前 state 不析构；destroy 仍在 Worker join 之后。若未来换为异步 SDK，必须重审这一契约，不能照搬栈对象。

首回调是“首个非空文本 chunk”，可能包含多个 token；不宣称精确 token 级 TTFT。VISION 的 inference_begin 到首 chunk 包含预处理、RKNN 和语言 decoder；decoder_begin 到

首 chunk 只含语言阶段，二者不混用。5.3 的 RGB/Embedding 缓存、输出 RAII、输入转换及固定英文视觉问题全部保留。

操作 5：TTS 和 ALSA 的测量边界

`TTSService::initialize` 仍仅启动时创建一次 SDK 对象。候选把默认 num_threads 从 2 改为 4，命令行显式传 2 可回退；speaker、speed、silence_scale、模型和 provider 的生产默认均未改变。没有新增 TTS 合成线程，也没有自动裁剪答案。

`AudioProgress` 持有 trace 与原子 seen 标记。首个 `n>0 && samples!=nullptr` 回调只记录时间和长度，始终返回 1；不持有 SDK 借用 PCM 指针，不在回调播放，不在回调停止或截断生成。`GenerateWithConfig` 是同步调用，progress 活到调用返回。

`AudioOwner` 仍用 unique_ptr 和 sherpa 删除器托管返回音频。无效输出直接失败，PCM vector::assign 异常返回明确错误，owner 随作用域释放，成功也先复制再释放。分别记录 SDK Generate、应用 PCM copy、完整自有音频 ready。callback 给出第一批音频可用时间，但当前播放路径仍等待整个 Generate 返回；不能把 callback 时间当首个 ALSA 提交时间。

诊断开关 `STAGE5_TTS_ORT_PROFILE_CONFIG` 只用于独立 ORT 实验；配置必须存在，所有输出位于新报告目录。它不是生产默认配置，也不进入主公平对照。ORT model_run 能看到模型会话区间，API 减去 model_run 的剩余值包含文本前端、输出处理、调度及诊断开销，不能直接命名为“纯音素化时间”。

ALSA 沿原 float→S16 转换和写入循环。对首次成功 writei 保存“调用前”时点和成功返回时点；只有 written>0 才记首次提交，错误返回不冒充已提交。全段 writei 完成后的旧 `[ALSA write]` 日志仍保留，但不用于首次提交指标。

`snd_pcm_drain` 前后另记事件。`alsa_drain_complete` 表示驱动/ALSA 排空语义，不是麦克风测到的实际声学尾音。buffer=12000、period=3000、start_threshold=6000 的现有配置不改；首次 write 成功也不代表耳机那一刻已经发声。

操作 6：固定输入、回放与信号等待

`include/app/profiling_inputs.h` 仅在显式测试环境变量启用。NV12 元数据检查非零偶数尺寸、stride≥width、上限、最少 NV12 字节数及文件长度，再读入自有 ImageFrame。没有伪造 received_at，所以固定帧 Frame Age 不可用。该强引用在入口作用域活到异步 Worker join 之后，后台 Camera 的 V4L2 重用不会覆盖这个独立 vector。

音频加载限制 16kHz 单声道 float、最长 16 秒、字节数整除 sizeof(float)，并逐样本检查有限数和范围。部署机 ARM64/Windows 均为小端，文件声明 f32le；不能把其他编码直接送入此接口。fixture 写入拒绝已存在的相同 ID 文件。它用于当前受控新建目录，不是面向不可信并发写者的通用存储服务。

完整入口的 on_text(recognized,supplied_trace={}) 在路由前继承 ASR ID。原文本回放没有 trace 时仍分配 next_id。音频回放生产者使用独立 audio_replay_sequence，不与 ASR callback 线程同时修改 next_id，避免新数据竞争。

STAGE5_REPLAY_AUDIO_FILE 走原 ASRWorker、IntentRouter、InferenceWorker、TTSWorker 和真实 ALSA；绕过麦克风与 VAD，必须在输出和报告中注明。固定帧只允许与显式回放和 VISION 一起使用，普通自然语音仍取最新 Camera 帧。

原文本回放等待使用 sleep_until，长达 100 秒的未来输入会推迟信号响应。候选改为每 20ms 检查既有 StopSignals，再处理原定时刻；没有在信号 handler 做锁、iostream、join 或 destroy。main 退出仍先停生产者/ASR，再排空或取消推理，join 后释放 Qwen，再完成/取消 TTS，最后关闭播放、Camera 与 VLM。取消只作用未开始任务，正在执行的 SDK 调用仍等待其返回；有限超时由测试守护独立记录，强制终止不能算优雅退出。

操作 7：验证如何覆盖这些语句

- request_profile_tests：同一 request_id、不同 frame_id、跨真实 std::thread 的显式回调、TLS 恢复、JSONL flush；固定 PCM roundtrip、NaN 拒绝、覆盖拒绝、NV12 stride 检查及无伪造采集时间。
- async_pipeline_tests：保留原满载/取消/到期/交替任务/停止重启测试，增加 trace 在推理、完成回调、Fake TTS、ASR context callback 间传递及 weak_ptr 最终释放。
- image_pipeline_tests/camera_service_tests：继续验证 5.3 Buffer 所有权、复用与并发读取；使用 Fake，不能当真实摄像头或 NPU 结果。
- 独立手动 benchmark：原 SDK 的真实 ASR/VLM/TTS，固定输入、warm-up 和 CSV；不注册为自动 CTest，避免普通 ctest 隐式占硬件或运行数十分钟。
- 板端完整入口：真实设备释放、信号、排空/取消、队列过期与拒绝；自然语音的耳机效果由当前用户确认，不由 success=true 推断。

测试逐项结果、退出码和剩余边界见主报告及 validation_summary.json。没有 ASan/TSan/Valgrind 的全进程硬件证明，不把本轮有限回归写成任意运行时长下绝无泄漏。

操作 8：手动 benchmark、CMake 与分析脚本

stage5_04_tts_bench.cpp 的 argv 长度必须为 5，包括程序名；stoi 后限制线程 1～8、有效重复 1～64。initialize 在循环外调用一次；双层循环外层 iteration，内层中文/英文，顺序固定。iteration0 与 1 分别记录 first_use/warmup，剩余 N 才是 measured。TTSAudio 每轮为局部对象，合成返回后按实际样本数/采样率求音频秒数；RTF 使用毫秒除以音频秒数×1000。逐样本检查 finite、abs 最大值、平方和均方根和逐字节 FNV，不能由 hash 不同直接推断退化。仅首个 measured 保存实际 PCM，其余仍逐行 flush CSV。

stage5_04_asr_bench.cpp 用 SDK Wave 读取器和自定义删除器持有固定 16kHz 单声道输入，同一 ASRService 循环使用，保存识别文本并检查一致。wall 包含调用外层开销，ASR API 时间来自原服务内部。CSV helper 将引号变为双引号，符合 CSV 约定，不用 C++ quoted 的反斜杠逃逸冒充 CSV。

stage5_04_vision_bench.cpp 启动时加载三个固定 NV12 自有帧，循环 i%3 选择 fixture，避免每轮读盘混进模型耗时。2 轮初始调用后 30 个样本，恰好每 fixture10 个；这不代表每张图都独立做两次预热。真实 VLMEngine 和既有模型路径保持。输入 RGB 和 64×576 float 输出可选保存，文件以递增 sample 编号命名，重复使用 frame_id 不会覆盖其他样本。

CMake 复用 multimodal_core/audio/tts 及现有 SDK 传递依赖；target_link_libraries PRIVATE 表示仅该手动程序使用依赖，原库的 PUBLIC 头文件/链接传播仍保留。TTS target 的 STAGE5_BENCH_TRACE=1 只启用诊断探针的上下文，不改变库的模型参数。手动探针只 add_executable，不 add_test，因此 ctest 不会隐式占用麦克风、Camera/NPU 或运行 30 轮大模型。request_profile_tests 才注册为带超时的确定性测试。

stage5_04_prepare.py 把实际字节和 SHA 放入 payload，源目录/快照必须不存在。先从冻结 5.3 副本复制，再逐项核对 before 哈希后覆盖**新的本节副本**；不会覆盖已验收路径。revision 支持 v1/v2/v3，旧产物全保留。baseline v2 仅把默认线程退回 2，其他测量路径保持同源。每个构建 step 记录 argv/stdout/stderr/退出码/磁盘采样；余量低于 500MiB 或超时才终止本次新建编译进程组。

stage5_04_run.py 和 voice_run 逐次发现 HyperX，查询四个实际设备节点是否空闲。采样器只读/proc 和 sysfs，CLK_TCK 来自 sysconf。设备权限不靠更改系统配置；sudo 仅只读 fuser。每个 case 目录独占创建。信号守护核对 exe 和/proc 启动 token 后只对本次 PID 发信号，异常强杀单独标失败。voice_run 的 SSH 传输成功与 child.returncode 分别记录，不相互覆盖。

stage5_04_analyze.py 先按 case+request_id 关联事件，再以整数纳秒差求毫秒；按语言、fixture、phase、route、status 分组，缺少事件保持 None。quantile 采用 nearest-rank，不对稀疏失败样本补零。资源统计分别保存 CPU 单核口径、RSS/HWM、频率和原始 NPU 字符串；不能靠读到 100 就声称硬件满载。ORT 诊断是另一 case，算子 dur 之和不等同总墙钟。

stage5_04_plot.py 从原始统计绘制散点/箱线、RTF 经验 CDF、CPU 曲线/RSS 及请求阶段分解，写 PNG/SVG 可分享文件；不用虚构平滑采样。Gantt 各请求从各自首次事件归零，图题明确这不是所有请求共享的全局起点。

操作 9：最终音频回放错误检查的最小修正

文本回放原本不启动 ASR，因此旧 replay_mode || consumer.ok() 的意图成立；加入真实音频回放后不能继续对所有 replay 跳过 ASR 状态。v3 把通过条件改为 replay_mode ? (!audio_replay_path || consumer.ok()) : (producer.ok() && consumer.ok())：文本回放仍免检查，音频回放必须 consumer.ok，自然语音同时检查生产者/消费者。对应错误输出条件也加入 audio_replay_path。

这只修复新测试入口可能出现的假阳性，不改变正常识别、模型调用或队列。之前有效输入 v2 的性能证据保持有效，但不能冒称 v3 执行；最终自然语音、信号和完整构建另用 v3 复测。未对真实 SDK 注入所有内部故障，故不宣称所有错误路径已实机覆盖。

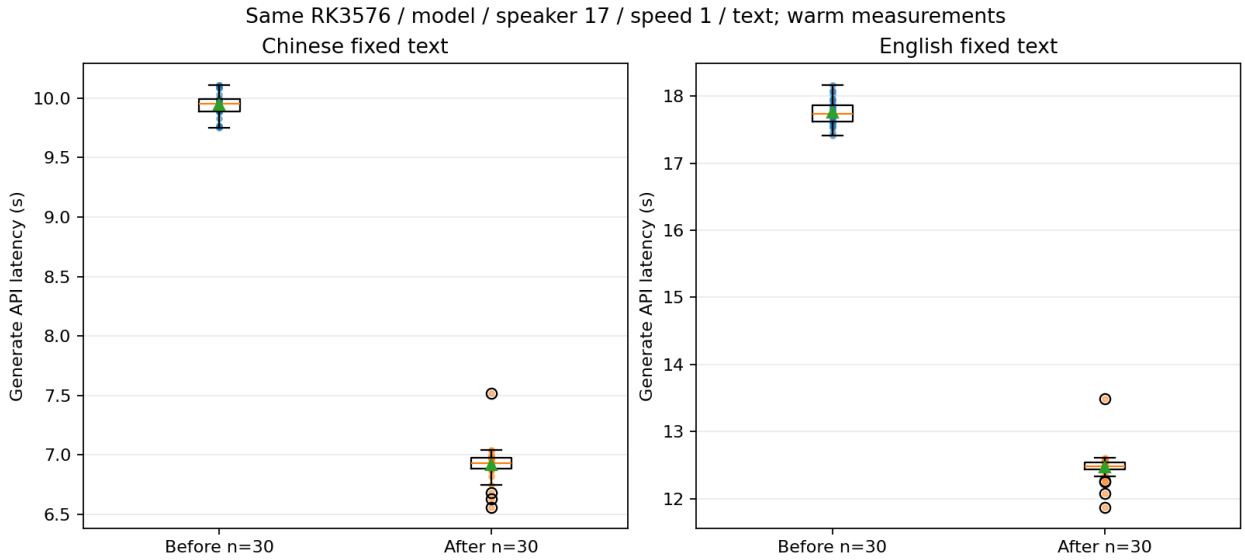
固定文本 每语言 n30	2 线程平均/P50/P95 ms	4 线程平均/P50/P95 ms	平均 RTF 前后
一加一等于二	9945.095/9952.273/10112.211	6920.197/6925.889/7042.744	5.090→3.545
A computer monitor displaying a webpage with some text and images	17758.202/17730.550/18084.155	12476.125/12473.974/12609.867	4.316→3.032

同一台板、同一冻结 5.3 TTS 库和 benchmark、同模型、sid17、speed1、silence_scale0.2 及相同文本；唯一参数差为线程数。每种语言另记 first-use 与 warmup，不混入 30 暖态样本。中文与英文平均合成耗时分别下降 30.42%和 29.74%，RTF 仍大于 1；CPU 约 199.65%到

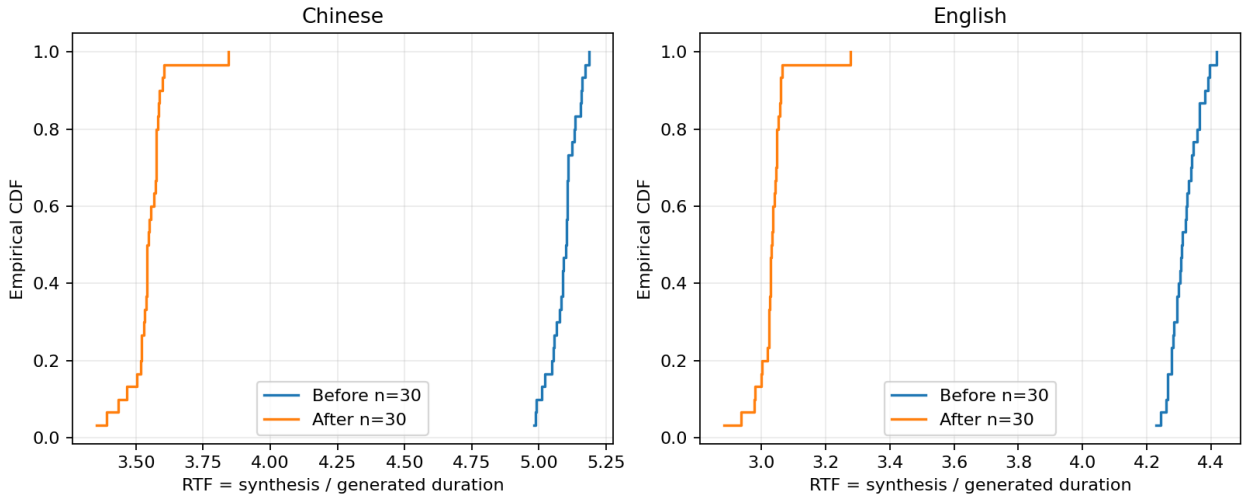
398.44%（单核 100%），最高观测温度 76.692℃到 85.888℃。这是短时合成收益及资源代价，不是所有子系统都变快。

独立 ORT 诊断 6 次中 model_run 累计占 Generate API 约 99.9766%，支持主要瓶颈为计算；文本前端/音素化没有独立回调，残差不能叫纯音素化时间。没有重复初始化，因此没有编造“去掉重加载”的收益。1 线程更慢，6 线程中文更慢且 CPU 更高，原实验保留未采用。没有更换声音、语速、截断回答或增加并发合成实例。

同音频/固定帧三组 VISION→TEXT 有限观测：VISION 从 ASR 入队至 drain 均值 19.914→16.693 秒，TEXT29.630→24.334 秒；两边均使用 60 秒 TTS 等待和 180 秒期限，避免靠过期制造收益。TEXT 模型服务本身 1149.5→1422.1ms 有回退，不能藏在总延迟下降中。生产 12 秒/45 秒默认策略仍保留。



5.4 固定文本暖态合成分布



5.4 RTF 实测分布

5.4 性能工程面试题与参考答案

答案以本工程实际实现为依据，实测数值见性能报告和原始 CSV。没有测到的阶段不补数。

操作 1：为什么不能使用系统时间计算请求延迟？

system_clock 对应日历时间，NTP、人工校时可能使它前跳或后退；两次相减可能失真。steady_clock 保证单调，适合持续时间。本项目在所有 C++ 阶段记录 steady 纳秒，分析时先相减再转毫秒。它没有跨主机 UTC 同步意义；Linux /proc 采样和 Python monotonic 只在资源曲线各自坐标中使用，不靠猜测把绝对 epoch 拼接。日志接收时间也不能代替模型内部事件时间。

追问：CPU frequency 变化是否改变 steady 计时口径？计时仍测实际经过时间，但频率会改变被测计算速度，因此必须同时保留频率、温度和负载。

操作 2：为什么需要 Request ID，frame_id 为什么不能代替？

异步 ASR、模型和 TTS 的日志会交错，按日志行顺序拼接容易把不同请求相加。request_id 从完整语音段开始传递，TaskOptions、回调和 SDK context 持有同一小型 trace。Camera 独立采集，一帧可被不同请求重复使用；frame_id 描述图像，不能作为请求主键。离线分析使用 case_name + request_id，因为每个进程的编号重新开始。

追问：为什么不建全局请求 map？队列和 callback 已经管理请求寿命；再建 map 会产生清理、锁竞争和无界保留风险。本轮只随现有有界任务携带 shared_ptr，完成后释放，分析表在进程外生成。

操作 3：TLS 为什么不能自动解决 SDK 回调关联？

thread_local 是每个线程独立一份，SDK 在自己的线程回调时看不到提交线程的 TLS。本项目同步业务调用用 RAII scope 设置/恢复 TLS，SDK context 则显式复制 RequestTracePtr。callback 内用该指针 mark，不假设线程身份。回调若保存主线程 scope 地址，scope 退出后就可能悬空。

追问：shared_ptr 是否让对象内容自动线程安全？不会。它保证控制块引用计数安全，RequestTrace ID 不可变，日志另有 mutex；VLM 的 text/finished/failed 仍必须用自己的 mutex。

操作 4：TTFT 应该如何定义？

需要先说明起点和终点。对 TEXT，可测 inference_begin 到首次非空文本回调；对 VISION，整个推理起点包含预处理和 RKNN，而语言 decoder_begin 是另一边界。SDK text chunk 可能包含多个 token，本项目只报告首文本 chunk 延迟，精确 token 粒度 TTFT 未测。异步 submit 返回的 0.0xms 是调度开销，不是 TTFT 或模型推理耗时。

追问：为什么不由输出字数推算 token 时间？tokenizer 切分、批次回调和调度都不确定，推算不能替代观测事件。

操作 5：TTS 首批音频就绪，是否等于用户听到声音？

不是。Kokoro callback 只说明 SDK 已生成第一批 PCM；当前代码仍等待整段合成返回，再复制、转成 S16、调用 ALSA。首次成功 writei 是首次音频提交，也受 ALSA 启动阈值和缓冲影响；drain 完成是驱动排空语义。没有设备声学时间戳或外部回录测量，不能称为实际声学延迟。用户确认“听到了”验证真实播放，但不提供毫秒级声学起点。

追问：能否直接在 callback 里异步播放 samples？不能把借用指针交给其他线程，必须复制或获得 SDK 明确的所有权契约；还需重新设计错误、排队和退出语义。本轮不为名义流式播放冒这个风险。

操作 6：如何用 RTF 公平比较 TTS？

RTF=合成墙钟时间/生成音频时长，两者使用同单位。RTF<1 才表示合成快于播放速度。必须同时固定模型、声音、语速、语言、文本和生成参数，分别报告中文/英文；不同回答长度不能直接混合。即便同文本，SDK 的随机生成可能使采样数和 PCM hash 不同，所以记录每次实际长度、RTF、peak、RMS、非有限数。不能通过缩短答案或提高语速制造同条件加速。

追问：RTF 能代表整请求延迟吗？不能。排队、ASR、NPU、复制和播放都在外；整请求需要独立阶段时间线。

操作 7：为什么线程数越多不一定越快？

线程竞争、同步、缓存、内存带宽和大小核差异都会增加成本，持续负载还可能升温降频。本轮筛查 1、2、4、6 线程，不能只挑一条最快数据；候选需足量复测，报告 CPU 占用

和温度。num_threads 会进入 SDK session 配置，但进程还有主线程、Worker 和 Runtime 其他线程，配置值不等于总线程数。实际线程数和 affinity 通过 /proc/PID/task 取证。

追问：为什么不全机改 performance governor？这超出隔离目录授权，会改变测试条件和系统功耗；本轮保留 ondemand 并记录频率，不把未控制变量隐藏掉。

操作 8：如何定位 TTS 队列等待的根因？

先分别测 enqueue→dequeue、Generate、PCM copy、float→S16、write/drain。当前 TTSWorker 单线程先合成整段再完整播放，下一条等待的是前条总服务时间。应用复制只有很小开销时，减少一次复制不可能解释秒级改善；模型合成慢才是主要瓶颈。线程配置优化减少服务时间，可能间接缩短下一条队列等待，但不会把串行队列变成并行。

追问：为何不立即再开一个合成线程？同一个 TTS 实例的并发契约未确认，多实例又增加内存和 CPU 竞争；必须先证明吞吐收益和生命周期安全。本轮保留串行语义。

操作 9：如何区分冷启动、预热和稳态？

初始化加载模型单列；每个文本 iteration0 标 first_use，iteration1 标 warmup，其后样本才进入主统计。英文 first_use 是该文本首次调用，并非整个实例第一次调用，因为中文已先运行。独立 benchmark 保持同一模型实例，避免每轮重载。真实完整入口的样本少、模型共驻且 Camera 在运行，不能与纯 TTS 进程稳态结果混为一张无条件对照表。

追问：为什么 P95 不能随便插值？可以选择插值，但必须统一说明。本项目用 nearest-rank，排序后取 ceil(0.95n)-1；n=3 的 P95 就是最大值，不能声称已精确估计长期尾延迟。

操作 10：如何证明一次优化是实际计算收益？

主 TTS 对照使用同一个直接链接冻结 5.3 TTS 库的 benchmark 二进制，仅线程参数 2/4 不同，输入和 Runtime 相同，保留全部暖态样本。这样不把新埋点或新业务代码差异归到优化。完整链路另用同一留存音频和 NV12 回放验证调度与端到端影响，并披露回放绕过麦克风/VAD。自然语音仅有限交互回归，不把用户说话时长差异解释成性能收益。

追问：先后运行还有什么偏差？热状态、DVFS 和背景负载仍可能变化；保留温度/频率/CPU，说明不是大规模随机交叉试验，不外推任意环境的固定百分比。

操作 11：取消等待与排空退出有什么不同？

排空：关闭输入，已接收任务按原 FIFO 完成；取消等待：标记 ticket，未开始的推理/TTS 以 cancelled 终态完成，不再新增下游任务；已经进入同步 SDK 的调用仍等待安全返回。两者都先让上游不再提交、唤醒等待线程、join，再 destroy Runtime。signal handler 不做锁、日志或资源释放，既有 sigwait 线程把信号转成协作状态。

追问：timeout 后 kill 是否证明退出成功？不是。守护只能对本次确认 exe/start-token 的 PID 发送最终强杀，必须标 forced_kill=true 和退出失败。成功要求未强杀、正常退出码、完整 shutdown 标记、设备重新可用。

操作 12：长回放等待为什么会破坏信号回归？

原 sleep_until 可能一直等未来输入，即使信号线程已经置位，main 也不能及时进入关闭流程。候选按不超过约 20ms 的片段检查 StopSignals，同时保持原定提交时刻。需要测试一个 100 秒后的未来输入，在前一个请求运行时发信号，确认不是恰好回放已经结束才退出。这项改动只影响显式诊断回放，不重构生产 Worker。

追问：频繁 sleep_for 会不会忙轮询？它会主动休眠，不忙占 CPU；20ms 带来有限调度误差，所以排队输入实际 enqueue 时点仍由 steady 记录，不能用计划偏移替代真实提交时间。

操作 13：为什么不能把 NPU sysfs 的 load 直接当利用率？

本板在无模型进程和 CPU-only TTS 时也读到固定 `100@950000000Hz`。它的有效语义未验证，只能保留原字符串和频率。没有可信利用率、带宽或功耗工具时应填未测量；RKNN run 区间、RKLLM 日志和成功结果仍能证明实际调用，但不能反推出硬件利用率百分比。

追问：CPU 使用率为何可能超过 100%？本项目用单核为 100%的进程口径，多核同时运行可以超过 100%；不能与整机归一化 0～100%的 top 模式混用。

操作 14：怎样评估资源泄漏与测试覆盖的限度？

观察 RSS、VmHWM、线程数、fd、设备 fuser、队列终态和 shutdown 顺序，结合 Fake 的停止/重启、weak_ptr 和异常测试。峰值内存高不等于泄漏；Allocator 缓存也可能保留 RSS。有限实机运行后所有进程结束、设备释放只能证明这组场景没有明显残留，不能替代长稳态压力测试或 sanitizer 对所有 SDK 内部路径的证明。

追问：为什么保留失败实验？错误和回退能说明候选选择依据，也防止把慢配置、小样本或工具失败从证据中删掉。原始日志只追加新 case，不覆盖已验收版本。

7 第五阶段 5.5 Qt 和持续稳定性

5.5 的真实问题、热条件、失败记录、所有版本与验收边界以本节报告为准。下面引用的工作记录在正式生成时与最终数据同步；不把较早热对照源码冒充最后 GUI/结构化文本修复的精确版本。

5.5 Qt 集成和最终系统验收

状态：正在执行，不能据此宣告最终验收通过。继承已由用户验收的 5.1～5.4，仅在隔离工程内开发；最终数据将在本报告实测章节追加。

操作 1 范围冻结与源码复核

本地 96 项生产源码先与 5.4 最终 ARM64 v3 部署清单逐字节 SHA256 核对，全部一致后，冻结 171 项源码、测试、工具、配置及文档到 backups/stage5_05_before/。清单为 reports/stage5_05_local/baseline_manifest.json。板端原稳定工程 321 项和 5.1～5.4 四个已验收语音程序分别保存指纹；5.4 源码和构建保留原处。

当前真实调用：VAD 采集线程→容量 4 音频段队列 →ASRWorker/SenseVoice→IntentRouter→PipelineAdmission→单消费者 InferenceWorker→TEXT/Qwen 或 VISION/最新自有 Camera 帧/CPU 缓存预处理 /RKNN/SmolVLM2→容量 2 TTSWorker/Kokoro/ALSA。完整语音入口未经过 RequestProcessor；另一 CLI 入口保留。推理和 TTS 的默认等待/期限、取消和排空契约沿用 5.2，3 槽图像池、融合 CPU 转换/Resize 和 512RGB 缓冲复用沿用 5.3，Request ID 与 steady_clock 埋点沿用 5.4。

5.4 的可复核结论是固定中英文各 30 暖态样本的合成耗时下降约 30.4%和 29.7%，不是整个系统必然同比加速。模型 RTF 仍大于 1，CPU 和温度增加，小时级稳定性当时未验证。RGA 失败、没有 DMA-BUF 零拷贝、精确 token TTFT/声学输出/NPU 利用率未可靠测得等限制继续保留。

操作 2 Qt 与运行环境只读预检

板端为 aarch64、Qt5.15.8 Widgets/Test 开发环境，存在 qmake 和平台插件，GNOME/X11 的 DISPLAY 为:0，DSI 已连接。没有安装软件、迁移原车载工程或修改显示配置。已有工程没有 Qt 目标，因此本节增加默认关闭的 BUILD_QT_GUI，保留原 CLI。

最小方案是 QProcess 适配已验收的完整 CLI：模型加载、音频/Camera 操作和 Worker 均在原后台子进程，Qt 线程只接收 readyRead/finished 事件和更新控件。参数使用 QStringList，不拼接 shell。GUI 日志与诊断事件有界；关闭窗口先 ignore 并请求 SIGINT，收到子进程 finished 后才关闭；超时明确报失败，不把 QProcess 析构强杀当成功。StopSignals 从原入口完整移至共用头文件，仍在 Qt 应用及 SDK 线程创建前阻塞 SIGINT/SIGTERM，普通 sigwait 线程接收，Qt 定时读取状态；没有异步 signal handler 执行 Qt、mutex 或 join。

接口显示 Camera/ASR/LLM/VLM/TTS、开始/停止、2/4 线程、排空/取消、识别、TEXT/VISION、回答、等待队列快照、最近延迟与错误。自动回归模式和 Fake 子进程有明确标签，不冒充真实麦克风或 SDK。窗口截图必须由实际运行的 QWidget::grab 产生，并注明 xcb 实屏或 offscreen 测试。

操作 3 分阶段热稳定性方案

预检读得 little-core thermal passive trip 为 75℃、85℃，critical 为 115℃；其他已读 zone critical 为 115℃。这是当前 Linux 配置，不是芯片额定安全温度证明。本轮将 90℃作为保守测试停止线，同时记录 cooling_device cur_state、scaling_cur_freq/scaling_max_freq；不能仅凭温度超过 passive 或瞬时频率降低就声称热降频。

执行顺序：相同固定音频、相同 NV12、原模型/SDK、相同参数，2 线程 180 秒输入、4 线程 180 秒输入；短测通过后分别 2/4 线程 1800 秒输入。每轮开始前所有已读 zone 需降至 55℃以内，最多等待 600 秒；保持原 ondemand，不改频率、亲和性、风扇或系统设置。Camera 始终真实采集，但固定帧 VISION 的 Frame Age 标未测量；另以自然摄像头回归测 Frame Age。

每秒读取/proc 和 sysfs；90℃、温度不可读、MemAvailable 低于 256MiB、磁盘低于 500MiB 均触发本次确认 PID 的 SIGTERM 并记录停止原因。活动同步 SDK 调用允许完成，等待任务按既有取消语义处理；超过 70 秒只能对已确认 PID/exe/start-token 最终终止，并判失败。整个进程有有限超时，原始退出码与 SSH 传输码分开。

长测按 TEXT/TEXT/VISION/VISION/TEXT/VISION 循环，使用 5.4 保存的真实语音段做预分段回放，真实执行 ASR/NPU/TTS/ALSA，不能称为真实麦克风/VAD 连续采集。每 15 秒提供一个输入，原 12 秒 TTS 等待与 45 秒期限保留；拒绝、到期及取消均记录，不能只用完成项计算无损吞吐。自然麦克风与耳机另做最终确认。

操作 4 实施与验收映射

改动或测试	原因与验收方式
语音入口时长 1..3600 秒，回放偏移 0..3600000ms	原 120 秒上限阻碍长测；默认 12 秒不变，实际 180/1800 秒回放验证，信号检查仍 20ms
共用 StopSignals 头文件	GUI 和 CLI 复用 5.1 机制；比对类正文，分别 ARM64 信号回归
可选 Qt 适配层	避免 UI 阻塞及重复 AI 实现；QtTest 验证事件心跳、显示、启动失败/恢复、停止、重启与活跃窗口关闭
扩展既有 Fake 并发测试	推理标准/未知异常，TTS 返回失败/异常/无效 PCM、播放失败和后续恢复；核对终态、错误和 worker 继续运行
原 Fake Camera/ASR 测试	安全注入 open 失败、取帧超时/异常、ASR 异常、空输入、回调异常；不冒称真设备断开恢复
分阶段实机矩阵	相同音频/图像/SDK，CPU/RSS/线程/温度/频率、模型延迟、队列、退出及设备保护
最终构建和自然闭环	全目标 ARM64、CTest、真实 Camera/ASR/Qwen/VLM/Kokoro/耳机、SIGINT/SIGTERM、原工程保护
完整手册	汇编历史事实与最终源码，中文楷体/英文 Times New Roman 小四、1.5 倍行距、两字

改动或测试	原因与验收方式
	符首缩进；代码等宽，渲染后检查

所有实际命令、参数、输出、原因与结果按实测继续补齐。当前仅为实施记录，长测和GUI硬件验收结果尚未全部产生。

5.5 Qt 适配与稳定性工具逐句说明

本文对应真实 BackendSession、SessionWindow、Qt 入口、共用 StopSignals 及测试工具。完整源码将由导出器逐文件保留。Qt 代码尚需真实 ARM64 构建和运行，不能因本地 SDK 关闭的 9 项测试通过就宣称 GUI 可运行。

BackendSession 的对象与启动语句

class BackendSession final : public QObject 使适配器进入 Qt 对象和信号系统，final 阻止派生类改变其停止契约。Q_OBJECT 由 AUTOMOC 生成元对象信息；没有它，自定义信号不能连接。QProcess process_、QFile terminal_和 QTimer stopDeadline_是成员值对象，全部由 Qt 线程持有。本类不持有模型、Frame 或 SDK 借用指针。

构造函数将通道设为 MergedChannels，让 SDK stderr 与业务 stdout 按实际到达顺序进入同一日志。connect(sender,signal,this,lambda)中的 this 是接收上下文，Qt 在上下文销毁时断开连接。lambda 只在上下文寿命内访问成员，不把裸窗口指针交给 AI Worker。QProcess 和窗口同属 Qt 线程，因此 readyRead/finished 在该线程更新控件；跨进程传来的是复制字节，绝不是后台 C++对象的悬空引用。

start()先判断 active()，重复启动返回 false 并产生可见错误。QFileInfo(executable).isExecutable()检查外层程序；真正 exec 失败仍由 errorOccurred(FailedToStart)处理。日志使用 WriteOnly|NewOnly，已经存在就拒绝，避免擦掉上轮证据。partial、stopping 和 shutdownSeen 仅在新会话清理，不在旧会话运行时重置。

process_.setProcessEnvironment(environment)设置本次子进程环境，不调用全局 qputenv 与其他线程竞争。process_.start(executable,arguments,ReadOnly)异步启动，立即返回；QStringList 中的每个参数独立传递，不进行 shell 插值。模型路径、设备名含空格或特殊字符也不被解释成命令。生产外层是/usr/bin/stdbuf，argv 顺序为-oL,-

eL,CLI,capture,ASR,VAD,Qwen,Kokoro,playback,seconds,threads,sid,VLM,Vision,Camera,
threads 的零基下标是 10。

started 只证明操作系统成功启动外层进程，不代表模型和设备全部就绪。页面必须依据原入口的加载、Camera ready 和 Listen 日志更新状态。停止若发生于 Starting，先记 stopping_，等 started 再向已得到的子进程 PID 发送 SIGINT；不会对 PID 0 或进程组发送信号。

停止与窗口关闭

requestStop()先检查状态，再设协作停止标志。::kill(processId,SIGINT)在普通 Qt 槽中执行，只发给 QProcess 拥有的子进程。这里没有异步 signal handler。90 秒单次定时器只报告“停止未通过”，不调用 kill 或 waitForFinished；活跃同步 SDK 仍需要自然完成。真正永久阻塞的 SDK 无法由共享 flag 保证退出，这一限制必须保留。

closeEvent()发现后台仍在运行时先设置 closing，调用 event->ignore()保留窗口和回调接收者，再请求停止。收到 finished 时先排空可读输出、关闭诊断文件、恢复按钮、保存真实退出码和截图，再通过带上下文的 singleShot 关闭窗口。对象没有在 Worker 或 QProcess 回调中自我析构，QProcess 析构也不是正常停止手段。

orderly 同时要求 NormalExit、exitCode=0 和原入口[Shutdown] complete 标记；GUI 不会只凭窗口消失或 kill 返回 0 宣称清理完成。设备释放仍需进程外 fuser、ALSA status、进程存在性和原工程指纹复核。90 秒 GUI 提示与守护 70/90 秒失败清理是不同层次，最终强制终止一律记录失败。

输入缓冲与界面内存

consume()保留 UTF-8 字节到完整换行后才调用 fromUtf8，避免一次 readyRead 正好拆开中文字符。原始字节原样写入独占 terminal.log；超长未换行超过 64KiB 时清理显示缓冲并显式报错，原日志仍保留。窗口日志 400 块、答案 100 块、任务历史 64 条有上限，不把持续交互变为无界容器。

SessionWindow 的控件由父窗口拥有，BackendSession 为值成员；关闭事件保证其析构前子进程已经 finished。startSession 先创建 UTC 时间加 UUID 的独占子目录，旧会话 profile 关闭、任务表清空，再打开新会话。每次启动是一个新的 CLI 进程，不冒称已经专项验证真实 SDK 的进程内 Worker restart。

profile 定时器最多读取 64KiB 并按完整 JSONL 解析，Request ID 与队列阶段分开。每条入队/出队都保留 steady 时间戳，只接受同一请求阶段较新的状态；即使日志写入顺序与实际时间顺序不同，也不会用较旧入队覆盖已观察到的出队。它是异步观测快照，不宣称瞬时硬实时队列长度。退出后不重新打开旧 profile，避免把旧 begin 事件显示成新任务。

`Q_ASSERT(QThread::currentThread()==qApp->thread())`检查控件访问线程。界面只解析文本、诊断小记录并更新控件，没有加载模型、调用 NPU、等待 ALSA 或同步 join。自动回归使用真实 Qt 事件循环和按钮信号；Fake 进程的文本在原日志明确标 Fake，不计入硬件功能或性能。

共用信号类与 Qt 入口

`StopSignals` 的类正文从 5.1 修复后的入口移至头文件，算法不变。必须在 QApplication 和 SDK 线程之前构造，后续线程继承阻塞集；普通 sigwait 线程接收 SIGINT/SIGTERM 并写 atomic。Qt 定时器读取该状态，发起普通 closeEvent。没有在 signal handler 调用 Qt、mutex、分配内存或 join。

Qt 的 `signals` 是宏，因此对象变量命名为 `stopSignals`，不能用单独的 `signals` 标识符。QApplication 先析构，StopSignals 最后 join 接收线程并恢复旧掩码。子 CLI 也使用同一类；即使继承了阻塞集，仍由其普通 sigwait 线程处理，停止语义不会退回默认异步终止。

`QFileInfo(reportRoot).canonicalFilePath()`与隔离 reports 前缀校验限制产物位置；独占创建失败就拒绝启动。6 个模型变量从已有路径脚本导出，GUI 没有拷贝模型或编辑 SDK 配置。`BUILD_QT_GUI` 默认 OFF，只有明确启用才 find Qt5；SDK 关闭的 portable 构建不依赖 Qt 或 Linux 信号头。

故障测试和长测工具

扩展现有 Fake 异步测试，在 worker 空闲的同步点才改变测试注入变量。等待空闲所使用的互斥锁/条件变量建立先行关系，不能在合成运行时改普通 int。推理标准/未知异常、TTS 返回 false/抛异常/无效 PCM、ALSA 返回 false 分别检查失败计数和错误文本；随后同一 Worker 执行成功项，验证错误不是被吞成成功，也没有让线程入口异常终止。

QtTest 使用没有 SDK 和设备的独立 Fake 进程。它验证点击开始、识别/答案显示、100ms 心跳、点击停止、新子进程重启、活跃窗口关闭、不可执行/exec 失败、恢复和拒绝

覆盖日志。Fake 的 signal handler 只写 sig_atomic_t；先安装 handler 再解除继承的阻塞信号，避免测试自身的早到信号竞态。

实机守护每秒读取/proc 与 sysfs，明确记录温度、CPU 频率、cooling state、线程和 RSS，保留实际模型及 TTS 队列事件。90℃是本项目保守测试线而非芯片安全额定值。cooling state 为正只能说明对应 Linux cooling 设备活动；频率下降还可能来自 ondemand 和当前负载，不能自动归因于热降频。

分析器复用 5.4 的 steady 事件定义与 nearest-rank 统计。TTS 时长由 tts_full_audio_ready 的实际样本数除以应用采样率计算；采样率从 ALSA 日志的 buffer_frames/buffer_ms 还原，并核对本模型 24kHz 条件。它不是耳机声学时长。RSS 趋势排除前 5 分钟，并限制到进程第 30 分钟以前，避免模型加载或退出释放伪造斜率；同时比较 5～10 与 25～30 分钟的中位数，不能只凭进程没 OOM 就证明无泄漏。

完整命令、失败修复、实机数据与未验证条件以最终报告为准。文档辅助工具使用打包 python-docx/Poppler，原生 Word 导出仅操作本工程 DOCX；没有捏造 GUI 截图、热曲线或运行日志。

完整关键源码 include/gui/backend_session.h

```
#pragma once
#include <QObject>
#include <QProcess>
#include <QProcessEnvironment>
#include <QFile>
#include <QTimer>

// Owns one CLI child. No SDK or device calls execute on the Qt thread.
// Owner must defer destruction until finished; stop never calls waitForFinished/kill.
class BackendSession final : public QObject {
    Q_OBJECT
public:
    explicit BackendSession(QObject* parent=nullptr);
    bool start(const QString& executable,const QStringList& arguments,
               const QProcessEnvironment& environment,const QString& terminalPath);
    void requestStop();
    bool active() const { return process_.state()!=QProcess::NotRunning; }
    qint64 pid() const { return process_.processId(); }
signals:
    void lineReceived(QString line);
    void problem(QString message);
    void running();
    void finished(int code,bool orderly);
private:
    void consume();
    QProcess process_;
    QFile terminal_;
    QByteArray partial_;
    QTimer stopDeadline_;
```

```
    bool stopping_=false,shutdownSeen_=false;
};
```

完整关键源码 src/gui/backend_session.cpp

```
#include "gui/backend_session.h"
#include <QFileInfo>
#include <cerrno>
#include <cstring>
#include <signal.h>

BackendSession::BackendSession(QObject* parent):QObject(parent) {
    process_.setProcessChannelMode(QProcess::MergedChannels);
    connect(&process_,&QProcess::readyReadStandardOutput,this,&BackendSession::consume);
    connect(&process_,&QProcess::started,this,[this] {
        emit running();
        if(stopping_)requestStop(); // Stop pressed during asynchronous startup.
    });
    connect(&process_,&QProcess::errorOccurred,this,[this](QProcess::ProcessError error) {
        emit problem(process_.errorString());
        if(error==QProcess::FailedToStart) {
            stopDeadline_.stop();terminal_.close();emit finished(-1,false);
        }
    });
    connect(&process_,qOverload<int,QProcess::ExitStatus>(&QProcess::finished),this,
            [this](int code,QProcess::ExitStatus status) {
        consume();
        if(!partial_.isEmpty()){emit
lineReceived(QString::fromUtf8(partial_));partial_.clear();}
        stopDeadline_.stop();terminal_.close();
        emit finished(code,status==QProcess::NormalExit && code==0 && shutdownSeen_);
    });
    stopDeadline_.setSingleShot(true);
    connect(&stopDeadline_,&QTimer::timeout,this,[this] {
        emit problem(QStringLiteral("停止等待超过 90 秒，退出未通过；仍保留窗口和进程，不以强杀冒
充安全退出"));
    });
}
bool BackendSession::start(const QString& executable,const QStringList& arguments,
                           const QProcessEnvironment& environment,const QString&
terminalPath) {
    if(active()){emit problem(QStringLiteral("已有交互进程，拒绝重复启动"));return false;}
    if(!QFileInfo(executable).isExecutable()){emit problem(QStringLiteral("入口程序不可执行
"));return false;}
    terminal_.setFileName(terminalPath);
    if(!terminal_.open(QIODevice::WriteOnly|QIODevice::NewOnly)) {
        emit problem(QStringLiteral("拒绝覆盖日志或无法创建日志"));return false;
    }
    partial_.clear();stopping_=shutdownSeen_=false;
    process_.setProcessEnvironment(environment);
    // argv is structured; paths/text are never interpolated into a shell.
    process_.start(executable,arguments,QIODevice::ReadOnly);
    return true;
}
void BackendSession::requestStop() {
    if(!active())return;
    stopping_=true;
    if(process_.state()!=QProcess::Running)return;
```

```
    const auto child=process_.processId();
    if(child<=0)return;
    if(::kill(static_cast<pid_t>(child),SIGINT)!=0 && errno!=ESRCH)
        emit problem(QStringLiteral("发送 SIGINT 失败:
")+QString::fromLocal8Bit(std::strerror(errno)));
    if(!stopDeadline_.isActive())stopDeadline_.start(90000);
}
void BackendSession::consume() {
    const auto data=process_.readAllStandardOutput();
    if(terminal_.isOpen() && !data.isEmpty()) {
        if(terminal_.write(data)!=data.size())emit problem(QStringLiteral("终端日志写入失败
"));
        terminal_.flush();
    }
    partial_+=data;
    for(int end;(end=partial_.indexOf('\n'))>=0;) {
        const auto bytes=partial_.left(end);partial_.remove(0,end+1);
        const auto line=QString::fromUtf8(bytes);
        shutdownSeen_|=line.contains(QStringLiteral("[Shutdown] complete"));
        emit lineReceived(line);
    }
    if(partial_.size()>65536) {
        partial_.clear();emit problem(QStringLiteral("异常超长未换行输出已丢弃，原始内容仍在终端
日志"));
    }
}
```

完整关键源码 include/gui/session_window.h

```
#pragma once
#include "gui/backend_session.h"
#include <QWidget>
#include <QMap>
#include <QLabel>
#include <QPushButton>
#include <QComboBox>
#include <QPlainTextEdit>
#include <QElapsedTimer>

struct GuiLaunch {
    QString program;
    QStringList arguments;
    QProcessEnvironment environment;
    QString reportRoot;
    int threadArgument=10; // stdbuf argv: -oL -eL CLI device ... seconds threads sid ...
    bool automatic=false;
    int stopAfterAnswers=0;
    int cycles=1;
};
class SessionWindow final:public QWidget {
    Q_OBJECT
public:
    explicit SessionWindow(GuiLaunch settings,QWidget* parent=nullptr);
    bool active() const {return backend_.active();}
    bool verified() const {return lastOrderly_;}
    int heartbeatCount() const {return heartbeats_;}
    int completedCycles() const {return completedCycles_;}
public slots:
    void startSession();
```

```
    void stopSession();
protected:
    void closeEvent(QCloseEvent* event) override;
private:
    void line(const QString& text);
    void pollProfile();
    void screenshot(const QString& leaf);
    GuiLaunch settings_;
    BackendSession backend_;
    QPushButton *start_,*stop_;
    QComboBox *threads_,*stopMode_;
    QLabel *status_,*asr_,*route_,*metrics_,*queue_,*error_;
    QPlainTextEdit *answers_,*logs_;
    QMap<QString,QLabel*> modules_;
    struct QueueEvent {qint64 timestamp=0;bool pending=false;};
    QMap<quint64,QMap<QString,QueueEvent>> tasks_;
    QFile profile_;
    QByteArray profilePartial_;
    QString runFolder_;
    QTimer poll_;
    QElapsedTimer stopClock_;
    bool closing_=false,stopping_=false,lastOrderly_=true;
    int completedAnswers_=0,completedCycles_=0,heartbeats_=0;
};
```

完整关键源码 src/gui/session_window.cpp

```
#include "gui/session_window.h"
#include <QCloseEvent>
#include <QDateTime>
#include <QDir>
#include <QGridLayout>
#include <QJsonDocument>
#include <QJsonObject>
#include <QRegularExpression>
#include <QTextStream>
#include <QThread>
#include <QApplication>
#include <QUuid>
#include <QVariant>

SessionWindow::SessionWindow(GuiLaunch settings,QWidget* parent)
    :QWidget(parent),settings_(std::move(settings)) {
    setWindowTitle(QStringLiteral("RK3576 端侧多模态交互系统"));resize(980,700);
    auto* grid=new QGridLayout(this);
    auto* title=new QLabel(QStringLiteral("RK3576 多模态语音交互"));
    title->setStyleSheet("font-size:22px;font-weight:bold");grid->addWidget(title,0,0,1,5);
    int column=0;
    for(const auto& name:{"Camera","ASR","LLM","VLM","TTS"}) {
        auto* label=new QLabel(QString::fromLatin1(name)+QStringLiteral(": 未启动"));
        label->setObjectName(QString::fromLatin1(name));modules_[name]=label;grid-
>addWidget(label,1,column++);
    }
    start_=new QPushButton(QStringLiteral("开始语音交互"));start_->setObjectName("start");
    stop_=new QPushButton(QStringLiteral("安全停止"));stop_->setObjectName("stop");stop_-
>setEnabled(false);
    threads_=new QComboBox;threads_->addItems({"TTS 2 线程","TTS 4 线程"});
    threads_->setCurrentIndex(settings_.threadArgument>=0 &&
```

```
settings_.arguments.value(settings_.threadArgument)=="2"?0:1);
    stopMode_=new QComboBox;stopMode_->addItems({QStringLiteral("排空已接收任务
"),QStringLiteral("取消等待任务")});
    stopMode_-
>setCurrentIndex(settings_.environment.value("STAGE5_STOP_MODE")=="cancel_pending"?1:0);
    grid->addWidget(start_,2,0);grid->addWidget(stop_,2,1);grid-
>addWidget(threads_,2,2);grid->addWidget(stopMode_,2,3,1,2);
    status_=new QLabel(QStringLiteral("就绪，点击开始；首次启动需要加载模型"));
    asr_=new QLabel(QStringLiteral("识别文本：等待"));asr_->setObjectName("transcript");
    route_=new QLabel(QStringLiteral("路由：等待"));route_->setObjectName("route");
    status_->setWordWrap(true);asr_->setWordWrap(true);route_->setWordWrap(true);
    metrics_=new QLabel(QStringLiteral("延迟：尚无实测数据"));metrics_->setWordWrap(true);
    queue_=new QLabel(QStringLiteral("推理等待：0  TTS 等待：0（异步事件快照）"));
    error_=new QLabel;error_->setWordWrap(true);error_->setStyleSheet("color:#a02020");
    grid->addWidget(status_,3,0,1,5);grid->addWidget(asr_,4,0,1,5);grid-
>addWidget(route_,5,0,1,5);
    grid->addWidget(metrics_,6,0,1,5);grid->addWidget(queue_,7,0,1,5);grid-
>addWidget(error_,8,0,1,5);
    answers_=new QPlainTextEdit;answers_->setReadOnly(true);answers_-
>setObjectName("answers");answers_->setMaximumBlockCount(100);
    logs_=new QPlainTextEdit;logs_->setReadOnly(true);logs_->setMaximumBlockCount(400);
    grid->addWidget(answers_,9,0,1,5);grid->addWidget(logs_,10,0,1,5);grid-
>setRowStretch(9,2);grid->setRowStretch(10,1);
    connect(start_,&QPushButton::clicked,this,&SessionWindow::startSession);
    connect(stop_,&QPushButton::clicked,this,&SessionWindow::stopSession);
    connect(&backend_,&BackendSession::lineReceived,this,&SessionWindow::line);
    connect(&backend_,&BackendSession::problem,this,[this](const QString& message){error_-
>setText(message);QTextStream(stdout)<<"[GUI error] "<<message<<Qt::endl;});
    connect(&backend_,&BackendSession::running,this,[this]{status_->setText(QStringLiteral("
后台进程已启动，正在加载"));});
    connect(&backend_,&BackendSession::finished,this,[this](int code,bool orderly){

pollProfile();profile_.close();stopping_=false;lastOrderly_=orderly;++completedCycles_;
        start_->setEnabled(true);stop_->setEnabled(false);threads_-
>setEnabled(true);stopMode_->setEnabled(true);
        status_->setText(QStringLiteral("进程退出
码 %1；%2").arg(code).arg(orderly?QStringLiteral("清理完成"):QStringLiteral("未通过安全退出检
查")));
        if(!orderly)error_->setText(QStringLiteral("启动或运行失败，请查看保留的原始日志"));
        for(auto* label:modules_)label->setText(label->objectName()+QStringLiteral("：进程已
退出"));
        QTextStream(stdout)<<"[GUI finished] code="<<code<<" orderly="<<orderly<<"
cycles="<<completedCycles_<<" heartbeat="<<heartbeats_<<Qt::endl;
        QFile result(runFolder_+"/result.json");
        if(result.open(QIODevice::WriteOnly|QIODevice::NewOnly)) {
            QJsonObject
object{{"returncode",code},{"orderly",orderly},{"heartbeat",heartbeats_},{"answers_completed
",completedAnswers_},{"cycles_completed",completedCycles_}};
            result.write(QJsonDocument(object).toJson());
        } else error_->setText(QStringLiteral("结果写入失败"));
        screenshot("finished.png");
        if(settings_.automatic && completedCycles_<settings_.cycles &&
orderly)QTimer::singleShot(200,this,&SessionWindow::startSession);
        else if(closing_ || settings_.automatic)QTimer::singleShot(0,this,&QWidget::close);
    });
    poll_.setInterval(100);
    connect(&poll_,&QTimer::timeout,this,[this]{++heartbeats_;pollProfile();});poll_.start();
    if(settings_.automatic)QTimer::singleShot(100,start_,&QPushButton::click);
}
void SessionWindow::startSession() {
```

```
    if(backend_.active() || closing_)return;

runFolder_=QDir(settings_.reportRoot).filePath("run_"+QDateTime::currentDateTimeUtc().toStri
ng("yyyyMMdd_HHmmss_zzz")+"_"+QUuid::createUuid().toString(QUuid::Id128));
    if(!QDir().mkdir(runFolder_)){error_->setText(QStringLiteral("无法创建独占运行目录
"));return;}
    auto env=settings_.environment;
    env.insert("STAGE5_PROFILE",runFolder_+"/profile.jsonl");
    env.insert("STAGE5_STOP_MODE",stopMode_->currentIndex()==0?"drain":"cancel_pending");
    auto args=settings_.arguments;
    if(settings_.threadArgument>=0 &&
settings_.threadArgument<args.size())args[settings_.threadArgument]=threads_-
>currentIndex()==0?"2":"4";

completedAnswers_=0;lastOrderly_=false;stopping_=false;tasks_.clear();profilePartial_.clear(
);profile_.close();
    profile_.setFileName(runFolder_+"/profile.jsonl");
    error_->clear();answers_->clear();logs_->clear();
    asr_->setText(QStringLiteral("识别文本: 等待新会话"));route_->setText(QStringLiteral("路由:
等待新会话"));
    metrics_->setText(QStringLiteral("延迟: 等待新会话实测"));
    queue_->setText(QStringLiteral("推理等待: 0  TTS 等待: 0（异步事件快照）"));
    if(backend_.start(settings_.program,args,env,runFolder_+"/terminal.log")) {
        start_->setEnabled(false);stop_->setEnabled(true);threads_-
>setEnabled(false);stopMode_->setEnabled(false);
        status_->setText(QStringLiteral("异步启动中"));
        QTextStream(stdout)<<"[GUI run] "<<runFolder_<<Qt::endl;
    } else if(settings_.automatic)QTimer::singleShot(0,this,&QWidget::close);
}
void SessionWindow::stopSession() {
    if(!backend_.active() || stopping_)return;
    stopping_=true;stopClock_.start();backend_.requestStop();stop_->setEnabled(false);
    status_->setText(QStringLiteral("正在协作停止，等待后台线程和资源释放"));
    screenshot("stopping.png");
}
void SessionWindow::closeEvent(QCloseEvent* event) {
    if(backend_.active()){closing_=true;event->ignore();stopSession();return;}
    event->accept();
}
void SessionWindow::screenshot(const QString& leaf) {
    if(!runFolder_.isEmpty() && !grab().save(runFolder_+'/'+leaf))error_-
>setText(QStringLiteral("界面截图保存失败"));
}
void SessionWindow::line(const QString& text) {
    Q_ASSERT(QThread::currentThread()==qApp->thread());
    logs_->appendPlainText(text);QTextStream(stdout)<<text<<Qt::endl;
    const auto module=[this](const char* key,const QString& state){modules_[key]-
>setText(QString::fromLatin1(key)+": "+state);};
    if(text.contains("Loading RKLLM"))module("LLM",QStringLiteral("加载中"));
    if(text.contains("Loading SenseVoice")){module("LLM",QStringLiteral("已加载
"));module("ASR",QStringLiteral("加载中"));}
    if(text.contains("Loading Kokoro")){module("ASR",QStringLiteral("已加载
"));module("TTS",QStringLiteral("加载中"));}
    if(text.contains("[ALSA config]"))module("TTS",QStringLiteral("已加载"));
    if(text.contains("VLM and Camera ready")){module("Camera",QStringLiteral("采集中
"));module("VLM",QStringLiteral("已加载"));}
    if(text.contains("[Listen]") || text.contains("[Replay ready]")){status_-
>setText(text);screenshot("listening.png");}
    if(text.contains("success=false") || text.contains("[Fatal]") || (text.contains("[Init]")
&& text.contains("failed")) || text.contains("rejected #") || text.contains("[TTS
```

```
expired"))error_->setText(text);
}
void SessionWindow::pollProfile() {
    if(!backend_.active() && !profile_.isOpen())return; // Do not replay old events while
idle.
    if(!profile_.isOpen() && !profile_.fileName().isEmpty() &&
QFile::exists(profile_.fileName()))profile_.open(QIODevice::ReadOnly);
    if(!profile_.isOpen())return;
    profilePartial_+=profile_.read(65536);
    for(int end;(end=profilePartial_.indexOf('\n'))>=0;) {
        const auto bytes=profilePartial_.left(end);profilePartial_.remove(0,end+1);
        const auto object=QJsonDocument::fromJson(bytes).object();
        if(object.value("kind")!="request")continue;
        const auto id=object.value("request_id").toVariant().toULongLong();
        const auto name=object.value("event").toString();
        const auto at=object.value("at_ns").toVariant().toLongLong();
        const QStringList
timed={"asr_enqueued","asr_begin","asr_end","inference_begin","inference_end","tts_enqueued"
,"tts_dequeued","tts_generate_begin","tts_generate_end","alsa_first_submit_begin","request_c
omplete"};
        if(timed.contains(name))tasks_[id]["at:"+name].timestamp=at;
        if(name=="asr_text" || name=="input_text")asr_-
>setText((name=="asr_text"?QStringLiteral("识别 #%1: "):QStringLiteral("文本输入 #%1:
")).arg(id)+object.value("text").toString());
        if(name=="router_end")route_->setText(QStringLiteral("路由
#%1: %2").arg(id).arg(object.value("value").toInt()==1?"VISION":"TEXT"));
        if(name=="answer_text")answers_->appendPlainText(QStringLiteral("回答
#%1: %2").arg(id).arg(object.value("text").toString()));
        if(name=="request_error")error_->setText(object.value("text").toString());
        if(name=="request_terminal")status_->setText(object.value("text").toString());
        for(const auto& stage:{QString("inference"),QString("tts")}) {
            if(name==stage+"_enqueued" || name==stage+"_dequeued") {
                auto& record=tasks_[id][stage];

if(at>=record.timestamp){record.timestamp=at;record.pending=name.endsWith("_enqueued");}
            }
        }
        if(name.endsWith("_begin"))status_->setText(QStringLiteral("执行
#%1: %2").arg(id).arg(name));
        if(timed.contains(name)) {
            const auto& task=tasks_[id];
            const auto duration=[&](const char* begin,const char* end) {
                const auto a=task.value("at:"+QString::fromLatin1(begin)).timestamp;
                const auto b=task.value("at:"+QString::fromLatin1(end)).timestamp;
                return a>0 && b>=a?QString::number((b-a)/1e6,'f',1)+" ms":QStringLiteral("未
测量");
            };
            metrics_->setText(QStringLiteral("请求#%1  ASR %2  推理 %3  TTS %4  TTS 排队 %5
ASR 入队→首次提交 %6")
                .arg(id).arg(duration("asr_begin","asr_end"),duration("inference_begin","inf
erence_end"),

duration("tts_generate_begin","tts_generate_end"),duration("tts_enqueued","tts_dequeued"),
                    duration("asr_enqueued","alsa_first_submit_begin")));
        }
        if(name=="request_complete") {

++completedAnswers_;screenshot("answer_"+QString::number(completedAnswers_)+".png");
            if(settings_.stopAfterAnswers>0 &&
completedAnswers_>=settings_.stopAfterAnswers)stopSession();
        }
```

```
        while(tasks_.size()>64)tasks_.erase(tasks_.begin());
    }
    if(profilePartial_.size()>65536){profilePartial_.clear();error_-
>setText(QStringLiteral("异常超长诊断事件，已记录错误"));}
    int inference=0,tts=0;
    for(const auto&
task:tasks_){inference+=task.value("inference").pending;tts+=task.value("tts").pending;}
    queue_->setText(QStringLiteral("推理等待：%1  TTS 等待：%2（异步事件快照）
").arg(inference).arg(tts));
}
```

完整关键源码 src/gui/main.cpp

```
#include "app/stop_signals.h"
#include "gui/session_window.h"
#include <QApplication>
#include <QCommandLineParser>
#include <QDir>
#include <QFileInfo>
#include <QTextStream>
#include <QScreen>

int main(int argc,char** argv) try {
    StopSignals stopSignals; // Must precede QApplication and every descendant thread.
    QApplication app(argc,argv);
    app.setApplicationName("rk3576_multimodal_gui");
    QCommandLineParser parser;parser.addHelpOption();
    parser.addOption({"capture","ALSA capture device, rediscover HyperX before
launch","device"});
    parser.addOption({"playback","ALSA playback device","device"});
    parser.addOption({"report-root","Existing isolated report directory; each run creates a
new child","path"});
    parser.addOption({"threads","Kokoro threads (2 or 4)","number","4"});
    parser.addOption({"seconds","Finite listening duration, 1..3600","number","120"});
    parser.addOption({"auto-start","Automated regression: click start after event loop
begins"});
    parser.addOption({"stop-after-answers","Automated regression: stop after N successful
TTS answers","number","0"});
    parser.addOption({"cycles","Automated regression: fresh child sessions,
1..4","number","1"});
    parser.process(app);
    const auto env=QProcessEnvironment::systemEnvironment();
    const auto review=env.value("STAGE5_REVIEW");
    const auto reports=QFileInfo(parser.value("report-root")).canonicalFilePath();
    if(review.isEmpty() || reports.isEmpty()
|| !reports.startsWith(QDir(review).canonicalPath()+"/reports/stage5_05_board/"))
        throw std::runtime_error("report-root must be an existing isolated Stage5.5 report
directory");
    const auto number=[&](const char* key,int lo,int hi){bool ok=false;int
n=parser.value(key).toInt(&ok);if(!ok || n<lo || n>hi)throw std::runtime_error("invalid
numeric GUI argument");return n;};
    const int threads=number("threads",2,4),seconds=number("seconds",1,3600);
    if(threads!=2 && threads!=4)throw std::runtime_error("threads must be 2 or 4");
    if(parser.value("capture").isEmpty() || parser.value("playback").isEmpty())throw
std::runtime_error("explicit ALSA devices required");
    for(const auto*
key:{"STAGE5_ASR","STAGE5_VAD","STAGE5_QWEN","STAGE5_KOKORO","STAGE5_VLM","STAGE5_VISION"})
        if(env.value(key).isEmpty())throw std::runtime_error("source and export
stage5_board_paths.sh first");
```

```
    GuiLaunch
launch;launch.program="/usr/bin/stdbuf";launch.environment=env;launch.reportRoot=reports;
    launch.arguments=QStringList{"-oL","-
eL",QCoreApplication::applicationDirPath()+"/asr_llm_tts_smoke",parser.value("capture"),env.
value("STAGE5_ASR"),env.value("STAGE5_VAD"),env.value("STAGE5_QWEN"),env.value("STAGE5_KOKOR
O"),parser.value("playback"),QString::number(seconds),QString::number(threads),"17",env.valu
e("STAGE5_VLM"),env.value("STAGE5_VISION"),"/dev/video11"};
    launch.automatic=parser.isSet("auto-start");launch.stopAfterAnswers=number("stop-after-
answers",0,32);launch.cycles=number("cycles",1,4);
    SessionWindow window(std::move(launch));
    if(auto* screen=QGuiApplication::primaryScreen()) {
        const auto size=screen->availableGeometry().size();
        window.resize(qMin(980,size.width()-40),qMin(700,size.height()-60));
    }
    window.show();
    QTimer pollSignals;pollSignals.setInterval(100);
    QObject::connect(&pollSignals,&QTimer::timeout,&window,[&] {
        if(stopSignals.received() || stopSignals.failed())window.close();
    });pollSignals.start();
    const int code=app.exec();
    return code?code:(stopSignals.failed() || !window.verified()?1:0);
} catch(const std::exception& error) {
    QTextStream(stderr)<<"[GUI fatal] "<<error.what()<<Qt::endl;return 1;
}
```

完整关键源码 src/audio/asr_llm_tts_smoke.cpp

```
#include "app/stop_signals.h"

#include "audio/asr_pipeline.h"
#include "app/pipeline_admission.h"
#include "app/pipeline_metrics.h"
#include "app/profiling_inputs.h"
#include <fstream>
#include "audio/asr_service.h"
#include "audio/vad_audio_worker.h"
#include "audio/tts_service.h"
#include "audio/alsa_playback_service.h"
#include "audio/tts_worker.h"

#include "ai/llm_worker.h"
#include "ai/rk_backends.h"
#include "router/intent_router.h"
#include "ai/vlm_engine.h"
#include "video/camera_service.h"
#include "video/latest_frame_buffer.h"

#include <chrono>
#include <algorithm>
#include <fstream>
#include <sstream>
#include <cstdlib>
#include <atomic>
#include <cerrno>
#include <condition_variable>
#include <cstddef>
#include <cstdint>
#include <exception>
#include <iostream>
```

```
#include <memory>
#include <mutex>
#include <stdexcept>
#include <string>
#include <utility>
#include <thread>

#include <pthread.h>
#include <signal.h>

#include <unistd.h>



// ========================================================
// Stage 3.5
//
// ALSA Capture
//     ↓
// Silero VAD
//     ↓
// SenseVoice ASR
//     ↓
// IntentRouter
//     ↓
// LLMWorker -> RKLLM / Qwen2.5
//     ↓
// TTSWorker -> Kokoro
//     ↓
// AlsaPlaybackService
//     ↓
// USB Headset
//
// TEXT and VISION share one asynchronous bounded inference FIFO.
// ========================================================

int main(int argc, char** argv) try
{
    // ------------------------------------------------------
    // 1. 命令行参数检查
    // ------------------------------------------------------

    if (argc < 7 || (argc > 10 && argc != 13))
    {
        std::cerr
            << "Usage: asr_llm_tts_smoke "
            << "<capture_device> "
            << "<asr_model_dir> "
            << "<vad_model> "
            << "<rkllm_model> "
            << "<kokoro_dir> "
            << "<playback_device> "
            << "[record_seconds=12] "
            << "[tts_threads=4] "
            << "[sid=17] "
            << "[vlm_rkllm vision_rknn camera_device]\n";

        return 2;
    }

    const std::string capture_device = argv[1];
    const std::string asr_dir = argv[2];
```

```
const std::string vad_model = argv[3];
const std::string llm_model = argv[4];
const std::string kokoro_dir = argv[5];
const std::string playback_device = argv[6];

unsigned int record_seconds = 12;

int tts_threads = 4;
int sid = 17;

// 兼容旧命令。新增三项参数时启用完整视觉语音链路。
// 语音模式须传三个原有可选参数，再传 VLM 模型与 Camera 设备。
const bool enable_vision = (argc == 13);
const std::string vlm_model = enable_vision ? argv[10] : "";
const std::string vision_model = enable_vision ? argv[11] : "";
const std::string camera_device = enable_vision ? argv[12] : "";

// ------------------------------------------------------
// 2. 解析可选参数
// ------------------------------------------------------

try
{
    const auto parse = [](
        const char* arg,
        int lo,
        int hi
    ) -> int
    {
        const std::string value(arg);

        std::size_t used = 0;

        const int result =
            std::stoi(value, &used);

        if (used != value.size() ||
            result < lo ||
            result > hi)
        {
            throw std::invalid_argument(
                "out of range"
            );
        }

        return result;
    };

    if (argc >= 8)
    {
        record_seconds =
            static_cast<unsigned int>(
                parse(argv[7], 1, 3600)
            );
    }

    if (argc >= 9)
    {
        tts_threads =
            parse(argv[8], 1, 8);
    }
```

```
    if (argc >= 10)
    {
        sid =
            parse(argv[9], 0, 10000);
    }
}
catch (const std::exception&)
{
    std::cerr
        << "[Args] Invalid record_seconds / "
        << "threads / sid\n";

    return 2;
}

// ------------------------------------------------------
// 3. 检查 ASR / VAD / LLM 模型文件
// ------------------------------------------------------

const std::string asr_model =
    asr_dir + "/model.int8.onnx";

const std::string asr_tokens =
    asr_dir + "/tokens.txt";

for (const auto& path :
     {asr_model, asr_tokens, vad_model, llm_model})
{
    if (::access(path.c_str(), R_OK) != 0)
    {
        std::cerr
            << "[Init] Cannot read model file: "
            << path
            << '\n';

        return 1;
    }
}

if (enable_vision)
{
    for (const auto& path : {vlm_model, vision_model})
    {
        if (::access(path.c_str(), R_OK) != 0)
        {
            std::cerr << "[Init] Cannot read VLM file: " << path << '\n';
            return 1;
        }
    }
}

std::string error;

StopSignals stop_signals;

// ------------------------------------------------------
// 4. 初始化 RKLLM
// ------------------------------------------------------

LLMEngine llm(
    std::make_unique<RKLLMBackend>()
);
```

```
std::cout
    << "[Init] Loading RKLLM...\n"
    << std::flush;

if (!llm.initialize(
        BackendConfig{llm_model, {}},
        error))
{
    std::cerr
        << "[Init] RKLLM failed: "
        << error
        << '\n';

    return 1;
}

// ------------------------------------------------------
// 5. 初始化 SenseVoice
// ------------------------------------------------------

ASRService asr;

std::cout
    << "[Init] Loading SenseVoice...\n"
    << std::flush;

if (!asr.initialize(
        asr_model,
        asr_tokens,
        error))
{
    std::cerr
        << "[Init] ASR failed: "
        << error
        << '\n';

    llm.shutdown();

    return 1;
}

// ------------------------------------------------------
// 6. 初始化 Kokoro TTS
// ------------------------------------------------------

TTSService tts;

std::cout
    << "[Init] Loading Kokoro...\n"
    << std::flush;

if (!tts.initialize(
        kokoro_dir,
        error,
        tts_threads,
        sid))
{
    std::cerr
        << "[Init] Kokoro failed: "
        << error
        << '\n';
```

```
        llm.shutdown();

        return 1;
    }

    // ------------------------------------------------------
    // 7. 初始化 ALSA Playback
    // ------------------------------------------------------

    AlsaPlaybackService playback;

    if (!playback.open(
            playback_device,
            tts.sampleRate(),
            error))
    {
        std::cerr
            << "[Init] ALSA playback failed: "
            << error
            << '\n';

        llm.shutdown();

        return 1;
    }

    // Callback state precedes workers: reverse destruction joins workers first.
    // ------------------------------------------------------
    // 10. 创建 ASR 音频队列和线程同步对象
    // ------------------------------------------------------

    AudioBufferQueue audio_queue(4);

    // 保护不同线程的日志输出。
    std::mutex print_mutex;

    // 保护 LLM/TTS 任务完成计数。
    std::mutex state_mutex;

    // 通知主线程有任务完成。
    std::condition_variable completed_cv;

    IntentRouter router;

    // 由单个 ASRWorker 回调生成请求 ID。
    std::uint64_t next_id = 1;

    const int max_inflight = pipelineEnv("STAGE5_MAX_INFLIGHT", 2, 1, 8);
    const int request_lifetime_ms = pipelineEnv("STAGE5_REQUEST_LIFETIME_MS", 45000, 1,
300000);
    const int inference_queue_ms = pipelineEnv("STAGE5_INFERENCE_QUEUE_MS", 10000, 1, 60000);
    const int tts_queue_ms = pipelineEnv("STAGE5_TTS_QUEUE_MS", 12000, 1, 60000);
    const char* stop_value = std::getenv("STAGE5_STOP_MODE");
    const std::string stop_mode = stop_value ? stop_value : "drain";
    if (stop_mode != "drain" && stop_mode != "cancel_pending") throw
std::invalid_argument("Invalid STAGE5_STOP_MODE");
    PipelineAdmission admission(static_cast<std::size_t>(max_inflight));
    std::size_t dispatch_calls=0, pipeline_rejected=0, cancelled_before_submit=0;
    std::size_t vision_accepted=0, vision_rejected=0;
    std::size_t llm_cancelled=0, llm_expired=0, vision_cancelled=0, vision_expired=0;
```

```
    std::size_t tts_cancelled=0, tts_expired=0, downstream_cancelled=0;
    // Must outlive the asynchronous inference worker, including exception unwinding.
    LatestFrameBuffer frames;
    std::unique_ptr<CameraService> camera;
    std::unique_ptr<VLMEngine> vlm;
    LatestFrameBuffer::FramePtr profiling_frame;
    if(const char* folder=std::getenv("STAGE5_REPLAY_FRAME_DIR")){
        if(!enable_vision || (!std::getenv("STAGE5_REPLAY_FILE")
&& !std::getenv("STAGE5_REPLAY_AUDIO_FILE")))throw std::invalid_argument("Fixed frame is
allowed only with explicit replay and VISION");
        profiling_frame=loadProfilingFrame(folder);
        std::cout<<"[InputMode] fixed NV12 for replay VLM; Camera still captures
independently\n";
    }

    // ------------------------------------------------------
    // 11. LLM 和 TTS 统计
    // ------------------------------------------------------

    std::size_t llm_accepted = 0;
    std::size_t llm_completed = 0;
    std::size_t llm_successful = 0;
    std::size_t llm_failed = 0;
    std::size_t llm_rejected = 0;

    std::size_t skipped_vision = 0;
    std::size_t vision_completed = 0;
    std::size_t vision_successful = 0;
    std::size_t vision_failed = 0;

    std::size_t tts_accepted = 0;
    std::size_t tts_completed = 0;
    std::size_t tts_successful = 0;
    std::size_t tts_failed = 0;
    std::size_t tts_rejected = 0;
    std::size_t tts_empty = 0;

    // ------------------------------------------------------
    // 8. 创建并启动 TTSWorker
    // ------------------------------------------------------
    //
    // TTSWorker 不拥有 TTSService 和 PlaybackService。
    // 两个 Service 必须比 Worker 活得更久。
    //
    // capacity=2 表示最多允许两个等待任务。
    // ------------------------------------------------------

    TTSWorker tts_worker(
        tts,
        playback,
        2
    );

    if (!tts_worker.start(error))
    {
        std::cerr
            << "[Init] TTSWorker failed: "
            << error
            << '\n';

        llm.shutdown();
```

```
    return 1;
}

// ------------------------------------------------------
// 9. 创建并启动 LLMWorker
// ------------------------------------------------------

LLMWorker llm_worker(
    llm,
    2
);

if (!llm_worker.start(error))
{
    std::cerr
        << "[Init] LLMWorker failed: "
        << error
        << '\n';

    tts_worker.requestStop();
    tts_worker.join();

    llm.shutdown();

    return 1;
}

// Stage 4: 正式 Camera + VLM Backend。保持旧的 TEXT 链路不变。
if (enable_vision)
{
    constexpr std::uint32_t nv12 = std::uint32_t('N') |
        (std::uint32_t('V') << 8) |
        (std::uint32_t('1') << 16) |
        (std::uint32_t('2') << 24);

    camera = std::make_unique<CameraService>(
        camera_device, 3840, 2160, nv12, frames);
    if (!camera->start())
    {
        std::cerr << "[Init] Camera failed: "
                  << camera->stats().last_error << '\n';
        llm_worker.requestStop();
        llm_worker.join();
        tts_worker.requestStop();
        tts_worker.join();
        llm.shutdown();
        return 1;
    }

    vlm = std::make_unique<VLMEngine>(
        std::make_unique<RK3576VLMBackend>());
    std::cout << "[Init] Loading SmolVLM2 RKNN + RKLLM...\n"
              << std::flush;
    if (!vlm->initialize(
        BackendConfig{vlm_model, vision_model}, error))
    {
        std::cerr << "[Init] VLM failed: " << error << '\n';
        camera->stop();
        llm_worker.requestStop();
        llm_worker.join();
        tts_worker.requestStop();
```

```
            tts_worker.join();
            llm.shutdown();
            return 1;
        }
        std::cout << "[Init] VLM and Camera ready\n";
    }

    // ------------------------------------------------------
    // 12. ASR 识别完成回调
    // ------------------------------------------------------
    //
    // 此回调在 ASRWorker 线程执行。
    //
    // recognized 是 SenseVoice 的识别结果。
    // ------------------------------------------------------

    const auto on_text =
        [&](const std::string& recognized, RequestTracePtr supplied_trace = {})
    {
        const bool recognized_by_asr=static_cast<bool>(supplied_trace);
        const auto request_id = supplied_trace ? supplied_trace->id() : next_id++;
        if(supplied_trace)next_id=std::max(next_id,request_id+1);
        const auto trace=supplied_trace ? std::move(supplied_trace) :
makeRequestTrace(request_id);
        RequestTraceScope trace_scope(trace);
        if(trace)trace->text(recognized_by_asr?"asr_text":"input_text",recognized);
        traceMark("business_dispatch_begin");
        DispatchTimer dispatch_timer{print_mutex, ++dispatch_calls};
        // ------------------------------------------------
        // 12.1 意图判断
        // ------------------------------------------------

        const RouteDecision route =
            router.route(recognized);

        auto ticket = admission.tryAcquire(request_id,
std::chrono::milliseconds(request_lifetime_ms));
        if (!ticket)
        {
            traceMark("pipeline_rejected_busy");
            ++pipeline_rejected;
            std::lock_guard<std::mutex> out(print_mutex);
            std::cout << "[Pipeline rejected #" << request_id << "] route=" <<
requestTypeName(route.type)
                      << " reason=busy_or_closed limit=" << max_inflight << '\n';
            return;
        }

        if (route.type == RequestType::Vision)
        {
            if (!enable_vision || !vlm)
            {
                traceMark("vision_disabled");
                ++skipped_vision;
                std::lock_guard<std::mutex> out(print_mutex);
                std::cout << "\n[Route] VISION disabled: "
                          << recognized << '\n';
                return;
            }

            const auto question_at = std::chrono::steady_clock::now();
```

```
            // 先复用已在实机验证的英语单图问答；中文语音负责触发路由。
            // 避免 256M 模型不稳定的中文指令影响最终闭环验收。
            const std::string vlm_question =
                "What do you see? Answer briefly.";

            {
                std::lock_guard<std::mutex> out(print_mutex);
                std::cout << "\n[ASR->VISION #" << request_id << "] "
                          << recognized << '\n';
            }

            const auto status = llm_worker.submitTask(request_id,
                [&, vlm_question](const LLMStreamCallback&) {
                    // Fetch a fresh owned frame when the task executes, never pin queued
full frames.
                    return vlm->infer(profiling_frame?profiling_frame:frames.getLatest(),
vlm_question);
                }, {},
                [&, question_at, ticket, trace](std::uint64_t request_id, const
InferenceResult& result, double inference_wait_ms)
                {
                    if(trace)trace-
>text(result.success?"answer_text":result.disposition==TaskDisposition::Completed?"request_e
rror":"request_terminal",result.success?result.text:result.error_message);
                    {
                        std::lock_guard<std::mutex> lock(state_mutex);
                        ++vision_completed;
                        if (result.disposition==TaskDisposition::Cancelled)
++vision_cancelled;
                        else if (result.disposition==TaskDisposition::Expired)
++vision_expired;
                        else if (result.success && !result.text.empty()) ++vision_successful;
                        else ++vision_failed;
                    }

                    {
                        std::lock_guard<std::mutex> out(print_mutex);
                        std::cout << "[VLM done #" << request_id << "] success="
                                  << (result.success ? "true" : "false")
                                  << " queue_ms=" << inference_wait_ms
                                  << " disposition=" << dispositionName(result.disposition)
                                  << " frame_id=" << result.frame_id
                                  << " frame_age_ms=" << result.frame_age_ms
                                  << " latency_ms=" << result.latency_ms
                                  << '\n';
                        if (result.success)
                            std::cout << "[VLM Answer #" << request_id << "] "
                                      << result.text << '\n';
                        else
                            std::cout << (result.disposition==TaskDisposition::Completed ?
"[VLM Error] " : "[VLM terminal] ") << result.error_message << '\n';
                    }

                    if (result.success && !result.text.empty() && ticket->cancelled())
                    {
                        { std::lock_guard<std::mutex> lock(state_mutex);
++downstream_cancelled; }
                        std::lock_guard<std::mutex> out(print_mutex);
                        std::cout << "[Pipeline downstream cancelled #" << request_id << "]
stage=tts\n";
                    }
                    if (result.success && !result.text.empty() && !ticket->cancelled())
```

```
                    {
                        const auto status = tts_worker.submit(
                            request_id, result.text,
                            [&, question_at, ticket](
                                std::uint64_t done_id,
                                const TTSWorkerResult& outcome,
                                double queue_ms)
                            {
                                const double e2e_ms = std::chrono::duration<
                                    double, std::milli>(
                                    std::chrono::steady_clock::now() - question_at
                                ).count();
                                {
                                    std::lock_guard<std::mutex> out(print_mutex);
                                    std::cout << "[VISION TTS done #" << done_id
                                              << "] success="
                                              << (outcome.success ? "true" : "false")
                                              << " queue_ms=" << queue_ms
                                              << " synth_ms=" << outcome.synthesis_ms
                                              << " playback_ms=" << outcome.playback_ms
                                              << " asr_text_to_end_ms=" << e2e_ms
                                              << '\n';
                                    if (!outcome.success)
                                        std::cout <<
(outcome.disposition==TaskDisposition::Completed ? "[TTS error] " : "[TTS terminal] ")
                                                  << outcome.error_message << '\n';
                                }
                                {
                                    std::lock_guard<std::mutex> lock(state_mutex);
                                    ++tts_completed;
                                    if (outcome.success) ++tts_successful;
                                    else if (outcome.disposition==TaskDisposition::Cancelled)
++tts_cancelled;
                                    else if (outcome.disposition==TaskDisposition::Expired)
++tts_expired;
                                    else ++tts_failed;
                                }
                                completed_cv.notify_all();
                        }, ticket->options(std::chrono::milliseconds(tts_queue_ms), trace));

                    {
                        std::lock_guard<std::mutex> lock(state_mutex);
                        if (status == TTSSubmitStatus::Accepted)
                            ++tts_accepted;
                        else
                            ++tts_rejected;
                    }
                    if (status != TTSSubmitStatus::Accepted)
                    {
                        std::lock_guard<std::mutex> out(print_mutex);
                        std::cout << "[VISION TTS rejected #" << request_id
                                  << "] status=" << static_cast<int>(status)
                                  << '\n';
                    }
                }
                else if (result.success && result.text.empty())
                {
                    std::lock_guard<std::mutex> lock(state_mutex);
                    ++tts_empty;
                }
                // Inference/done run on the shared worker; ASR only submits.
                    }, ticket->options(std::chrono::milliseconds(inference_queue_ms), trace));
```

```
            {
                std::lock_guard<std::mutex> lock(state_mutex);
                if (status==LLMSubmitStatus::Accepted) ++vision_accepted;
                else if (status==LLMSubmitStatus::NotRunning && ticket->cancelled())
++cancelled_before_submit;
                else ++vision_rejected;
            }
            if (status!=LLMSubmitStatus::Accepted)
            {
                std::lock_guard<std::mutex> out(print_mutex);
                std::cout << (ticket->cancelled() ? "[VISION submission cancelled #" :
"[VISION rejected #") << request_id << "] status=" << static_cast<int>(status) << '\n';
            }
            return;
        }

        // ------------------------------------------------
        // 12.2 分配请求 ID
        // ------------------------------------------------

        const std::uint64_t id =
            request_id;

        // 记录 ASR 输出文本的时刻。
        const auto question_at =
            std::chrono::steady_clock::now();

        // 记录 LLM 是否已经输出前缀。
        auto prefix_printed =
            std::make_shared<bool>(false);

        {
            std::lock_guard<std::mutex>
                out(print_mutex);

            std::cout
                << "\n[ASR->LLM #" << id << "] "
                << recognized
                << '\n';
        }

        // ------------------------------------------------
        // 12.3 构造 LLM Prompt
        // ------------------------------------------------
        //
        // 限制回答长度，降低当前 Kokoro 的合成等待。
        // 不改变 ASR 原始文本。
        // ------------------------------------------------

        const std::string prompt =
            recognized +
            "\n 请直接用一句话回答，不超过 25 个汉字。";

        // ------------------------------------------------
        // 12.4 非阻塞提交 LLMWorker
        // ------------------------------------------------

        const auto status =
            llm_worker.submit(
                id,
                prompt,
```

```
                // ----------------------------------------
                // LLM 流式文本回调
                // ----------------------------------------

                [&, prefix_printed](
                    std::uint64_t request_id,
                    const std::string& chunk)
                {
                    if (chunk.empty())
                    {
                        return;
                    }

                    std::lock_guard<std::mutex>
                        out(print_mutex);

                    if (!*prefix_printed)
                    {
                        std::cout
                            << "\n[LLM #"
                            << request_id
                            << "] ";

                        *prefix_printed = true;
                    }

                    std::cout
                        << chunk
                        << std::flush;
                },

                // ----------------------------------------
                // LLM 推理完成回调
                // ----------------------------------------
                //
                // 关键: 这里把 LLM 答案交给 TTSWorker。
                // ----------------------------------------

                [&, prefix_printed, question_at, ticket, trace](
                    std::uint64_t request_id,
                    const InferenceResult& result,
                    double llm_queue_ms)
                {
                    if(trace)trace-
>text(result.success?"answer_text":result.disposition==TaskDisposition::Completed?"request_e
rror":"request_terminal",result.success?result.text:result.error_message);
                    {
                        std::lock_guard<std::mutex>
                            out(print_mutex);

                        if (*prefix_printed)
                        {
                            std::cout << '\n';
                        }

                        std::cout
                            << "[LLM done #"
                            << request_id
                            << "] success="
                            << (result.success
                                ? "true"
```

```
                                : "false")
                            << " queue_ms="
                            << llm_queue_ms
                            << " first_text_ms="
                            << result.first_text_ms
                            << " latency_ms="
                            << result.latency_ms
                            << '\n';

                        if (!result.success)
                        {
                            std::cout
                                << (result.disposition==TaskDisposition::Completed ? "[LLM
error] " : "[LLM terminal] ")
                                << result.error_message
                                << '\n';
                        }
                    }

                    // -------------------------------------
                     // 12.5 判断 LLM 是否产生有效文本
                    // -------------------------------------

                    TTSSubmitStatus tts_status =
                        TTSSubmitStatus::NotRunning;

                    std::string tts_submit_error;

                    if (result.success && !result.text.empty() && ticket->cancelled())
                    {
                        { std::lock_guard<std::mutex> lock(state_mutex);
++downstream_cancelled; }
                        std::lock_guard<std::mutex> out(print_mutex);
                        std::cout << "[Pipeline downstream cancelled #" << request_id << "]
stage=tts\n";
                    }
                    const bool has_answer =
                        result.success &&
                        !result.text.empty() && !ticket->cancelled();

                    if (has_answer)
                    {
                        try
                        {
                            // ------------------------------
                             // 12.6 向 TTSWorker 提交答案
                            // ------------------------------

                            tts_status =
                                tts_worker.submit(
                                    request_id,
                                    result.text,

                                    // ----------------------
                                     // TTS 完成回调
                                    // ----------------------

                                    [&, question_at, ticket](
                                        std::uint64_t tts_id,
                                        const TTSWorkerResult& tts_result,
                                        double tts_queue_ms)
                                    {
```

```
                                        // 从 ASR 输出文本到
                                        // 播放任务结束的耗时。
                                    const double e2e_ms =
                                        std::chrono::duration<
                                            double,
                                            std::milli
                                        >(
                                            std::chrono::
                                                steady_clock::now()
                                            - question_at
                                        ).count();

                                    {
                                        std::lock_guard<
                                            std::mutex
                                        > out(print_mutex);

                                        std::cout
                                            << "[TTS done #"
                                            << tts_id
                                            << "] success="
                                            << (tts_result.success
                                                ? "true"
                                                : "false")
                                            << " queue_ms="
                                            << tts_queue_ms
                                            << " synth_ms="
                                            << tts_result.synthesis_ms
                                            << " audio_sec="
                                            << tts_result.audio_seconds
                                            << " playback_ms="
                                            << tts_result.playback_ms
                                            << " xruns="
                                            << tts_result.xrun_count
                                            << " asr_text_to_end_ms="
                                            << e2e_ms
                                            << '\n';

                                        if (!tts_result.success)
                                        {
                                            std::cout
                                                <<
(tts_result.disposition==TaskDisposition::Completed ? "[TTS error] " : "[TTS terminal] ")
                                                << tts_result.error_message
                                                << '\n';
                                        }
                                    }

                                    // ------------------
                                     // 更新 TTS 任务统计
                                    // ------------------

                                    {
                                        std::lock_guard<
                                            std::mutex
                                        > lock(state_mutex);

                                        ++tts_completed;

                                        if (tts_result.success)
                                        {
                                            ++tts_successful;
```

```
                                            }
                                            else if
(tts_result.disposition==TaskDisposition::Cancelled) ++tts_cancelled;
                                            else if
(tts_result.disposition==TaskDisposition::Expired) ++tts_expired;
                                            else ++tts_failed;
                                        }

                                         // 唤醒等待中的主线程。
                                        completed_cv.notify_all();
                                    }, ticket-
>options(std::chrono::milliseconds(tts_queue_ms), trace)
                                    );
                            }
                            catch (const std::exception& e)
                            {
                                tts_submit_error = e.what();
                            }
                            catch (...)
                            {
                                tts_submit_error =
                                    "unknown exception";
                            }

                            // ---------------------------------
                             // 12.7 处理 TTS 提交失败
                            // ---------------------------------

                            if (tts_status !=
                                TTSSubmitStatus::Accepted)
                            {
                                std::lock_guard<std::mutex>
                                    out(print_mutex);

                                std::cout
                                    << "[TTS rejected #"
                                    << request_id
                                    << "] ";

                                if (!tts_submit_error.empty())
                                {
                                    std::cout
                                        << tts_submit_error;
                                }
                                else if (
                                    tts_status ==
                                    TTSSubmitStatus::QueueFull)
                                {
                                    std::cout
                                        << "queue full";
                                }
                                else
                                {
                                    std::cout
                                        << "not running";
                                }

                                std::cout << '\n';
                            }
                        }
                        else if (result.success)
                        {
```

```
                        // LLM 成功但回答为空,
                        // 不能向 TTS 提交空文本。
                        std::lock_guard<std::mutex>
                            out(print_mutex);

                        std::cout
                            << "[TTS skipped #"
                            << request_id
                            << "] LLM returned empty answer\n";
                    }

                    // -------------------------------------
                    // 12.8 更新任务计数
                    // -------------------------------------
                    //
                    // 重要:
                    // 先提交 TTS, 再标记 LLM completed。
                    //
                    // 这样主线程等完所有 LLM 后,
                    // 就能确认不再产生新的 TTS 请求。
                    // -------------------------------------

                    {
                        std::lock_guard<std::mutex>
                            lock(state_mutex);

                        ++llm_completed;

                        if (result.success)
                        {
                            ++llm_successful;
                        }
                        else if (result.disposition==TaskDisposition::Cancelled)
++llm_cancelled;
                        else if (result.disposition==TaskDisposition::Expired) ++llm_expired;
                        else ++llm_failed;

                        if (result.success &&
                            result.text.empty())
                        {
                            ++tts_empty;
                        }

                        if (has_answer)
                        {
                            if (tts_status ==
                                TTSSubmitStatus::Accepted)
                            {
                                ++tts_accepted;
                            }
                            else
                            {
                                ++tts_rejected;
                            }
                        }
                    }

                    completed_cv.notify_all();
                }, ticket->options(std::chrono::milliseconds(inference_queue_ms), trace)
            );
```

```
        // ------------------------------------------------
        // 12.9 处理 LLM 任务提交结果
        // ------------------------------------------------

        if (status == LLMSubmitStatus::Accepted)
        {
            std::lock_guard<std::mutex>
                lock(state_mutex);

            ++llm_accepted;
        }
        else
        {
            // 只有 ASRWorker 修改该计数。
            {
                std::lock_guard<std::mutex> lock(state_mutex);
                if (status==LLMSubmitStatus::NotRunning && ticket->cancelled())
++cancelled_before_submit;
                else ++llm_rejected;
            }

            std::lock_guard<std::mutex>
                out(print_mutex);

            std::cout
                << (ticket->cancelled() ? "[LLM submission cancelled #" : "[LLM rejected #")
                << id
                << "] "
                << (status ==
                    LLMSubmitStatus::QueueFull
                    ? "queue full"
                    : "not running")
                << '\n';
        }
    };

    // ------------------------------------------------------
    // 13. 创建 ASRWorker 和 VadAudioWorker
    // ------------------------------------------------------

    ASRWorker consumer(
        audio_queue,
        asr,
        print_mutex,
        {},
        [&](const std::string& text,const AudioSegment&
segment){on_text(text,segment.trace);}
    );

    VadAudioWorker producer(
        capture_device,
        vad_model,
        audio_queue,
        print_mutex,
        record_seconds
    );

    const char* replay_path=std::getenv("STAGE5_REPLAY_FILE");
    const char* audio_replay_path=std::getenv("STAGE5_REPLAY_AUDIO_FILE");
    if(replay_path && audio_replay_path)throw std::invalid_argument("Text and audio replay
are mutually exclusive");
    const bool replay_mode=replay_path!=nullptr || audio_replay_path!=nullptr;
```

```
    if (!replay_mode)
    {
        std::cout
            << "[Listen] "
            << record_seconds
            << " seconds. "
            << (enable_vision ? "Ask ONE short voice question (text or camera).\n"
                              : "Ask ONE short text question and pause.\n")
            << std::flush;
    }
    else std::cout << (audio_replay_path?"[Replay ready] fixed presegmented audio; real ASR,
no microphone/VAD\n":"[Replay ready] controlled text; microphone/VAD/ASR not run\n") <<
std::flush;

    // ------------------------------------------------------
    // 14. 启动录音/VAD 和 ASR 工作线程
    // ------------------------------------------------------

    if (replay_mode)
    {
        std::ifstream input(audio_replay_path?audio_replay_path:replay_path);
        if (!input) throw std::runtime_error("Cannot open replay input");
        std::cout << (audio_replay_path?"[InputMode] replay_audio
REAL_ASR_NO_MICROPHONE_OR_VAD\n":"[InputMode] replay_only
NO_MICROPHONE_ASR_OR_VAD_INFERENCE\n") << std::flush;
        if(audio_replay_path)consumer.start();
        const auto replay_start=std::chrono::steady_clock::now();
        std::uint64_t audio_replay_sequence=1; // Producer-only; next_id belongs to the ASR
callback thread.
        std::string line;
        while (std::getline(input,line))
        {
            if (line.empty()) continue;
            const auto tab=line.find('\t');
            if (tab==std::string::npos) throw std::runtime_error("Invalid replay line");
            std::size_t used=0;
            const long delay=std::stol(line.substr(0,tab),&used);
            if (used!=tab || delay<0 || delay>3600000) throw std::runtime_error("Invalid
replay offset");
            const auto scheduled=replay_start+std::chrono::milliseconds(delay);
            while(stop_signals.received()==0 && std::chrono::steady_clock::now()<scheduled)
                std::this_thread::sleep_for(std::chrono::milliseconds(20));
            if (stop_signals.received()!=0) break;
            std::cout << "[Replay] scheduled_ms=" << delay << (audio_replay_path?"
source=fixed_audio\n":" source=controlled_text\n") << std::flush;
            if(audio_replay_path){
                AudioSegment segment;segment.sequence=audio_replay_sequence++;
                segment.trace=makeRequestTrace(segment.sequence);
                segment.samples=loadProfilingAudio(line.substr(tab+1));
                segment.enqueued_at=std::chrono::steady_clock::now();
                if(segment.trace)segment.trace-
>mark("offline_audio_ready",segment.enqueued_at);
                if(!audio_queue.push(std::move(segment)))throw std::runtime_error("Audio
replay queue closed");
            }else on_text(line.substr(tab+1));
        }
        audio_queue.close();
        if(audio_replay_path)consumer.join();
    }
    else
    {
    try
```

```
{
    // ASR 消费者先启动。
    consumer.start();

    // VAD 录音生产者后启动。
    producer.start();
}
catch (const std::exception& e)
{
    std::cerr
        << "[Start] Cannot start capture/ASR thread: "
        << e.what()
        << '\n';

    producer.requestStop();
    audio_queue.close();

    producer.join();
    consumer.join();

    llm_worker.requestStop();
    llm_worker.join();

    tts_worker.requestStop();
    tts_worker.join();

    llm.shutdown();

    return 1;
}

// ---------------------------------------------------------
// 15. 等待录音/VAD 和 ASR 完成
// ---------------------------------------------------------

bool stop_reported = false;
while (!producer.isFinished())
{
    if (stop_signals.received() != 0 || stop_signals.failed())
    {
        producer.requestStop();
        if (stop_mode == "cancel_pending")
        {
            admission.close(true);
            llm_worker.requestStop(StopMode::CancelPending);
        }
        if (!stop_reported)
        {
            std::lock_guard<std::mutex> out(print_mutex);
            std::cout << "[Shutdown] signal=" << stop_signals.received()
                      << " stopping capture; draining accepted work\n" << std::flush;
            stop_reported = true;
        }
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(20));
}
producer.join();

consumer.join();
}
asr.shutdown();
admission.close(false);
```

```
    std::cout << (replay_mode ? "[Shutdown] replay input closed; recognizer destroyed\n" :
"[Shutdown] capture/VAD and ASR joined; recognizer destroyed\n") << std::flush;

    // 此时 ASRWorker 已经退出，
    // 不会继续向 LLMWorker 提交新请求。

    // ------------------------------------------------------
    // 16. 通用 Worker 完成等待函数
    // ------------------------------------------------------
    //
    // 在等待期间，主线程每 2 秒输出一次进度。
    //
    // accepted / completed 都必须由 state_mutex 保护。
    // ------------------------------------------------------

    const auto wait_for_jobs = [&](const char* stage, auto& worker, int deadline_seconds) ->
bool
    {
        const auto
deadline=std::chrono::steady_clock::now()+std::chrono::seconds(deadline_seconds);
        while (!worker.waitIdleFor(std::chrono::milliseconds(100)))
        {
            if ((stop_signals.received()!=0 || stop_signals.failed()) &&
stop_mode=="cancel_pending")
            {
                admission.close(true);
                llm_worker.requestStop(StopMode::CancelPending);
            }
            if (std::chrono::steady_clock::now()>=deadline)
            {
                std::cerr << "[Timeout] " << stage << " did not become idle\n";
                return false;
            }
        }
        return true;
    };

    // ------------------------------------------------------
    // 17. 等待全部 LLM 任务完成
    // ------------------------------------------------------

    const bool llm_drained =
        wait_for_jobs(
            "LLM",
            llm_worker,
            180
        );

    if (!llm_drained)
    {
        std::cerr
            << "[Timeout] LLM tasks "
            << "did not finish in 180s\n";
    }

    // ------------------------------------------------------
    // 18. 停止 LLMWorker
    // ------------------------------------------------------
    //
    // 必须等 LLMWorker 完全停止，
    // 才能确认不会再向 TTSWorker 提交任务。
```

```
// ----------------------------------------------------

llm_worker.requestStop();
llm_worker.join();

// LLM 不再被工作线程访问后,
// 才允许释放 RKLLM Runtime。
llm.shutdown();

std::cout << "[Shutdown] LLM joined; Qwen runtime destroyed\n" << std::flush;

// ----------------------------------------------------
// 19. 等待全部 TTS 任务完成
// ----------------------------------------------------
//
// 必须先等待 accepted == completed,
// 然后再 requestStop。
//
// 否则会把队列中等待的 TTS 任务取消。
// ----------------------------------------------------

const bool tts_drained =
    wait_for_jobs(
        "TTS",
        tts_worker,
        600
    );

if (!tts_drained)
{
    std::cerr
        << "[Timeout] TTS tasks "
        << "did not finish in 600s\n";
}

// ----------------------------------------------------
// 20. 停止 TTSWorker
// ----------------------------------------------------

tts_worker.requestStop();
tts_worker.join();

std::cout << "[Shutdown] TTS joined; playback_xruns=" << playback.xrunCount()
          << '\n' << std::flush;

// 只有 Worker 退出以后,
// 才能安全关闭 ALSA 和 TTSService。
playback.close();
tts.shutdown();
if (camera) camera->stop();
if (vlm) vlm->shutdown();
std::cout << "[Shutdown] playback/TTS closed; Camera stopped; VLM destroyed\n"
          << "[Shutdown] complete signal=" << stop_signals.received() << '\n'
          << std::flush;

// ----------------------------------------------------
// 21. 输出整个语音交互链路的统计
// ----------------------------------------------------

{
    std::lock_guard<std::mutex>
```

```
    out(print_mutex);

std::cout
    << "\n=== ASR -> LLM -> TTS summary ===\n"

    << "VAD segments: "
    << producer.producedCount()
    << '\n'

    << "ASR recognized: "
    << consumer.processedCount()
    << '\n'

    << "Audio queue dropped: "
    << audio_queue.droppedCount()
    << '\n'

    << "LLM accepted/completed/"
    << "success/failed/rejected: "
    << llm_accepted
    << '/'
    << llm_completed
    << '/'
    << llm_successful
    << '/'
    << llm_failed
    << '/'
    << llm_rejected
    << '\n'

    << "TTS accepted/completed/"
    << "success/failed/rejected/empty: "
    << tts_accepted
    << '/'
    << tts_completed
    << '/'
    << tts_successful
    << '/'
    << tts_failed
    << '/'
    << tts_rejected
    << '/'
    << tts_empty
    << '\n'

    << "Skipped VISION: "
    << skipped_vision
    << '\n'
    << "VISION completed/success/failed: "
    << vision_completed << '/'
    << vision_successful << '/'
    << vision_failed << '\n';

if (!replay_mode && !producer.ok())
{
    std::cerr
        << "[VAD] "
        << producer.error()
        << '\n';
}

if ((!replay_mode || audio_replay_path) && !consumer.ok())
```

```
        {
            std::cerr
                << "[ASR] "
                << consumer.error()
                << '\n';
        }
    }

    const auto audio_stats=audio_queue.stats();
    std::cout << "[AudioQueueStats] pushed=" << audio_stats.pushed << " popped=" <<
audio_stats.popped
              << " dropped=" << audio_stats.dropped << " rejected_closed=" <<
audio_stats.rejected_closed
              << " peak=" << audio_stats.peak << " pending=" << audio_stats.pending << '\n';
    const auto inference_stats=llm_worker.stats();
    const auto tts_stats=tts_worker.stats();
    const auto admission_stats=admission.stats();
    printWorkerStats("inference", inference_stats);
    printWorkerStats("tts", tts_stats);
    std::cout << "[PipelineStats] dispatched=" << dispatch_calls << " accepted=" <<
admission_stats.accepted
              << " rejected=" << pipeline_rejected << " inflight=" <<
admission_stats.inflight
              << " peak=" << admission_stats.peak << " limit=" << max_inflight
              << " vision_accepted=" << vision_accepted << " vision_rejected=" <<
vision_rejected
              << " llm_cancelled=" << llm_cancelled << " llm_expired=" << llm_expired
              << " vision_cancelled=" << vision_cancelled << " vision_expired=" <<
vision_expired
              << " cancelled_before_submit=" << cancelled_before_submit
              << " downstream_cancelled=" << downstream_cancelled
              << " tts_cancelled=" << tts_cancelled << " tts_expired=" << tts_expired <<
'\n';

    // ------------------------------------------------------
    // 22. 最终验收条件
    // ------------------------------------------------------

    const bool passed =
        (replay_mode ? (!audio_replay_path || consumer.ok()) : (producer.ok() &&
consumer.ok())) && audio_queue.droppedCount()==0 &&
        llm_drained && tts_drained && !stop_signals.failed() &&
        (stop_signals.received()!=0 || llm_successful+vision_successful>0) &&
        inference_stats.accepted==inference_stats.completed &&
tts_stats.accepted==tts_stats.completed &&
        inference_stats.failed==0 && tts_stats.failed==0 &&
        inference_stats.callback_errors==0 && tts_stats.callback_errors==0 &&
        admission_stats.inflight==0 && llm_rejected==0 && vision_rejected==0 &&
tts_rejected==0;

    std::cout
        << (passed ? (stop_signals.received() != 0 ? "STOPPED\n" : "PASS\n") : "FAIL\n");

    return passed ? 0 : 1;
}
catch (const std::exception& e)
{
    std::cerr << "[Fatal] Voice pipeline exception: " << e.what() << '\n';
    return 1;
}
catch (...)
{
```

```
    std::cerr << "[Fatal] Voice pipeline: unknown exception\n";
    return 1;
}
```

5.5 Qt 与系统稳定性面试题及参考答案

1 为什么选择 QProcess 适配 CLI 而不是在窗口里重新创建模型

项目已经有完整的 Service、Worker、队列和退出机制。Qt 只负责控制和显示，另写一套初始化及推理会产生模型参数、设备所有权和信号行为差异。QProcess 复用原 CLI，在子进程中保持所有已验收线程，使模型加载与同步 SDK 天然离开 GUI 线程。代价是日志协议耦合和进程间观测延迟，需要独立测试解析、启动失败和安全关闭。

2 QProcess 的 started 是否意味着系统可接受语音

不是。它只说明外层进程启动，Qwen、SenseVoice、Kokoro、Camera 和 SmolVLM2 仍可能在加载或失败。必须分别显示加载状态，直到原程序打印 Listen 才提示用户说话。反过来，started 之后发生 exec 或模型失败也应处理 errorOccurred/finished 和真实退出码，不能一直停留在“运行中”。

3 关闭 Qt 窗口时如何避免后台访问已销毁控件

活跃时 closeEvent 先 ignore，保留接收者，再协作停止子进程。所有 connect 使用窗口或控制器作 context，只有 finished 之后才用 singleShot 关闭。后台模型只输出复制的字节，不持有窗口引用；页面不持有 Frame 或 SDK 裸指针。析构不能作为停止手段，尤其不能借 QProcess 析构中的强制终止宣称安全退出。

4 为什么 Qt 中不能直接调用 waitForFinished

GUI 线程阻塞后不能处理绘制、按键和正常状态更新，用户会把长合成误认为程序死锁。finished 信号允许事件循环继续运行；停止只发送协作信号并启动有限观察定时器。测试要同时验证后台真的完成和 GUI 心跳继续推进，单看窗口能显示一次不够。

5 如何在 Qt 程序处理 SIGINT 而不在 handler 调用 Qt

本工程复用 5.1 的信号阻塞与普通 sigtimedwait 线程，在创建 QApplication 和 SDK 线程前完成 pthread_sigmask。线程只写 atomic 状态，Qt 定时器在正常事件循环中读取并调用 close。这样没有异步 signal handler 操作 mutex、内存分配、QObject 或 join。子 CLI 也按同一协议处理信号，而不是依赖默认终止动作。

6 2 线程与 4 线程的短时加速为什么不能决定最终默认值

更多 ORT 线程可以降低一次合成墙钟时间，却增加 CPU 占用、温度以及对 ASR 的竞争。持续运行可能触发 Linux cooling、降频或队列策略，使整体体验变差。应使用相同固定输入、初始温度规则、原模型和相同监控比较长测，再依据实际稳定性决定默认值，保留显式 2/4 线程选择。不能为了保留好看数字隐藏过热或到期请求。

7 如何区分 DVFS 和热降频

ondemand 会随负载主动改变频率；瞬时频率低并不能证明温度造成限制。应同时记录 thermal trip、对应 cooling 设备类型与 cur_state、scaling_max_freq、当前频率、温度和负载。观察正 cooling state 及上限下降能提供 Linux 限制活动证据，但仍需按对应 CPU/GPU/NPU 设备解释，不能把 GPU cooling 状态当 CPU 降频。

8 什么证据支持“没有发现明显内存泄漏”

先排除加载和首次推理缓存，查看稳定区间 RSS 的时间序列、分箱中位数、斜率和波动范围；结合有界帧池/队列的源码所有权、终态计数、设备释放与多轮重启。RSS 增长也可能是 allocator 缓存，RSS 稳定也不能证明闭源 SDK 所有路径无泄漏。因此结论应限定工作负载和实际时长，不应只说“没 OOM 就是无泄漏”。

9 为什么压力测试中的推理成功率和播放交付率要分开

推理可能全部成功，但 TTS 任务在原等待期限下过期、因背压拒绝或停止时取消。模型成功数除以实际执行数，与最终播放数除以输入请求数，是不同指标。必须报告所有输入的终态、拒绝/到期/取消计数及分母，不能只保留成功完成项来宣传系统 100%成功。

10 固定音频回放能替代真实麦克风验收吗

不能。它可以复现 SenseVoice 输入和后续调度，便于比较 CPU 竞争与热状态，但预分段音频没有 USB 采集、VAD 端点、XRUN 和声学环境。固定图像也没有实时 Frame Age。最终必须另做真实麦克风、Camera 及用户耳机确认，并且把 ALSA 提交时间与真实声学延迟区分开。

11 Fake 故障测试为什么仍然重要

真正断开设备或让 SDK 崩溃往往不可控，可能影响已有系统。Fake 可以确定性地注入返回 false、标准/未知异常、无效 PCM、回调异常和精确并发交错，检查错误计数、唤醒、join 和后续恢复。但 Fake 不证明 SDK 内部失败安全，板端无效设备名等非破坏性测试与正常硬件回归仍需单列，不能把两者合为“真实设备恢复全部通过”。

12 如何定义可复现的最终交付

冻结源码和历史版本，保留完整 diff、SHA 清单、原始命令/退出码、模型与 SDK 真实路径、CSV/JSON、输入条件、统计方法和截图来源。构建目录和测试 case 独占，普通用户运行，不覆写稳定工程。手册中的函数、参数和结论必须与最终源码和日志一致，Word 需要真实渲染检查；最终用户审核前不能把隔离版直接升级覆盖主工程。

8 构建运行复现和重要命令

正式命令章节将在最终 ARM64、Qt 与长测结束后，从 command.json 和原始退出码生成；当前没有补造输出。

所有源码/构建/报告目录必须新建且不存在。源码包先核对 SHA 和安全相对路径，普通 cat 用户构建；磁盘低于 500MiB 停止，不删除历史文件腾空间。主工程、模型、SDK、驱动、设备树和内核保持只读。复现时使用新 label/case，不直接重跑会覆盖同名证据的命令。

9 最终验收边界和局限

最终版本不得自动覆盖主稳定工程。Windows CTest、Fake/Mock、真实 SDK 固定输入、自然麦克风/耳机与 GUI 交互分别计数；同一请求 ID 必须带 case 或会话名。热对照与最终 GUI 长测标明源码版本、时长、输入间隔、冷却条件和保护停止。原始失败保留，若 4 线程触发测试护栏必须如实选择默认配置并重测，不能把强杀当优雅退出。

未测量或未实现：真实声学输出延迟、精确 token 粒度 TTFT、可信 NPU 利用率/带宽/功耗、完整 SDK 内部故障注入、DMA-BUF 端到端零拷贝、成功 RGA 生产路径、AEC/PTT/语音打断和任意环境下的永久稳定性。有限 RSS 稳定不是所有代码无泄漏证明，Linux trip 不等于芯片额定安全阈值。

10 简历项目描述与面试讲述

项目描述可写为：在 RK3576 端侧集成 USB 语音采集、Silero VAD、SenseVoice、Qwen2.5、IMX415、RKNN 加 SmolVLM2、Kokoro 与 ALSA，完成普通/视觉语音交互。使用 C++17 RAII、条件变量、有界 Worker 及跨阶段额度管理请求寿命，提供协作 SIGINT/SIGTERM 和可选 Qt 异步控制页面。

可核验性能成果：将同步 VLM 从 ASR 回调解耦，受控 VISION 提交由 1737.730ms 变为 0.034758ms，此为调度改善；缓存 CPU 预处理在 n12 比较中平均下降 24.67%；固定中英文各 n30 的 4 线程 TTS 合成平均耗时分别下降 30.42%与 29.74%，同时记录近双倍 CPU 和温度代价。持续稳定性与最终默认线程数必须附 5.5 真实条件，不能只写“全链路提速 30%”。

面试叙述顺序：先画实际线程和所有权图，解释 ASR 阻塞与回调寿命问题；再给出有界队列和取消/排空契约；用固定输入、样本量和原始日志说明局部优化；最后主动说明 RGA 失败、没有零拷贝、TTS 仍非实时以及热稳定性的取舍。不要宣称训练了这些模型、修改了驱动或实现了尚无证据的功能。

11 结果索引和源码一致性

本手册正文是 Word 与 Markdown 的共同来源。完整逐文件源码附录、Git 格式 diff、逐文件 SHA、补丁 apply 检查和字节 roundtrip、原始终端日志、CSV/JSON、真实 GUI 截图、输入及环境清单随最终归档交付。所选关键函数代码从当前真实文件抽取，文件和片段 SHA 记录在 `reports/stage5_05_local/manual_source_manifest.json`，不使用省略号代替函数体。